ASSUMPTION UNIVERSITY

Vincent Mary School of Engineering, Science and Technology

CSX4104/ITX4104 SOFTWARE TESTING

Section – 541

Submitted to

Asst. Prof. Dr. Darun Kesrarat

Term Project Report

By

Sai Aik Sei Mouk (6520057)

Sai Thaw Zin Aung (6520152)

Moe Myint Mo San (6530173)

Sokty Heng (6530353)

Somavatey Heng (6530354)

Semester 1/2025

Abstract

This project focuses on the software testing of a registration system aimed at automating and optimizing administrative tasks such as class scheduling and attendance tracking. The system, a web application, simplifies the scheduling process by removing manual verification of classroom and teacher availability, while also enabling teachers to record attendance and provide feedback digitally. The goal of the software testing phase is to ensure that the system functions as intended across all components—scheduling and attendance —by validating its usability, performance, security, and integration. Comprehensive testing will be conducted to identify issues, ensure seamless user experience, and confirm that the system provides a reliable, scalable, and error-free foundation for future enhancements.

Table of Content

Table of Figures

Introduction

In an effort to streamline academic operations and reduce manual workload, this project presents the development of an integrated registration system tailored for Kiddee Lab. The system is designed to simplify the class scheduling process for registrars by automating conflict checks related to classroom availability and teacher schedules. By transitioning from traditional, paper-based processes to a centralized digital platform, the system not only facilitates efficient scheduling but also empowers teachers to digitally manage attendance and feedback. This solution lays the groundwork for a unified system architecture that can support future expansions, ultimately aiming to enhance operational efficiency and data management within the organization.

Company detail

Kiddee Lab (KDL) is the first Robotics and Coding School in Thailand with a curriculum accredited by the Ministry of Education. They have more than 50 courses for kids and young adults (5-18 years old). Students are currently learning Line Coding, Scratch programming, 3D Modelling/Design, Minecraft Education, Python, Arduino, IoT, Robotics, Robot Arm, etc. Each course is designed to be both challenging and fun.

Kiddee Lab's curriculum revolves around STEAM (Science, Technology, Engineering, Arts and Mathematics) Education. Apart from technical knowledge, Kiddee Lab also trains their students to have practical skills like teamwork, creativity and critical thinking for tackling problems that they may face in everyday life.

Functional Scope

System objectives

- A system where the registrar can create class schedules without spending time to find out whether the classroom is vacant or the teacher who can teach the subject is available.
- A system where teachers can record student attendance and feedback.
- A system where the registrar can audit feedback before relaying to parents.
- A centralized and foundation system which can be used in future projects.

End-users

- Manager
- Registrar
- Teacher

System Requirements

Course & Users Management

- mgt001 The system shall allow the manager to create new courses.
- mgt002 The system shall provide the manager with the ability to create new registrar.

Student Management

- std001 The system shall provide the registrar with the ability to register new students.
- std002 The system shall provide the registrar with the ability to register new parents.

Scheduling

1. sked001 The system shall provide the register with the ability to view existing schedules as well as create new schedules.
2. sked002 The system shall be able to show warning if the schedule has conflict.
3. sked003 The system shall support the following class types:
    a. 12 times fixed

b. 12 times check

c. Camp class

4. sked004 The system shall provide the registrar with the ability to add package course to a student.

Attendance and Feedback Management

- atn001 The system shall provide the teacher with the ability to log student attendance and feedback.
- atn002 The system shall provide the registrar with the ability to edit student feedback.

Daily Overview

- util001 The system shall display the classes scheduled for today.
- util002 The system shall display the list of students attending today's class.

Security

- sec001 The system shall implement role base access control for authentication.

Scope and Limitation

The developed system is a web-based learning management system designed primarily to assist registrars in scheduling classes without conflicts and to enable teachers to record student attendance and feedback efficiently. The testing scope of the system is for the following key features:

- **Schedule Conflict Warning:** Allows registrars to schedule classes while preventing overlaps in the availability of teachers, rooms, and students.
- **Schedule Display:** Provides detailed views of student and teacher schedules to facilitate planning and management.
- **Feedback Recording:** Enables teachers to submit feedback for students, which can subsequently be reviewed and approved by registrars before being finalized.

Our project scope is limited to the functionalities listed above and does not include other administrative tasks such as invoicing, receipt generation, payroll, detailed academic performance tracking, direct online payment processing, parental access to view their child's information or performance, class cancellation requests, or parent viewing student timetables.

Code for functional Requirement

Course Management

1. **Function to create course**

```
function createCourse(courseTitle, courseDescription, ageRange, medium):
    if courseTitle is empty OR courseDescription is empty OR ageRange is empty
OR medium is empty:
        warn "All fields are required"
        return
    course = new Course(courseTitle, courseDescription, ageRange, medium)
    save course to database
    return course
```

2. **Function to add course package to students**

```
function addCourseToStudent(studentId, courseId, classType, scheduleDetails):
    if classType == "12 times fixed":
        generate 12 schedules with fixed days/times
    else If classType == "12 times check":
        generate 12 schedules with TBD fields
    else If classType == "Camp class":
        generate camp schedule based on input dates
    else:
          Warn “Invalid class”
          Return
    for each schedule:
        if room, student or teacher conflict:
            update conflictType
       else:
```

```
            conflictType = “Unknown”
        save schedule
    link course to student
    return success
```

User Management

1. **Function to create registrar**

```
function createRegistrar(name, email, password, photo):
    if name is empty OR email is empty OR password is empty:
        warn "Name, email, and password are required"
        return
    registrar = new Registrar(name, email, password, photo)
    save registrar to database
    return registrar
```

2. **Function to create teachers**

```
function createTeacher(name, email, password, photo):
    requiredFields = [photo, name, email, contact_no, line_id, address,
password, confirm_password, courses]
    for field in RequiredFields:
        if field is empty:
            warn field + " is required"
            return
    if password != confirm_password:
        warn "Passwords do not match"
        return
    teacher = new Teacher(photo, name, email, contact_no, line_id, address,
password, courses)
```

save teacher to database
return teacher

Student Management

1. **Function to register new student**

function registerNewStudent(name, nickname, school, nationalID, dob, gender, phone, parent, allergic, doNotEat, allowPhotoAds):
    if name is empty OR nickname is empty OR school is empty OR nationalID is empty OR dob is empty OR gender is empty:
        warn "Required fields must be filled"
        return "Registration failed"
    if nationalID exists in database:
        warn "Student already registered"
        return "Registration failed"
    studentID = generateStudentID()
    student = new Student(studentID, name, nickname, school, nationalID, dob, gender, phone, parent, allergic, doNotEat, allowPhotoAds)
    save student to database
    return "Registration successful"

2. **Function to register new parent**

function registerNewParent(name, email, contactNo, lineID, address):
    if name is empty OR email is empty OR contactNo is empty OR address is empty:
        warn "Required fields must be filled"
        return "Registration failed"
    if email is not in valid format:
        warn "Invalid email address"
        return "Registration failed"

```
    if email exists in database:
        warn "Parent already registered with this email"
        return "Registration failed"
    parentID = generateParentID()
    parent = new Parent(parentID, name, email, contactNo, lineID, address)
    save parent to database
    return "Registration successful"
```

Scheduling

1. **SKED001: Function to View and Create Schedules**

```
function createSchedule(date, startTime, endTime, additionalData,
scheduleRepository) {
    if (date is empty OR startTime is empty OR endTime is empty) {
        return "Missing required fields: date, startTime, endTime";
    }
    schedule = new Schedule(date, startTime, endTime, additionalData);
    scheduleRepository.save(schedule);
    return schedule;
}
```

2. **SKED002: Function to show Schedule Conflict Warning**

```
function checkForConflicts(scheduleData, scheduleRepository,
excludeScheduleId) {
    date = scheduleData.date;
    startTime = scheduleData.startTime;
    endTime = scheduleData.endTime;
    room = scheduleData.room;
```

```
    teacherId = scheduleData.teacherId;
    studentId = scheduleData.studentId;

    existingSchedule = scheduleRepository.findOverlapping(date, startTime,
endTime, room, teacherId, studentId, excludeScheduleId);

    if (existingSchedule is empty) {
        return null;
    }

    conflictDetails = buildConflictDetails(existingSchedule, scheduleData);
    return conflictDetails;
}

function buildConflictDetails(existing, proposed) {
    roomConflict = existing.room == proposed.room;
    teacherConflict = existing.teacherId == proposed.teacherId;
    studentConflict = existing.studentId == proposed.studentId;

    if (roomConflict AND teacherConflict AND studentConflict) {
        conflictType = "all";
    } else if (roomConflict AND teacherConflict) {
        conflictType = "room_teacher";
    } else if (roomConflict) {
        conflictType = "room";
    } else if (teacherConflict) {
        conflictType = "teacher";
    } else if (studentConflict) {
        conflictType = "student";
    } else {
        conflictType = "unknown";
```

```
    }
    conflictDetails = new ConflictDetails(conflictType, existing.room,
existing.startTime, existing.endTime);
    return conflictDetails;
}
```

3. **SKED003: Function to support Class Types**

```
function validateClassTypeData(courseType, classData) {
    classMode = courseType.classMode;

    if (classMode == "12 times fixed") {
        if (classData.fixedDays is empty) {
            return "Select days for fixed schedule";
        }
        if (classData.fixedStartTime is empty OR classData.fixedEndTime is
empty) {
            return "Set start and end times";
        }
    }

    if (classMode == "Camp class") {
        if (classData.campDates.length != 5) {
            return "Select 5 camp dates";
        }
    }
    return "Valid";
}
```

4. **SKED004: Function to add Package Course to Student**

```
function addPackageToStudent(studentId, packageType,
createStudentPackageFn) {
    if (studentId is empty) {
        return "Please select a student";
    }
    if (packageType is empty) {
        return "Please select a package";
    }

    result = createStudentPackageFn(studentId, packageType);
    return result;
}

function createStudentPackage(studentId, packageType, studentRepository,
packageRepository) {
    student = studentRepository.findById(studentId);
    if (student is empty) {
        return "Student not found";
    }

    remainingClasses = calculateRemainingClasses(packageType);
    packageRecord = new PackageRecord(studentId, packageType,
remainingClasses);
    packageRepository.save(packageRecord);
    return "Package added successfully";
}

function calculateRemainingClasses(packageType) {
    if (packageType contains "2") {
        return 2;
    }
```

```
    if (packageType contains "4") {
        return 4;
    }
    if (packageType contains "10") {
        return 10;
    }
    return 0;
}
```

Attendance and Feedback Management

1. **atn001 - Function to log student attendance and feedback**

```
function logAttendanceAndFeedback(scheduleId, studentId, attendanceStatus,
feedbackText):
    schedule = getSchedule(scheduleId, studentId)
    schedule.attendance_status = attendanceStatus
    if feedbackText is not empty:
        feedback = new feedback(scheduleId, feedbackText, “Pending”)
        save feedback to database
    save schedule to database
    return
```

2. **atn002 - Function to edit student feedback**

```
function editFeedback(scheduleId, feedbackText):
    feedback =  getFeedback(scheduleId)
    if feedback does not exist:
          feedback = new Feedback(scheduleId, feedbackText, “Pending”)
    else:
          feedback. feedback = feedbackText
          feedback.status = “Pending”
```

save feedback to database

return feedback

Daily Overview

1. **util001- Function to display the classes scheduled for today.**

```
function showTodaySchedule():
    todayDate = current date
    classes = getTodaySchedule()
    if classes is empty:
        return "No classes scheduled for today"
    display classes in timeline view
    for each class in classes:
        show class.courseTitle, class.startTime to class.endTime, class.teacher,
class.room
    return "Classes displayed"
```

2. **util002 - Function to display the list of students attending today's class.**

```
function showAttendingStudents(classId):
    classInfo = find schedule by classId
    students = find all students linked to classId
    display classInfo (courseTitle, time, teacher, room)
    display student list in table
    for each student in students:
        show student.profile, student.name, classInfo.teacher, classInfo.room,
student.attendanceStatus, student.remark, student.feedback
        if conflict exists:
        show conflictWarning
    return "Student list displayed"
```

Security

1. **sec001- Function to implement role base access control for authentication**

function login(email, password, role):
  if email is empty:
    warn "Email is required"
    return "Login failed"
  if password is empty:
    warn "Password is required"
    return "Login failed"
  if role is empty:
    warn "Please select a role"
    return "Login failed"
  if email is not in valid format:
    warn "Enter a valid email address"
    return "Login failed"
  user = find user by email
  if user does not exist OR user.password != password OR user.role != role:
    return "Login failed"
  if role == "Administrator":
    allow access to Today, Schedule, Courses, Students, Teachers, Parents, Registrars, Fees & Discounts, Statistics, Feedback, Create Invoice, Invoices & Receipts
    redirect to Admin Dashboard
  else if role == "Registrar":
    allow access to Today, Schedule, Courses, Students, Teachers, Parents, Feedback, Create Invoice, Invoices & Receipts
  else if role == "Teacher":
    allow access to Today, My Schedule, Attendance, Feedback

```
else:
    return "Login failed"
show "Login successful!"
return "Login success"
```

System Control Flow

Course Management

1.Course Creation (mgt001)

- The manager can create a new course by filling in details such as course name, description, age range, and medium.

State Transition:

- If all required fields are valid the system saves the course and updates the course list.
- If validation fails the system remains in the form with error messages.

Events:

- Manager submits valid details and the course is created.
- Manager leaves fields blank, and the system shows an error.
- Manager cancels and the system returns to the course list without saving.

User Management

1. Registrar Creation (mgt002)

- The manager can create new registrar accounts.

State Transition:

- If all required details are valid the registrar is created and saved in the system.
- If validation fails the system shows an error and stays in the form.

Events:

- Manager submits valid details and the registrar account is created.
- Manager submits missing or invalid details and the system shows an error message.
- Manager cancels creation and the system returns to the registrar list.

Student Management

1.Student Registration (std001)

- The registrar registers new students by entering personal details and national ID.

State Transition:

- If all required fields are valid and the national ID is unique the student is created with an auto-generated student ID.
- If validation fails or a duplicate ID is found the system stays in the form.

Events:

- Registrar submits valid details and the student is registered.

- Registrar submits missing details or a duplicate ID and the system shows an error.
- Registrar cancels and the system returns to the student list.

2. Parent Registration (std002)

- The registrar registers parents by entering details such as name, email, phone, and address.

State Transition:

- If all fields are valid and the email is unique the parent record is created.
- If validation fails the system remains in the form.

Events:

- Registrar submits valid details and the parent is registered.
- Registrar enters duplicate email or leaves fields empty and the system shows an error message.
- Registrar cancels and the system returns to the parent list.

Scheduling

1. View and Create Schedule (sked001)

- The registrar can view existing schedules and update with class details.

State Transition:

- If the schedule details are valid the schedule is saved and displayed in the list.
- If inputs are missing the system shows an error and remains in the form.

Events:

- Registrar creates a valid schedule and it is added.
- Registrar leaves fields blank and the system shows an error.
- Registrar cancels and the system returns to the schedule list.

2. Conflict Checking (sked002)

- When creating a schedule the system checks for conflicts such as room, teacher, or student overlap.

State Transition:

- If a conflict exists the system shows a warning and remains in the schedule form.

Events:

- Conflict detected and the system displays a warning.

3. Class Type Support (sked003)

- The system supports multiple class types: 12 times fixed, 12 times check, and camp class.

State Transition:

- Depending on the class type the system generates schedule rows automatically.
- If required inputs are missing the system stays in the form.

Events:

- Registrar selects 12 times fixed and the system generates fixed dates and time.
- Registrar selects 12 times check and the system creates 12 sessions to be determined.
- Registrar selects camp class and the system creates camp sessions.

4. Add Package Course to Student (sked004)

- The registrar can assign a course package to a student.

State Transition:

- If a valid student and package are chosen the system updates the student record.
- If inputs are invalid the system remains in the form.

Events:

- Registrar selects a valid student and package and the package is added.
- Registrar leaves inputs empty and the system shows an error.
- Registrar cancels and the system returns to the student profile.

Attendance and Feedback Management

1. Log Attendance and Feedback (atn001)

- Teachers can mark attendance and submit feedback for students.

State Transition:

- The system updates the attendance status and stores the feedback.

Events:

- The teacher marks attendance to update student status and submits feedback to save the record.
- Teacher skips action and no changes are made.

2. Edit Student Feedback (atn002)

- The registrar can edit or approve student feedback.

State Transition:

- Updated feedback replaces the old record in the system.
- If no record is found the system display no feedback to approve message.

Events:

- Registrar edits valid feedback and the update is saved.
- Registrar cancels and the system returns to the feedback list.

Daily Overview

1. Display Today's Classes (util001)

- The system shows classes scheduled for the current day.

State Transition:

- If classes exist the system displays the schedule timeline.
- If none exist the system shows a message stating there are no classes today.

Events:

- User opens Today's Schedule and the classes are displayed.
- No classes are found and an empty message is shown.

2. Display Attending Students (util002)

- The system lists all students attending today's class with a specific course.

State Transition:

- If students exist in the class, the system displays their names, profiles, and attendance status.

Events:

- User selects a class and the attending students are displayed.

Security

1. Login and Role Authentication (sec001)

- The system starts with the login screen where users enter email, password, and select their role (Manager, Registrar, Teacher).
- The system validates the credentials and grants access to the appropriate dashboard.

State Transition:

- The system remains in the login state until valid credentials are entered.
- On success the system transitions to the correct role dashboard.

Events:

- User enters valid credentials and gains access.
- User enters invalid credentials and the system shows an error message.
- User exits without logging in.

Unit Testing

Unit testing is a white box testing method where developers, with full access to the source code, design tests to validate the internal structure, logic, and implementation of individual functions. Unlike black box testing, which focuses only on input and output, unit testing examines internal logic paths, code coverage, algorithms, data structures, and boundary conditions. The goal is to ensure all branches and statements are executed correctly, internal state changes behave as expected, and edge cases are properly handled. Testing is conducted manually by developers who analyze the code, create specific test cases for each execution path, directly call functions with predetermined inputs, and verify both the outputs and internal states to ensure comprehensive coverage of all code branches and conditions.

1. **Create Course**

Objective: Test course creation with valid and invalid input

Steps:

1. Call createCourse with empty title
2. Call createCourse with all valid fields

Expected Result:

- Step 1: Function returns None and warns
- Step 2: Function returns course object with title 'Robotics'

2. **Add Course to students**

Objective: Test adding course package to student

Steps:

1. Call addCourseToStudent with valid student and course
2. Call addCourseToStudent with missing studentId

Expected Result:

- Step 1: Returns 'success
- Step 2: Returns None (error)

3. Create Registrar

Objective: Test registrar creation with valid and invalid input

Steps:

1. Call createRegistrar with empty name
2. Call createRegistrar with all valid fields

Expected Result:

- Step 1: Returns None (error)
- Step 2: Returns registrar object with name 'name'

4. Create Students

Objective: Verify student creation handles valid and invalid details.

Steps:

1. Call createStudent with missing name.
2. Call createStudent with all valid fields.

Expected Result:

- Step 1: Returns None (error).

- Step 2: Returns a student object with name 'John'.

5. Create Parents

Objective: Verify parent creation handles valid and invalid details.

Steps:

1. Call createParent with missing name.
2. Call createParent with all valid fields.

Expected Result:

- Step 1: Returns None (error).
- Step 2: Returns parent object with name 'Parent'.

6. Create Teachers

Objective: Verify teacher creation works for valid input and password mismatch.

Steps:

1. Call createTeacher with matching passwords.
2. Call createTeacher with mismatched passwords.

Expected Result:

- Step 1: Returns teacher object.
- Step 2: Returns None (error).

7. Generate StudentID

Objective: Verify student ID is generated correctly.

Steps:

1. Call generateStudentID.

Expected Result:

- Returns a formatted student ID.

8. Save Student to Database

Objective: Verify saving student to database stores correct data.

Steps:

1. Call saveStudentToDatabase with all valid fields.

Expected Result:

- Returns student object with correct studentID and name.

Control Flow Testing

Control flow testing is a white box testing technique that examines the execution paths through a program by analyzing its control structures such as decisions, loops, and branches. This method focuses on ensuring that all possible execution paths are tested by creating test cases that exercise different combinations of conditions, covering all branches (true/false outcomes), statements, and decision points within the code. Testing is conducted manually by developers who analyze control flow graphs, identify all possible execution paths, design test cases to achieve complete path coverage, and verify that each branch and condition is properly executed. The goal is to ensure comprehensive coverage of all control flow scenarios including edge cases, exception handling paths, and boundary conditions to validate the program's logical correctness.

Course Management

1.Function to create course

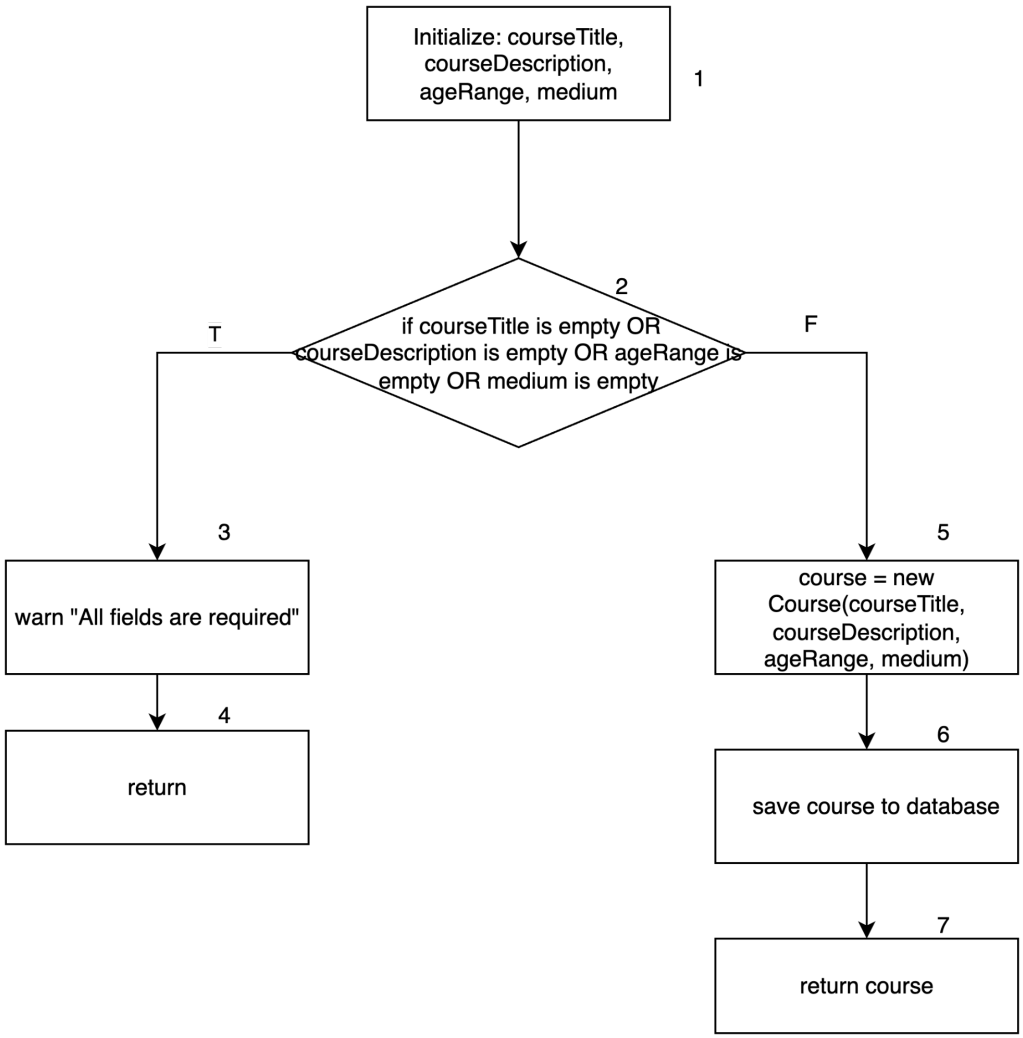


Figure 1 Function to create course

Statement Coverage

1. Path: 1 → 2 (T) → 3 → 4

Test Case:

courseTitle = "", courseDescription = "desc", ageRange = "5-10", medium = "online"

Expected: Warning message, function returns without creating course

2. Path: 1 → 2 (F) → 5 → 6 → 7

Test Case:

courseTitle = "Robotics", courseDescription = "Learn robotics", ageRange = "5-10", medium = "online"

Expected: Course object created, saved to database, and returned

2. Function to add Course To Student

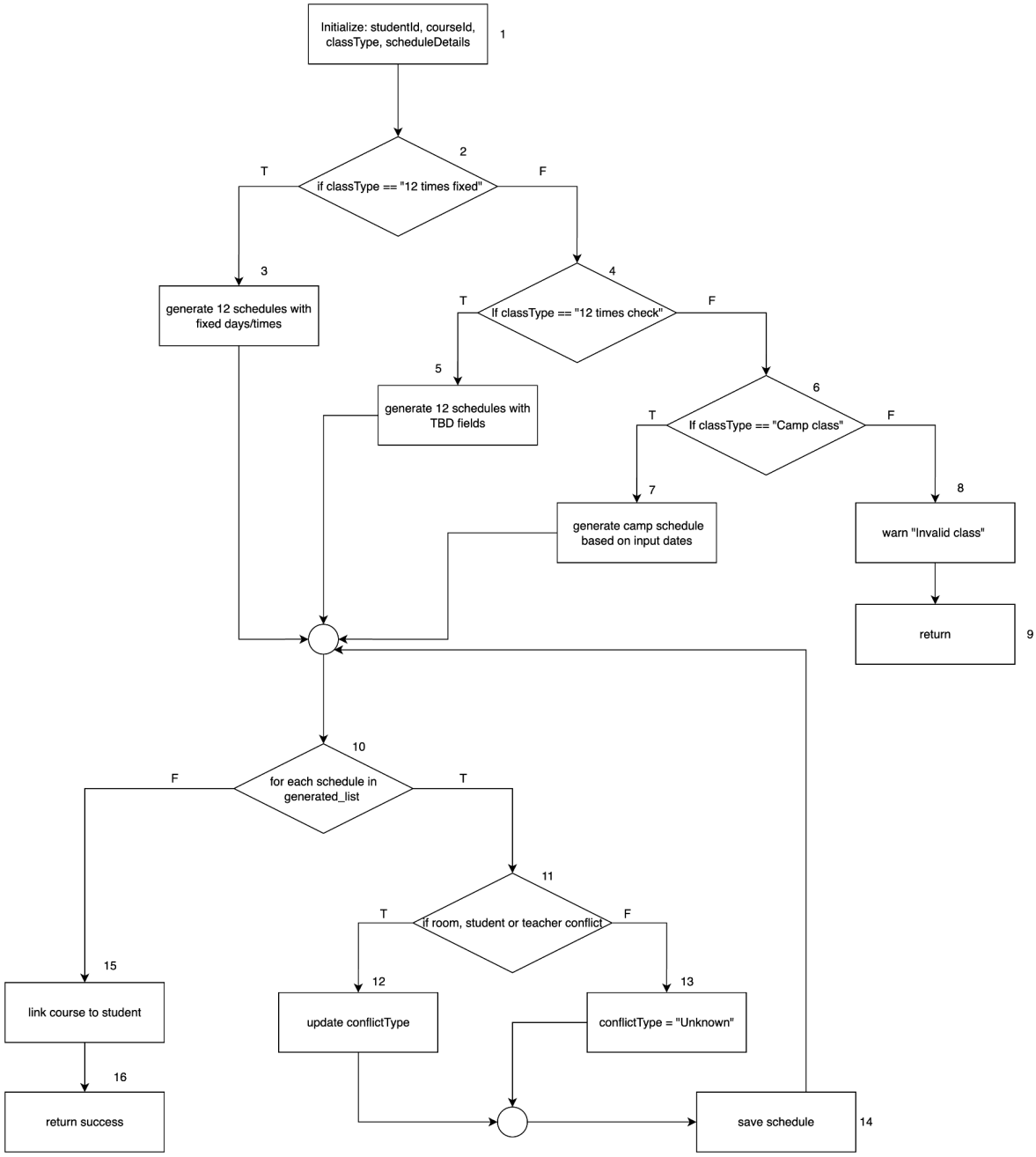


Figure 2 Function to add course to student

Statement Coverage

1. Path 1-2(T)-3-10(T)-11(T)-12-13-14-15-16 (12 times fixed with conflicts)

Input:

studentId="S001", courseId="C001", classType="12 times fixed",

scheduleDetails={days:["Mon","Wed"], startTime:"09:00", endTime:"11:00"}

Expected: 12 schedules generated with fixed times; conflict detected → conflictType updated → schedule saved → linked to student → return success.

2. Path 1-2(T)-3-10(T)-11(F)-14-15-16 (12 times fixed without conflicts)

Input:

studentId="S002", courseId="C002", classType="12 times fixed",

scheduleDetails={days:["Tue","Thu"], startTime:"13:00", endTime:"15:00"}

Expected: 12 schedules generated with fixed times; no conflicts → schedule saved → course linked → return success.

3.Path 1-2(F)-4(T)-5-10(T)-11(T)-12-13-14-15-16 (12 times check with conflicts)

Input:

studentId="S003", courseId="C003", classType="12 times check",

scheduleDetails={}

Expected: 12 schedules generated with TBD times; conflict detected → conflictType updated → schedule saved → linked to student → return success.

4. Path 1-2(F)-4(T)-5-10(T)-11(F)-14-15-16 (12 times check without conflicts)

Input:

studentId="S004", courseId="C004", classType="12 times check",

scheduleDetails={}

Expected: 12 schedules generated with TBD times; no conflicts → schedule saved → linked to student → return success.

5. Path 1-2(F)-4(F)-6(T)-7-10(T)-11(T)-12-13-14-15-16 (Camp class with conflicts)

Input:

studentId="S005", courseId="C005", classType="Camp class",

scheduleDetails={dates:["2024-01-15","2024-01-16"]}

Expected: Camp schedules generated; conflict detected → conflictType updated → schedule saved → linked to student → return success.

6. Path 1-2(F)-4(F)-6(T)-7-10(T)-11(F)-14-15-16 (Camp class without conflicts)

Input:

studentId="S006", courseId="C006", classType="Camp class",

scheduleDetails={dates:["2024-07-15","2024-07-16","2024-07-17"], duration:"half-day"}

Expected: Camp schedules generated; no conflicts → schedule saved → linked to student → return success.

7. Path 1-2(F)-4(F)-6(F)-8-9 (Invalid class type)

Input:

studentId="S007", courseId="C007", classType="invalid type", scheduleDetails={}

Expected: Warning “Invalid class” shown; function returns without further processing.

User Management

1. Function to create Registrar

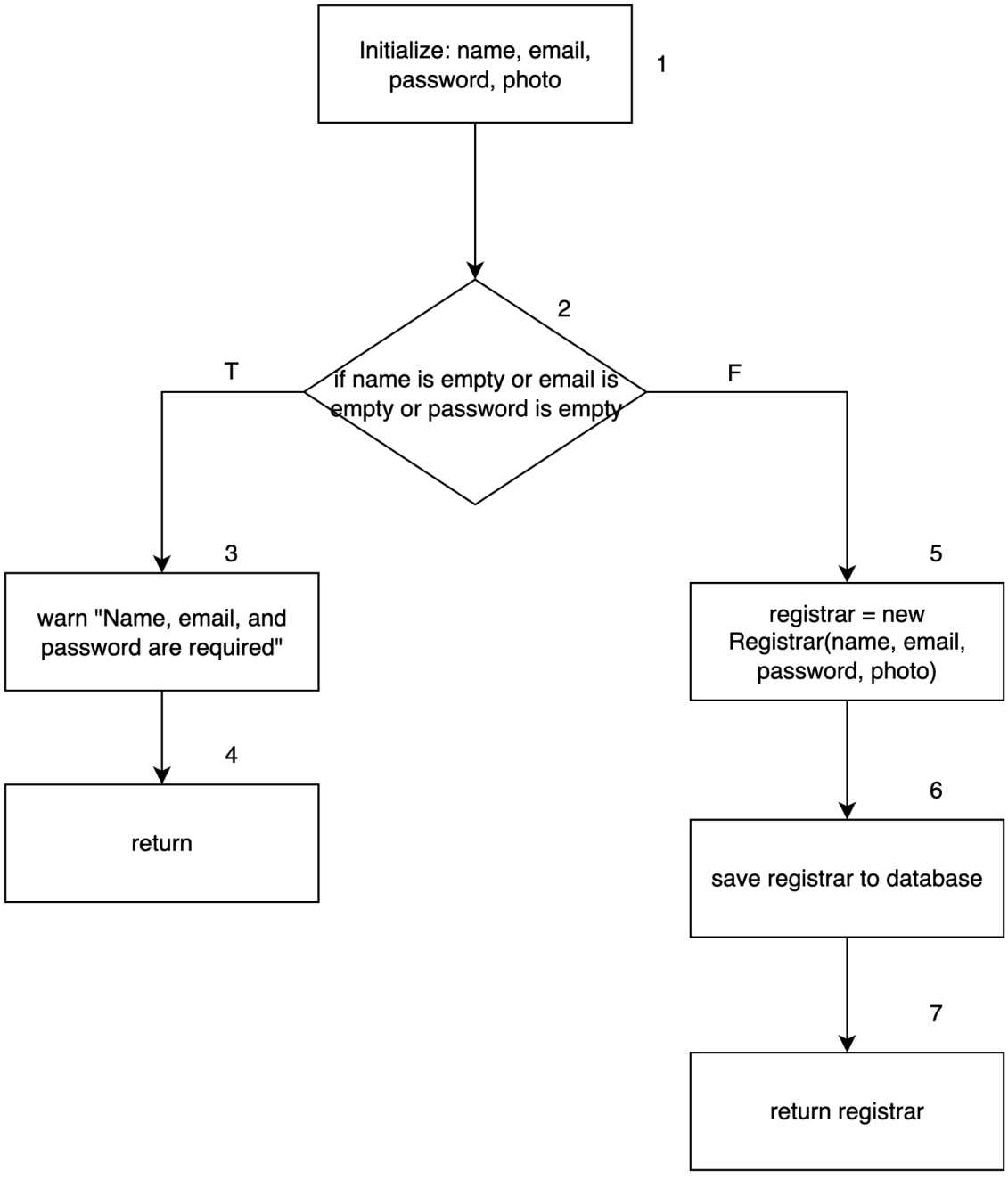


Figure 3 Function to create Registrar

Statement Coverage

1. Path 1-2(T)-3-4 Test Case 1a - Empty Name:

Input: name="", email="admin@school.com", password="securepass123", photo="admin.jpg"

Expected Result: Validation fails, warning displayed, early return

Test Case 1b - Empty Email:

Input: name="John Admin", email="", password="securepass123", photo="admin.jpg"

Expected Result: Validation fails, warning displayed, early return

Test Case 1c - Empty Password:

Input: name="John Admin", email="admin@school.com", password="", photo="admin.jpg"

Expected Result: Validation fails, warning displayed, early return

Test Case 1d – Multiple Empty Fields

Input: name="", email="", password="", photo="admin.jpg"

Expected: Validation fails, warning displayed, early return

2. Path 1-2(F)-5-6-7 (Successful Registration)

Test Case 2a – Complete Valid Data

Input: name="Sarah Johnson", email="sarah.johnson@school.com", password="admin2024!", photo="sarah_profile.jpg"

Expected: Registrar object created → saved in DB → returned successfully.

Test Case 2b – Valid Data Without Photo

Input: name="Michael Brown", email="michael.brown@school.com", password="secure123", photo=null

Expected: Registrar created without photo → saved → returned.

Test Case 2c – Valid Data with Special Characters

Input: name="María García-López", email="maria.garcia@school.com", password="P@ssw0rd2024", photo="maria.png"

Expected: Registrar created with Unicode/special chars → saved → returned.

2.Function to create new Student

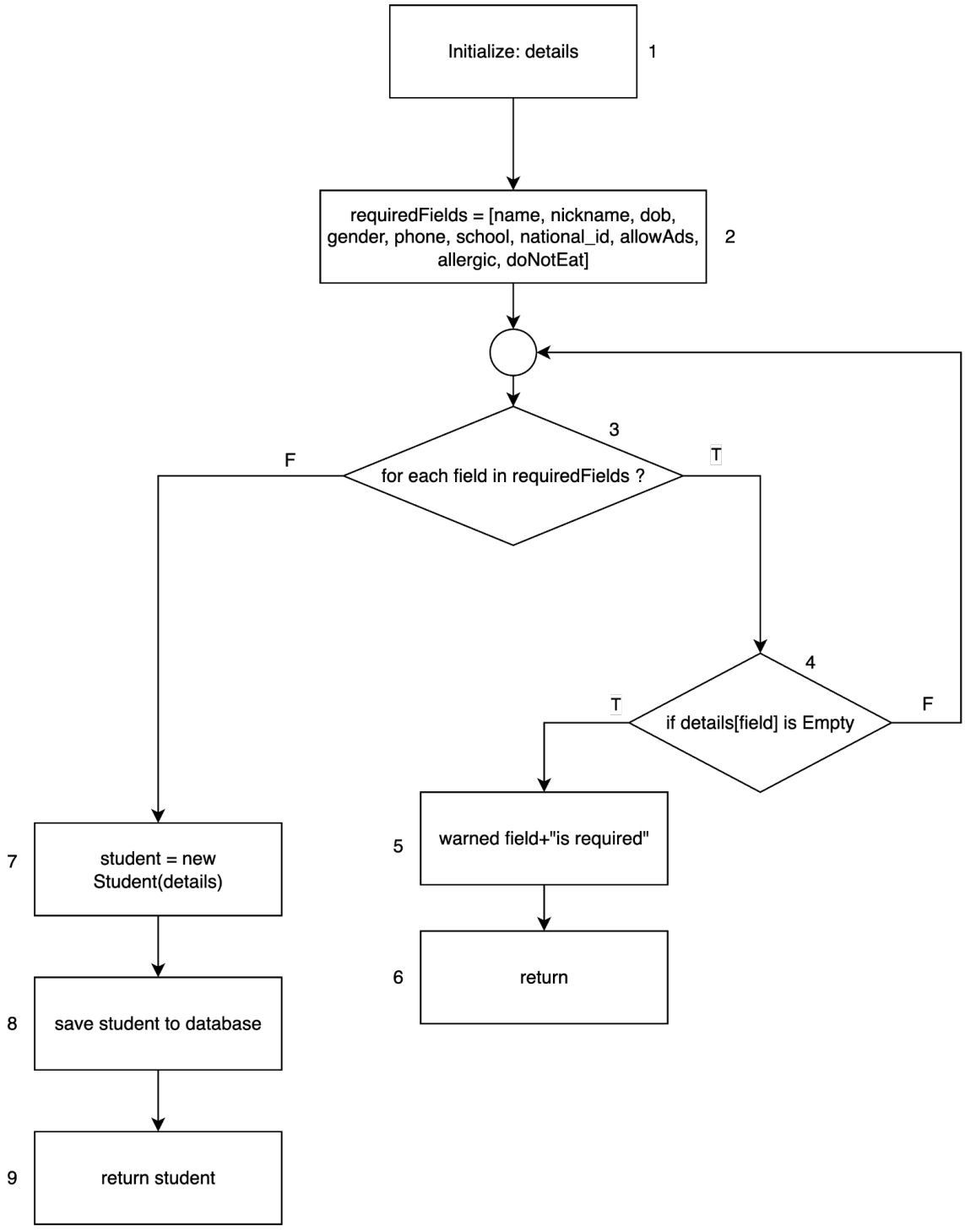


Figure 4 Function to create new Student

Statement Coverage

1. Path 1 → 2 → 3(T) → 4(T) → 5 → 6 (Validation fails: at least one field empty)

   Test Case 1a – Missing Name

   Input: details={name:"", nickname:"Nick", dob:"2010-05-15", gender:"M", phone:"1234567890", school:"ABC", national_id:"111", allowAds:true, allergic:"None", doNotEat:"None"}

   Expected: Warning “Name is required” → function returns → student not created or saved.

   Test Case 1b – Missing National ID

   Input: same as above, but national_id:""

   Expected: Warning “National ID is required” → function returns → no DB save.

   Test Case 1c – Multiple Missing Fields

   Input: details={name:"", dob:"", gender:"M", ...}

   Expected: Warning triggered on the first empty field checked → early return → no DB save.

2. Path 1 → 2 → 3(F) → 7 → 8 → 9 (Successful student creation)

Test Case 2a – All Fields Valid

Input:

{name:"Alice Johnson", nickname:"Ali", dob:"2010-08-20", gender:"F",

phone:"9876543210", school:"High School A", national_id:"222",

allowAds:false, allergic:"Peanuts", doNotEat:"Pork"}

Expected: Student object created → saved to DB → returned successfully.

Test Case 2b – Valid With Optional Empty Fields

Input: (if photo, allowAds, or similar are optional, set them null/empty)

Expected: Student created with defaults → saved → returned.

3.Function to create Parent

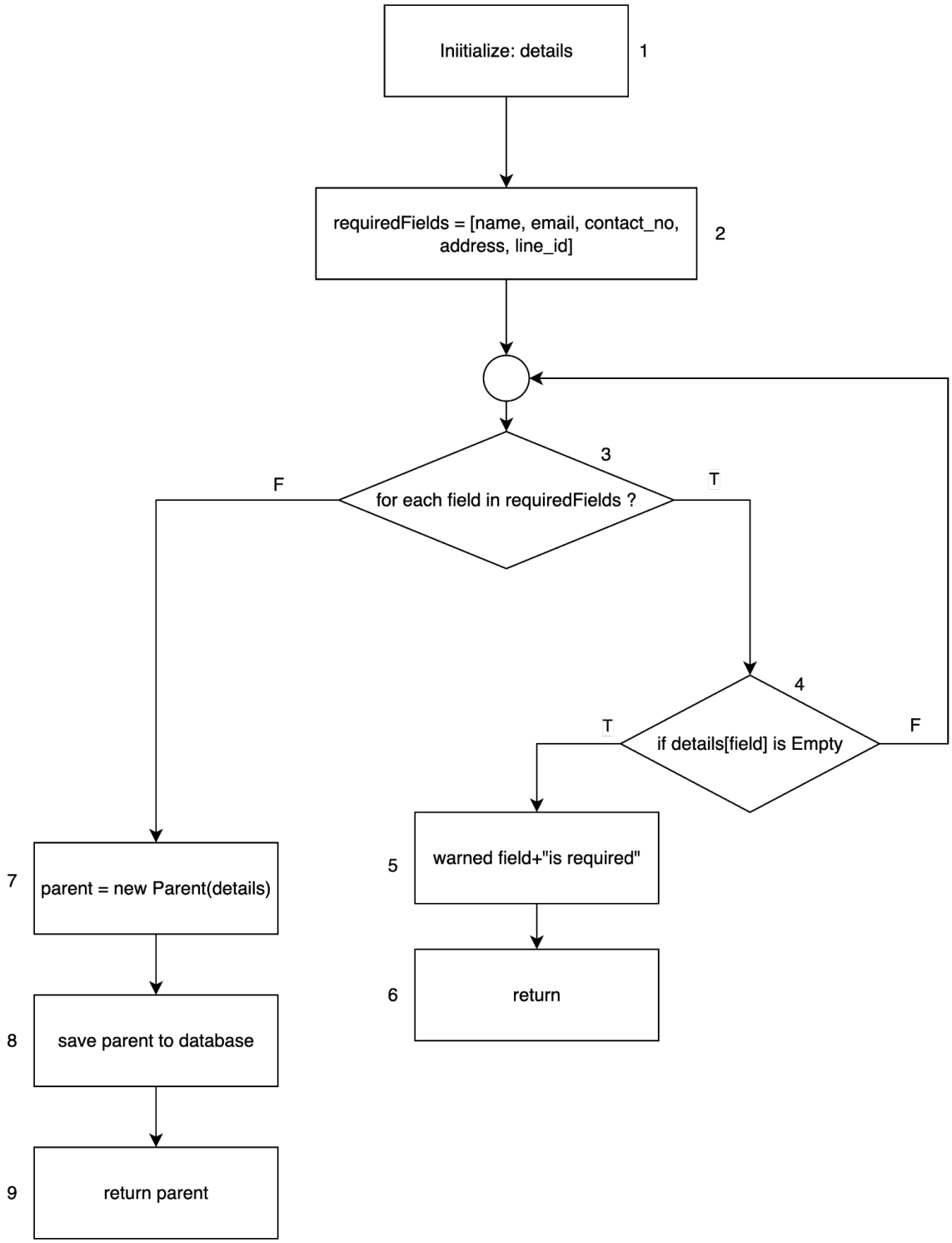


Figure 5 3.Function to create Parent

Statement Coverage

1. Path 1 → 2 → 3(T) → 4(T) → 5 → 6 (Validation fails: at least one required field empty)

Test Case 1a:

details = {name: "", email: "parent@example.com", contact_no: "123456789", address: "123 Main St", line_id: "line123"}

Expected: Warning "name is required", function returns without creating parent

Test Case 1b: details = {name: "John Doe", email: "", contact_no: "123456789", address: "123 Main St", line_id: "line123"}

Expected: Warning "email is required", function returns without creating parent

Test Case 1c: details = {name: "John Doe", email: "parent@example.com", contact_no: "", address: "123 Main St", line_id: "line123"}

Expected: Warning "contact_no is required", function returns without creating parent

Test Case 1d:details = {name: "John Doe", email: "parent@example.com", contact_no: "123456789", address: "", line_id: "line123"}

Expected:Warning "address is required", function returns without creating parent

Test Case 1e:

details = {name: "John Doe", email: "parent@example.com", contact_no: "123456789", address: "123 Main St", line_id: ""}

Expected: Warning "line_id is required", function returns without creating parent

2. Path 1 → 2 → 3(F) → 7 → 8 → 9 (Successful parent creation)

Test Case:

details = {name: "Jane Smith", email: "jane@example.com", contact_no: "987654321", address: "456 Oak Avenue", line_id: "jane_line"}

Expected: Parent object created, saved to database, and returned successfully

3.Path 1-2-3-4-3-4-5-6: (Multiple missing fields - first empty field triggers warning)

Test Case:

details = {name: "John", email: "", contact_no: "", address: "123 Main St", line_id: "line123"}

Expected: Warning for first empty field encountered (email), function returns without checking remaining fields

4.Function to create Teacher

Figure 6 Function to create Teacher

Statement Coverage

Test Paths:

1. Path 1 → 2 → 3(T) → 4(T) → 5 → 6 (Validation fails: missing required field)

Test Case 1a – Missing Name

Input: {name:"", email:"teacher@mail.com", password:"teach123", confirm_password:"teach123", contact_no:"0123456789", address:"123 Street", line_id:"line001", courses:["Math"]}

Expected: Warning “Name is required” → return early → no teacher created.

Test Case 1b – Missing Password

Input: {name:"John Teacher", email:"john@mail.com", password:"", confirm_password:"", contact_no:"0123456789", address:"123 Street", line_id:"line002", courses:["Science"]}

Expected: Warning “Password is required” → return early.

2. Path 1 → 2 → 3(F) → 7(T) → 8 → 9 (Validation fails: password mismatch)

Test Case 2a – Passwords Do Not Match

Input: {name:"Alice Teacher", email:"alice@mail.com", password:"abc123", confirm_password:"xyz789", contact_no:"9876543210", address:"45 Avenue", line_id:"line003", courses:["History"]}

Expected: Warning “Passwords do not match” → return early → no teacher saved.

3. Path 1 → 2 → 3(F) → 7(F) → 10 → 11 → 12 (Successful teacher creation)

Test Case 3a – Valid Data (All Required Fields Present)

Input: {name:"Michael Brown", email:"mike@mail.com", password:"teach2024!", confirm_password:"teach2024!", contact_no:"011223344", address:"77 Blvd", line_id:"line004", courses:["Math","English"]}

Expected: Teacher object created → saved to DB → returned successfully.

Test Case 3b – Valid Data With Special Characters

Input: {name:"José Martínez", email:"jose.martinez@mail.com", password:"P@ssword123", confirm_password:"P@ssword123", contact_no:"+34123456789", address:"Madrid, Spain", line_id:"line005", courses:["Biology"]}

Expected: Teacher created with Unicode and international values → saved → returned.

Student Management

1. Function to register New Student

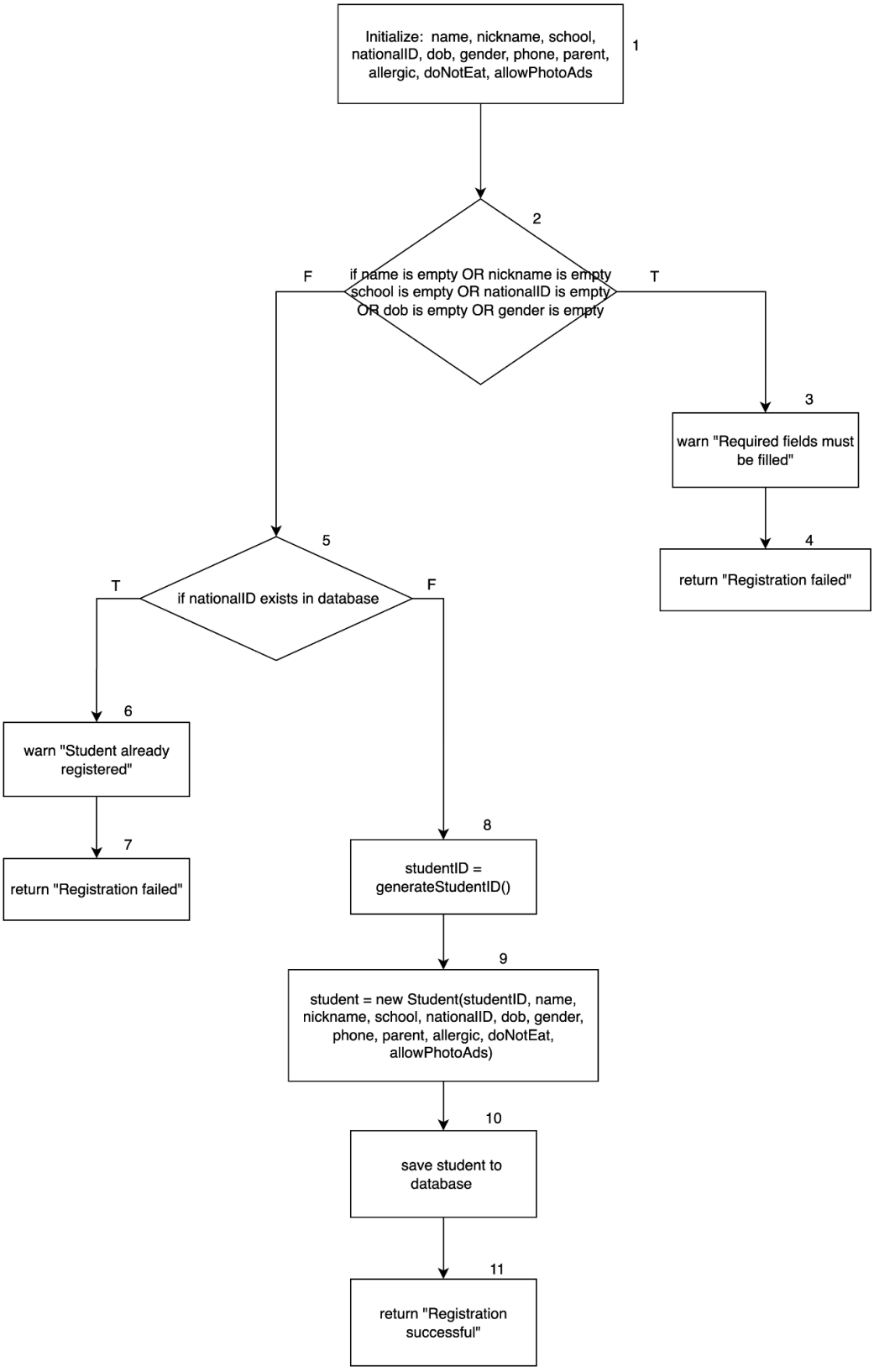


Figure 7 CFG of the Function to register New Student

Statement Coverage

1. Path 1 → 2(T) → 3 → 4 (Validation fails: missing required field)

Test Case 1a – Missing Name

Input: {name:"", nickname:"Sam", school:"Central High", nationalID:"12345", dob:"2010-01-01", gender:"M", phone:"0123456789", parent:"Parent A", allergy:"None", contactNo:"0987654321", allowToPickUp:"Yes"}

Expected: Warning “Required fields must be filled” → return “Registration failed”.

Test Case 1b – Multiple Missing Fields

Input: {name:"", nickname:"", school:"", nationalID:"", dob:"2010-01-01", gender:"", phone:"", parent:"", allergy:"None", contactNo:"", allowToPickUp:"No"}

Expected: Validation fails immediately → return “Registration failed”.

2. Path 1 → 2(F) → 5(T) → 6 → 7 (Validation passes, but student already exists)

Test Case 2a – Duplicate National ID

Input: {name:"John Doe", nickname:"Johnny", school:"Central High", nationalID:"12345", dob:"2010-02-01", gender:"M", phone:"0123456789",

parent:"Parent B", allergy:"Peanuts", contactNo:"0987654321",

allowToPickUp:"Yes"}

Precondition: nationalID=12345 already exists in DB.

Expected: Warning “Student already registered” → return “Registration failed”.

3. Path 1 → 2(F) → 5(F) → 8 → 9 → 10 → 11 (Successful student registration)

Test Case 3a – Valid Data (Unique National ID)

Input: {name:"Sarah Lee", nickname:"Sally", school:"Greenwood Elementary",

nationalID:"67890", dob:"2012-06-15", gender:"F", phone:"011223344",

parent:"Parent C", allergy:"None", contactNo:"099887766", allowToPickUp:"Yes"}

Precondition: nationalID=67890 does not exist in DB.

Expected: New student ID generated → student object created → saved in DB →

return “Registration successful”.

Test Case 3b – Valid Data With Edge Values

Input: {name:"王小明", nickname:"Xiao", school:"Beijing School",

nationalID:"CN999", dob:"2011-12-31", gender:"M", phone:"+8613800138000",

parent:"王大明", allergy:"Dust", contactNo:"+8613900123456",

allowToPickUp:"No"}

Expected: Student registered successfully with Unicode/international data → saved → return success.

2.Function to register new parent

Figure 8 CFG of the Function to register new parent

Statement Coverage

1. Path 1 → 2(T) → 3 → 4 (Validation fails: missing required field)

Test Case 1a – Missing Email

Input: {name:"Alice Parent", email:"", contactNo:"0123456789", lineID:"line123", address:"123 Street"}

Expected: Warning “Required fields must be filled” → return “Registration failed”.

Test Case 1b – Multiple Missing Fields

Input: {name:"", email:"", contactNo:"", lineID:"line999", address:""}

Expected: Immediate failure → warning → return “Registration failed”.

2. Path 1 → 2(F) → 5(T) → 6 → 7 (Validation passes, but invalid email format)

Test Case 2 – Invalid Email Format

Input: {name:"Bob Parent", email:"bob-at-mail.com", contactNo:"011223344", lineID:"line456", address:"45 Avenue"}

Expected: Warning “Invalid email address” → return “Registration failed”.

3. Path 1 → 2(F) → 5(F) → 8(T) → 9 → 10 (Validation passes, but duplicate email)

Test Case 3 – Duplicate Email

Input: {name:"Carol Parent", email:"parent@mail.com", contactNo:"099887766", lineID:"line789", address:"67 Road"}

Precondition: email=parent@mail.com already exists in DB.

Expected: Warning “Parent already registered with this email” → return “Registration failed”.

4. Path 1 → 2(F) → 5(F) → 8(F) → 11 → 12 → 13 → 14 (Successful parent registration)

Test Case 4a – Valid Data (Unique Email)

Input: {name:"David Parent", email:"david.parent@mail.com", contactNo:"088776655", lineID:"line101", address:"89 Boulevard"}

Precondition: Email not in DB.

Expected: New parent ID generated → parent object created → saved in DB → return “Registration successful”.

Test Case 4b – Valid Data With Unicode/International Values

Input: {name:"王大明", email:"wang.daming@example.cn",

contactNo:"+8613800138000", lineID:"line202", address:"Beijing, China"}

Expected: Parent registered successfully with Unicode and international data → return success.

Scheduling

1.Function to create schedule

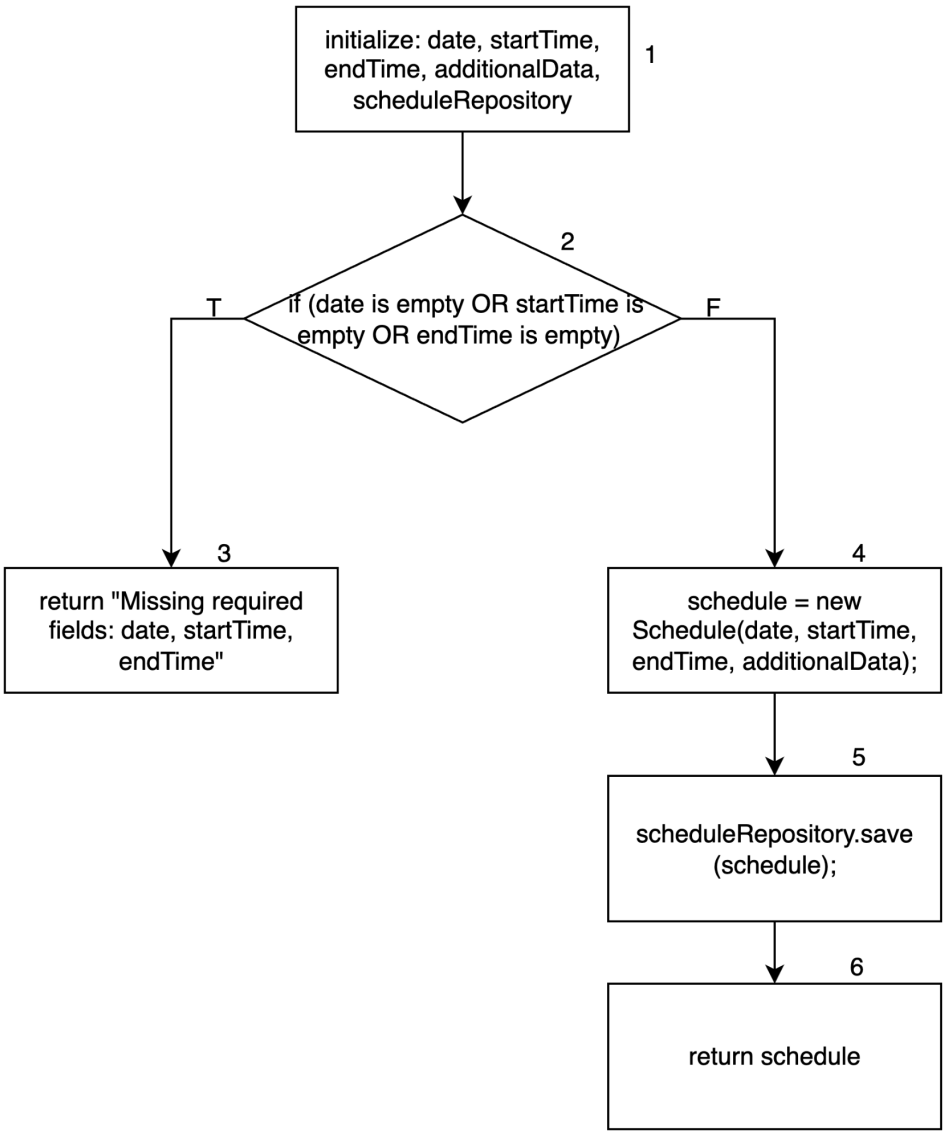


Figure 9 CFG of the Function to create schedule Control Flow Graph

Statement Coverage

1. Path 1 → 2(T) → 3 (Validation fails: missing required fields)

Test Case 1a – Missing Date

Input: {date:"", startTime:"09:00", endTime:"11:00", additionalData:"Math Class"}

Expected: Error “Missing required fields: date, startTime, endTime” → return failure.

Test Case 1b – Missing Start Time

Input: {date:"2025-09-22", startTime:"", endTime:"11:00", additionalData:"Science Class"}

Expected: Same error → return failure.

Test Case 1c – Multiple Missing Fields

Input: {date:"", startTime:"", endTime:"", additionalData:"English Class"}

Expected: Validation fails immediately → return failure.

2. Path 1 → 2(F) → 4 → 5 → 6 (Successful schedule creation)

Test Case 2a – Valid Schedule

Input: {date:"2025-09-25", startTime:"14:00", endTime:"16:00", additionalData:"Music Class"}

Expected: New Schedule object created → saved in repository → returned successfully.

Test Case 2b – Valid Schedule With Additional Metadata

Input: {date:"2025-09-30", startTime:"08:00", endTime:"09:30",

additionalData:"Room=201, Teacher=Mr. Lee"}

Expected: Schedule created with extra data → saved → returned.

2.Function to check for conflicts

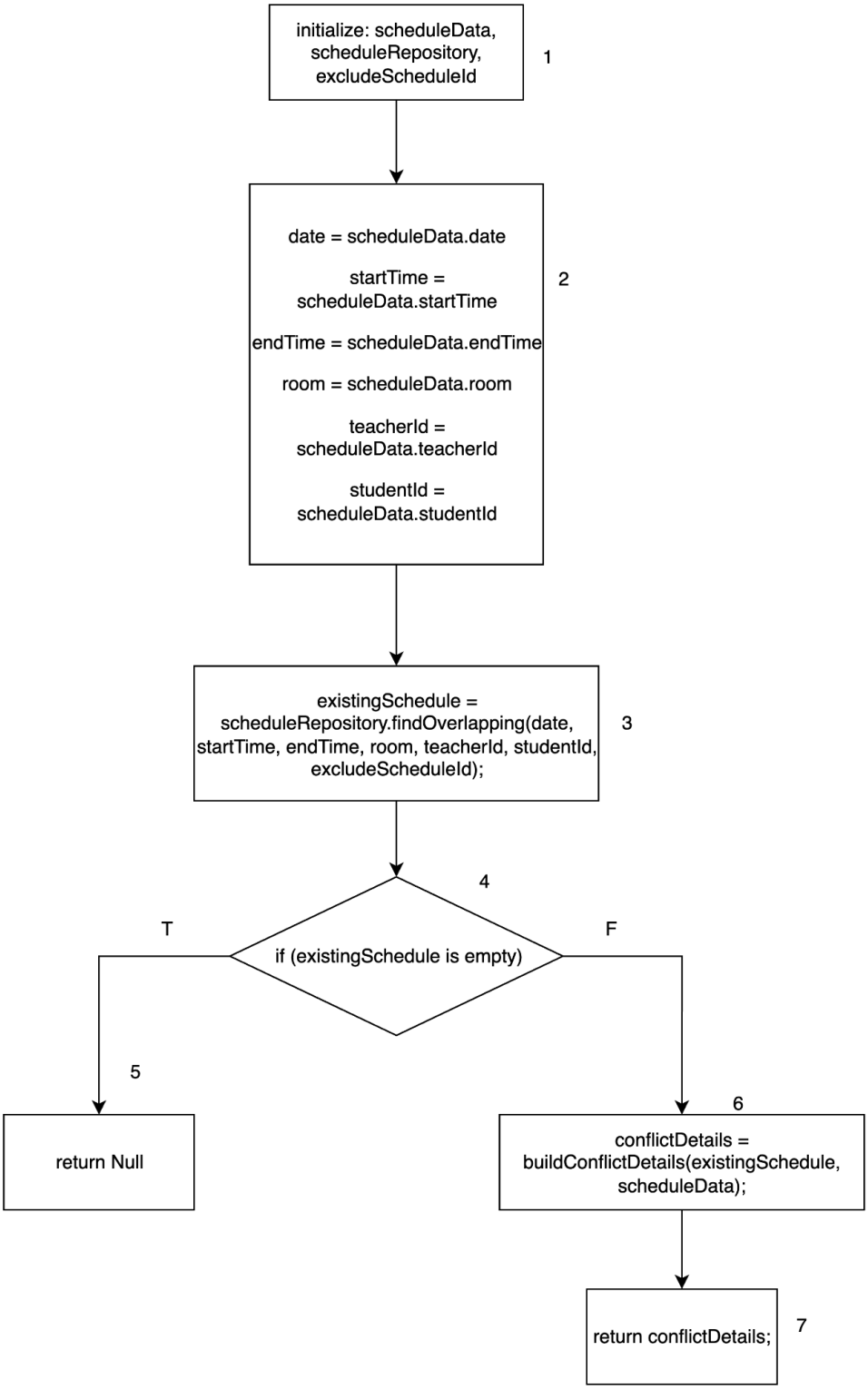


Figure 10 CFG of the Function to check for conflicts CFG

Statement Coverage

1. Path 1 → 2 → 3 → 4(T) → 5 (No conflicts found)

Test Case: scheduleData = {date: "2024-01-15", startTime: "09:00", endTime: "11:00", room: "Room A", teacherId: "T001", studentId: "S001"}, excludeScheduleId = null

Precondition: No existing schedules overlap with the given time, room, teacher, and student combination

Expected: Function returns Null indicating no conflicts detected

2. Path 1 → 2 → 3 → 4(F) → 6 → 7 (Conflict detected)

Test Case 2a – Room Conflict

Input: {date:"2025-09-25", startTime:"10:00", endTime:"12:00", room:"Room A", teacherId:"T001", studentId:"S001"}

Precondition: Another schedule already exists in Room A from 10:30–11:30.

Expected: Conflict detected → conflict details built → returned.

Test Case 2b – Teacher Conflict

Input: {date:"2025-09-25", startTime:"13:00", endTime:"15:00", room:"Room B", teacherId:"T001", studentId:"S002"}

Precondition: Teacher T001 has another class scheduled 14:00–15:00.

Expected: Teacher conflict → conflict details returned.

Test Case 2c – Student Conflict

Input: {date:"2025-09-25", startTime:"09:00", endTime:"11:00", room:"Room C", teacherId:"T002", studentId:"S001"}

Precondition: Student S001 already has another class during 09:30–10:30.

Expected: Student conflict → details returned.

Attendance and Feedback

1.Function to log Attendance And Feedback

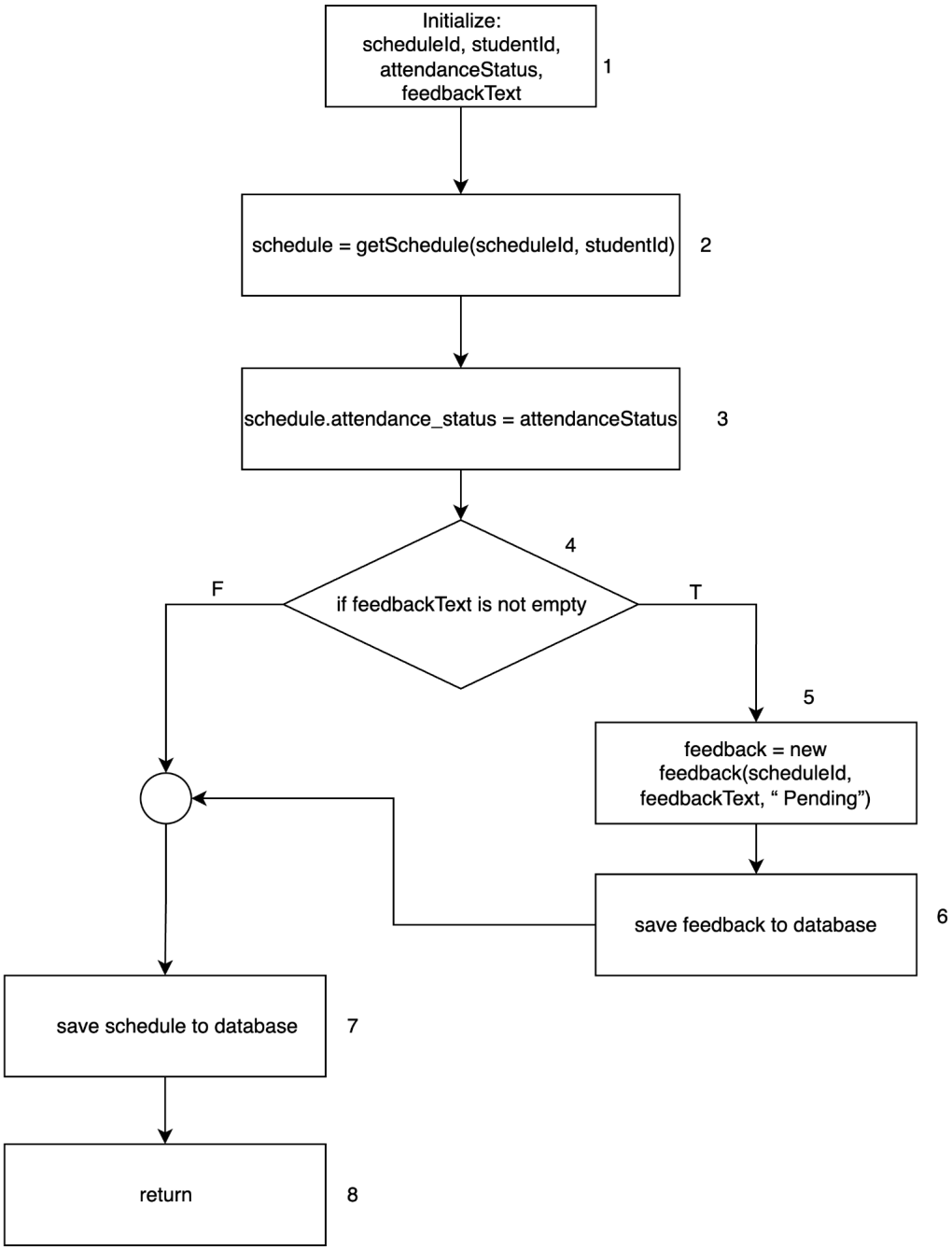


Figure 11 CFG of the Function to log Attendance And Feedback

Statement Coverage

1. Path 1 → 2 → 3 → 4(F) → 7 → 8 (Attendance only, no feedback)

Test Case 1a – Present Without Feedback

Input: scheduleId="SCH001", studentId="S001", attendanceStatus="Present", feedbackText=""

Expected: Attendance marked "Present", no feedback object created, schedule saved.

Test Case 1b – Absent Without Feedback

Input: scheduleId="SCH002", studentId="S002", attendanceStatus="Absent", feedbackText=""

Expected: Attendance marked "Absent", no feedback object created, schedule saved.

Test Case 1c – Late Without Feedback

Input: scheduleId="SCH003", studentId="S003", attendanceStatus="Late", feedbackText=""

Expected: Attendance marked "Late", no feedback created, schedule saved.

2. Path 1 → 2 → 3 → 4(T) → 5 → 6 → 7 → 8 (Attendance with feedback)

Test Case 2a – Absent With Feedback

Input: scheduleId="SCH002", studentId="S002", attendanceStatus="Absent", feedbackText="Did not attend class due to illness"

Expected:

Attendance status updated "Absent".

Feedback object created (status="Pending") and saved.

Schedule saved, function returns success.

Test Case 2b – Present With Feedback

Input: scheduleId="SCH003", studentId="S003", attendanceStatus="Present", feedbackText="Participated actively"

Expected: Attendance updated "Present", feedback saved, schedule saved, return success.

Test Case 2c – Late With Feedback

Input: scheduleId="SCH006", studentId="S006", attendanceStatus="Late", feedbackText="Arrived 20 minutes late"

Expected: Attendance updated "Late", feedback created & saved, schedule saved.

2.Function to edit Feedback

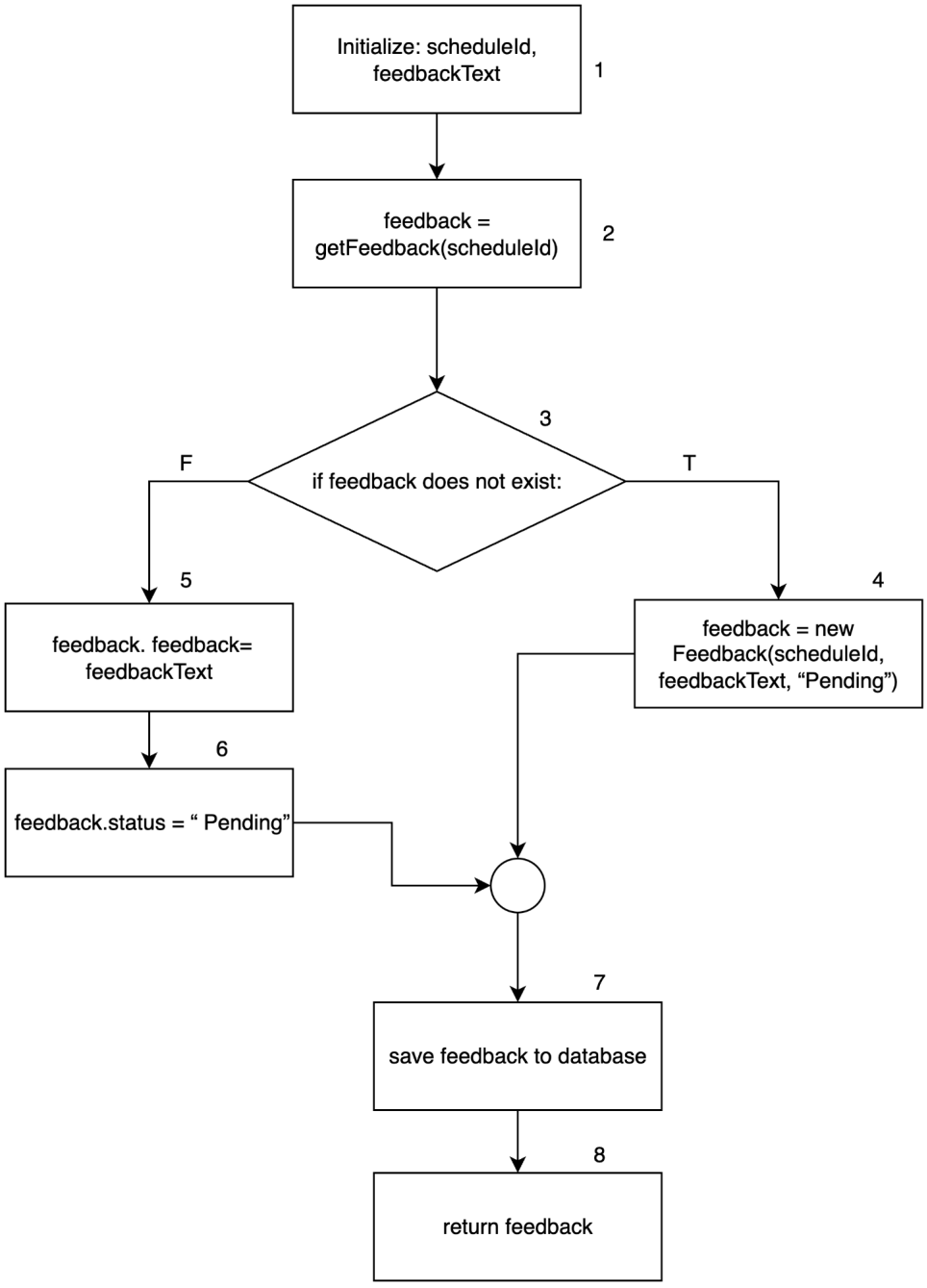


Figure 12 CFG of Function to edit Feedback

Statement Coverage

Test Paths:

1. Path 1 → 2 → 3(T) → 4 → 6 → 7 → 8: (New feedback creation)

Test Case 1a – New Feedback With Text

Input: scheduleId="SCH001", feedbackText="Great participation"

Expected Result:

- System does not find existing feedback.
- New feedback created with "Pending" status.
- Feedback saved to database.
- Returns created feedback.

Test Case 1b – New Feedback Empty Text

Input: scheduleId="SCH002", feedbackText=""

Expected Result:

- New empty feedback still created with "Pending" status.
- Saved to database and returned.

2.Path 1 → 2 → 3(F) → 5 → 6 → 7 → 8: (Update existing feedback)

Test Case 2a – Update Existing Feedback

Precondition: Feedback record already exists for scheduleId="SCH003".

Input: feedbackText="Needs improvement in attendance"

Expected Result:

- Existing feedback is retrieved.
- Its feedbackText updated.
- Status set to "Pending".
- Feedback saved and returned.

Test Case 2b – Update Existing Feedback With Empty Text

Precondition: Feedback exists for scheduleId="SCH004".

Input: feedbackText=""

Expected Result:

- Existing feedback updated with empty text.
- Status set to "Pending".
- Saved and returned.

Daily Overview

1.Function to show Today Schedule

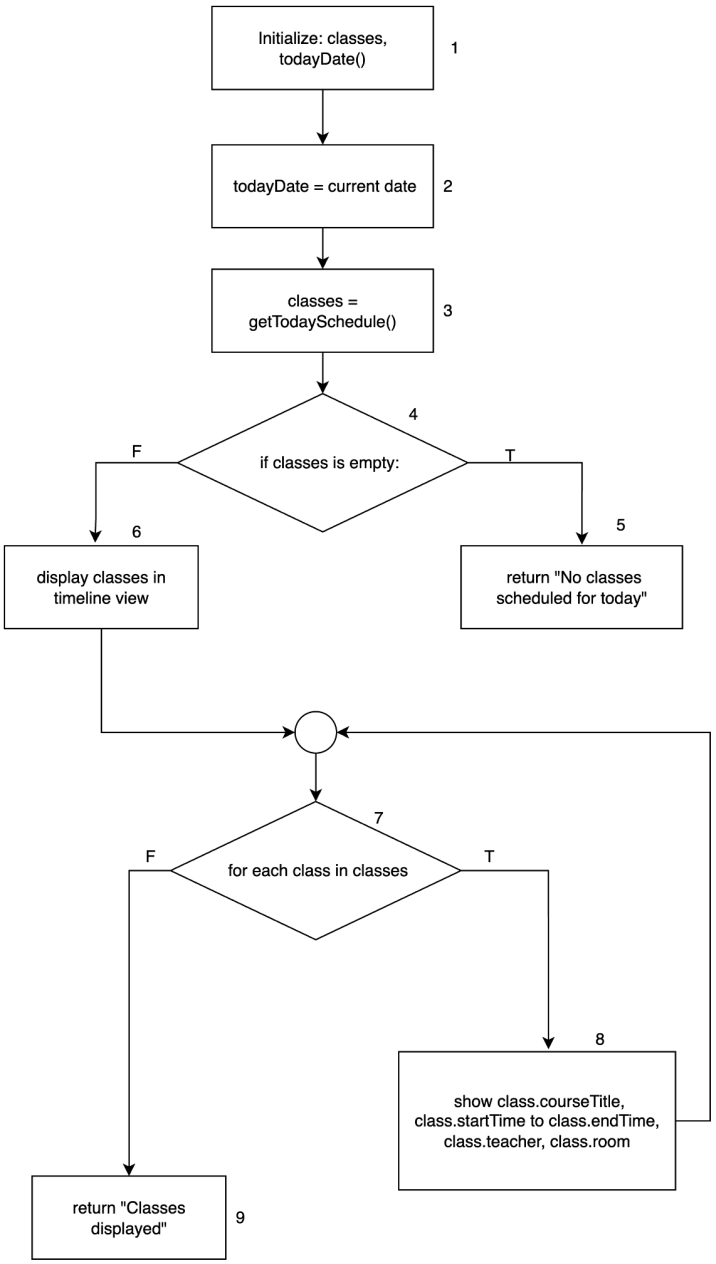


Figure 13 CFG of Function to show Today Schedule

Statement Coverage

1. Path 1 → 2 → 3 → 4(T) → 5 (No classes scheduled)

Test Case 1 – Empty Schedule

Input: todayDate="2024-01-15", getTodaySchedule() returns []

Precondition: No classes exist for the date.

Expected Result: Function returns message "No classes scheduled for today".

2. Path 1 → 2 → 3 → 4(F) → 6 → 7 → 8 → 7 → 9 (Single class scheduled)

Input:

[{courseTitle: "Robotics Basics", startTime: "09:00", endTime: "11:00", teacher: "Mr. Smith", room: "Room A"}]

Expected Result: Timeline view displays one class with details.

Function returns "Classes displayed".

3. Path 1 → 2 → 3 → 4(F) → 6 → 7 → 8 → 7 → 8 → 7 → 9 (Multiple classes scheduled)

Test Case: todayDate="2024-01-15", getTodaySchedule() returns [{courseTitle: "Robotics Basics", startTime: "09:00", endTime: "11:00", teacher: "Mr. Smith", room: "Room A"}, {courseTitle: "Programming 101", startTime: "13:00", endTime: "15:00", teacher: "Ms. Johnson", room: "Room B"}, {courseTitle: "AI Workshop", startTime: "16:00", endTime: "18:00", teacher: "Dr. Brown", room: "Room C"}]

Expected: Timeline view displays all three classes in chronological order with complete details, returns "Classes displayed"

4. Path 1 → 2 → 3 → 4(F) → 6 → 7 → 8 → 7 → 9 (Class with missing optional details)

Test Case:

todayDate="2024-01-15", getTodaySchedule() returns [{courseTitle: "Emergency Class", startTime: "10:00", endTime: "12:00", teacher: "", room: "TBD"}]

Expected: Timeline view displays class with available information, shows empty/TBD fields as provided, returns "Classes displayed"

2.Function to show Attending Students

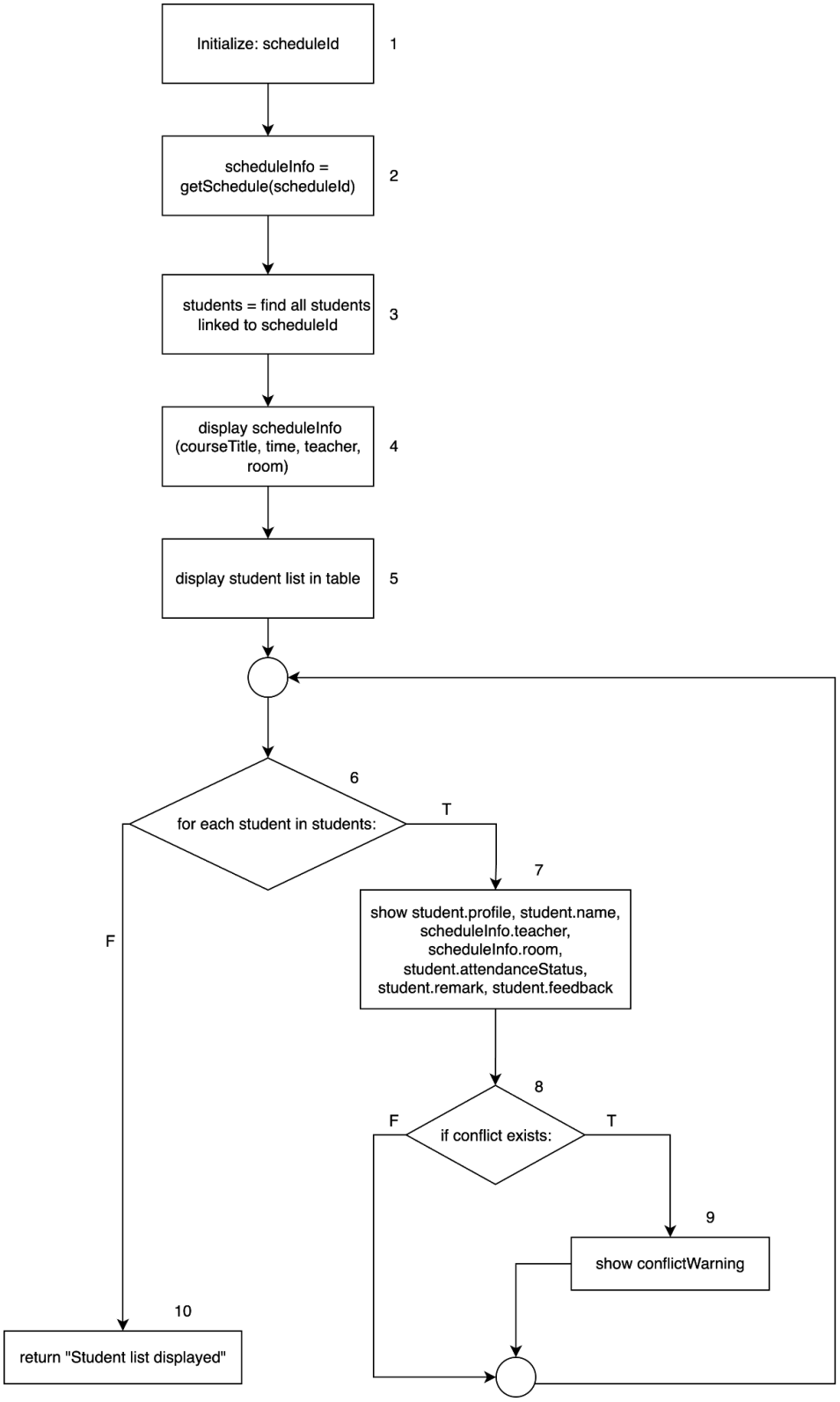


Figure 14 CFG of the Function to show Attending Students

Statement Coverage

1. Path 1 → 2 → 3 → 4 → 5 → 6(F) → 10 (No students attending)

Test Case 1 – Empty Student List

Input: scheduleId="SCH001", getScheduleInfo("SCH001") returns valid schedule, but no students enrolled.

Precondition: Schedule exists, but student list is empty.

Expected Result: Display schedule info only, student table is empty. Function returns "Student list displayed".

2. Path 1 → 2 → 3 → 4 → 5 → 6(T) → 7 → 8(F) → 10 (Students attending, no conflicts)

Test Case 2 – Students with No Conflicts

Input:

students = [

{id: "S001", name: "Alice", attendance: "Present", remark: "Good", feedback: "Engaged"},

{id: "S002", name: "Bob", attendance: "Present", remark: "Average", feedback: "Quiet"}

]

conflict.exists = false

Expected Result: Display each student's profile, attendance, remarks, and feedback. No conflict warning shown. Function returns "Student list displayed".

3. Path 1 → 2 → 3 → 4 → 5 → 6(T) → 7 → 8(T) → 9 → 10 (Students attending with conflicts)

Test Case 3 – Students with Conflicts

Input:

students = [

{id: "S003", name: "Charlie", attendance: "Absent", remark: "", feedback: ""},

{id: "S004", name: "Diana", attendance: "Present", remark: "Late", feedback: ""}

]

conflict.exists = true

Expected Result: Display student details. Conflict detected, so showConflictWarning() is triggered. Function returns "Student list displayed".

Security

Function to login with role based access

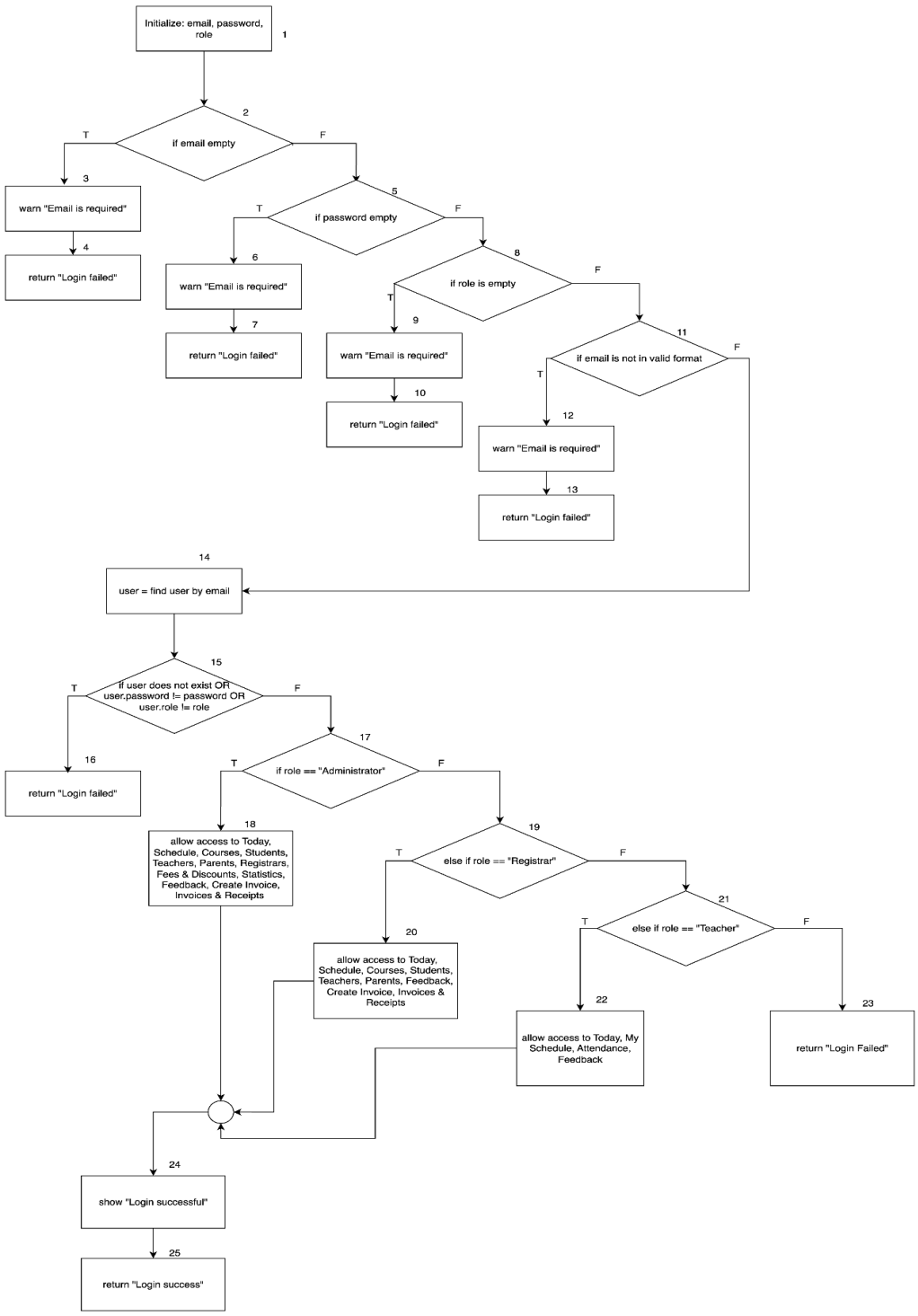


Figure 15 CFG of the function to login with role based access

Statement Coverage

1. Path 1 → 2(T) → 3 → 13 (Empty username/email)

Test Case 1 – Empty Email

Input: email="", password="admin123"

Precondition: Email not provided.

Expected Result: Warning “Email is required”, login fails.

2. Path 1 → 2(F) → 4(T) → 5 → 13 (Empty password)

Test Case 2 – Empty Password

Input: email="admin@school.com", password=""

Expected Result: Warning “Password is required”, login fails.

3. Path 1 → 2(F) → 4(F) → 6(T) → 7 → 13 (Email not in system)

Test Case 3 – Unregistered Email

Input: email="unknown@school.com", password="test123"

Precondition: Email not found in database.

Expected Result: Warning “Email not recognized”, login fails.

4. Path 1 → 2(F) → 4(F) → 6(F) → 8(T) → 9 → 13 (Incorrect password)

Test Case 4 – Wrong Password

Input: email="teacher@school.com", password="wrongPass"

Precondition: Email exists but password doesn’t match.

Expected Result: Warning “Incorrect password”, login fails.

5. Path 1 → 2(F) → 4(F) → 6(F) → 8(F) → 10 → 14 → 15(Admin role)

Test Case 5 – Successful Admin Login

Input: email="admin@school.com", password="admin2024!"

Precondition: Email exists, password matches, role=Admin.

Expected Result: System shows "Login successful" with admin dashboard access.

6. Path 1 → 2(F) → 4(F) → 6(F) → 8(F) → 10 → 14 → 16(Registrar role)

Test Case 6 – Successful Registrar Login

Input: email="registrar@school.com", password="reg@123"

Precondition: Valid credentials, role=Registrar.

Expected Result: "Login successful" with registrar panel access.

7. Path 1 → 2(F) → 4(F) → 6(F) → 8(F) → 10 → 14 → 17(Teacher role)

Test Case 7 – Successful Teacher Login

Input: email="teacher@school.com", password="teachPass"

Precondition: Valid credentials, role=Teacher.

Expected Result: "Login successful" with teacher dashboard access.

8. Path 1 → 2(F) → 4(F) → 6(F) → 8(F) → 10 → 14 → 18(Student role)

Test Case 8 – Successful Student Login

Input: email="student01@school.com", password="stud1234"

Precondition: Valid credentials, role=Student.

Expected Result: "Login successful" with student portal access.

9. Path 1 → 2(F) → 4(F) → 6(F) → 8(F) → 10 → 14 → 19(F) → 20 → 13 (Unknown role → login fails)

Test Case 9 – Unknown Role

Input: email="guest@school.com", password="guest123"

Precondition: Valid credentials but role not mapped (e.g., “Guest”).

Expected Result: Warning "Login failed".

Data Flow Testing

Data flow testing is a white box testing technique that examines how variables flow through a program by tracking their definitions (assignment/initialization), uses (references/access), and eliminations (scope exit/reassignment) throughout execution paths. This approach requires knowledge of internal code structure to identify data flow anomalies like uninitialized variables, unused variables, or variables used after being killed, ensuring proper data propagation and validating that all data dependencies are correctly handled across different execution paths.

1. Course Management

1.createCourse

courseTitle, courseDescription, ageRange, medium: Defined by input, used in validation and to create course.

course: Defined after validation, used to save to database, returned.

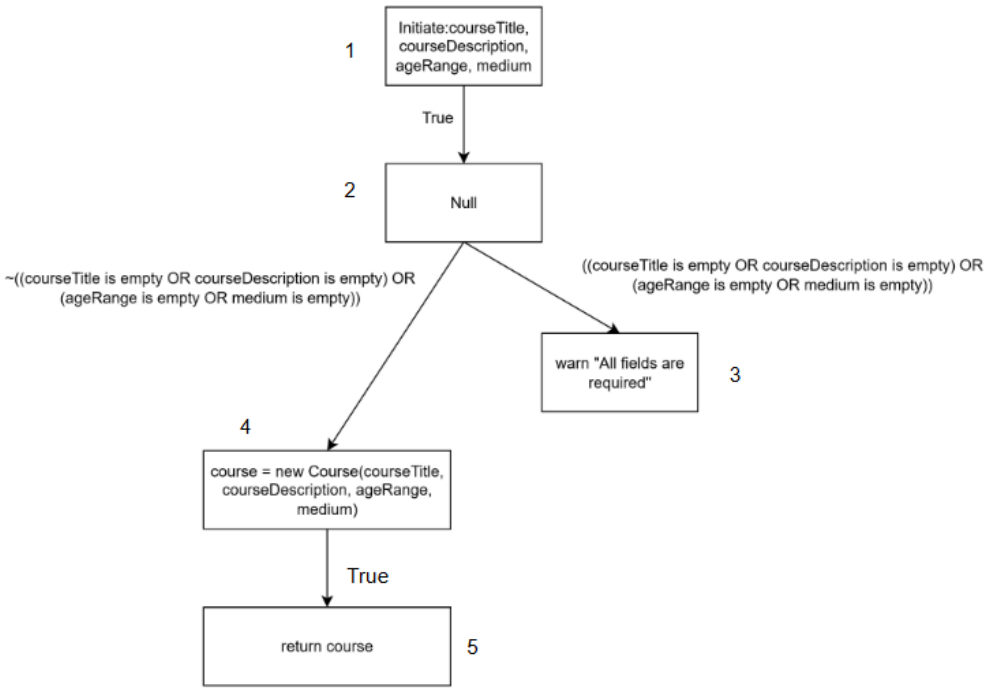


Figure 16 DFG of createCourse

Predicates and p-use() Set of Edges

No table of figures entries found.	Predicate (i,j)	p-use(i,j)
(1,2)	True	{}
(2,3)	True	{}
(2,4)	~(courseTitle empty OR courseDescription empty OR ageRange empty OR medium empty)	{courseTitle, courseDescription, ageRange, medium}

(4,5)	True	{course}

2.addCoursePackageToStudent

studentId, courseId, classType, scheduleDetails: Defined by input, used in logic to generate schedules.

schedule: Defined in loop, used for conflict checking and saving.

room, teacher: Used in conflict checking.

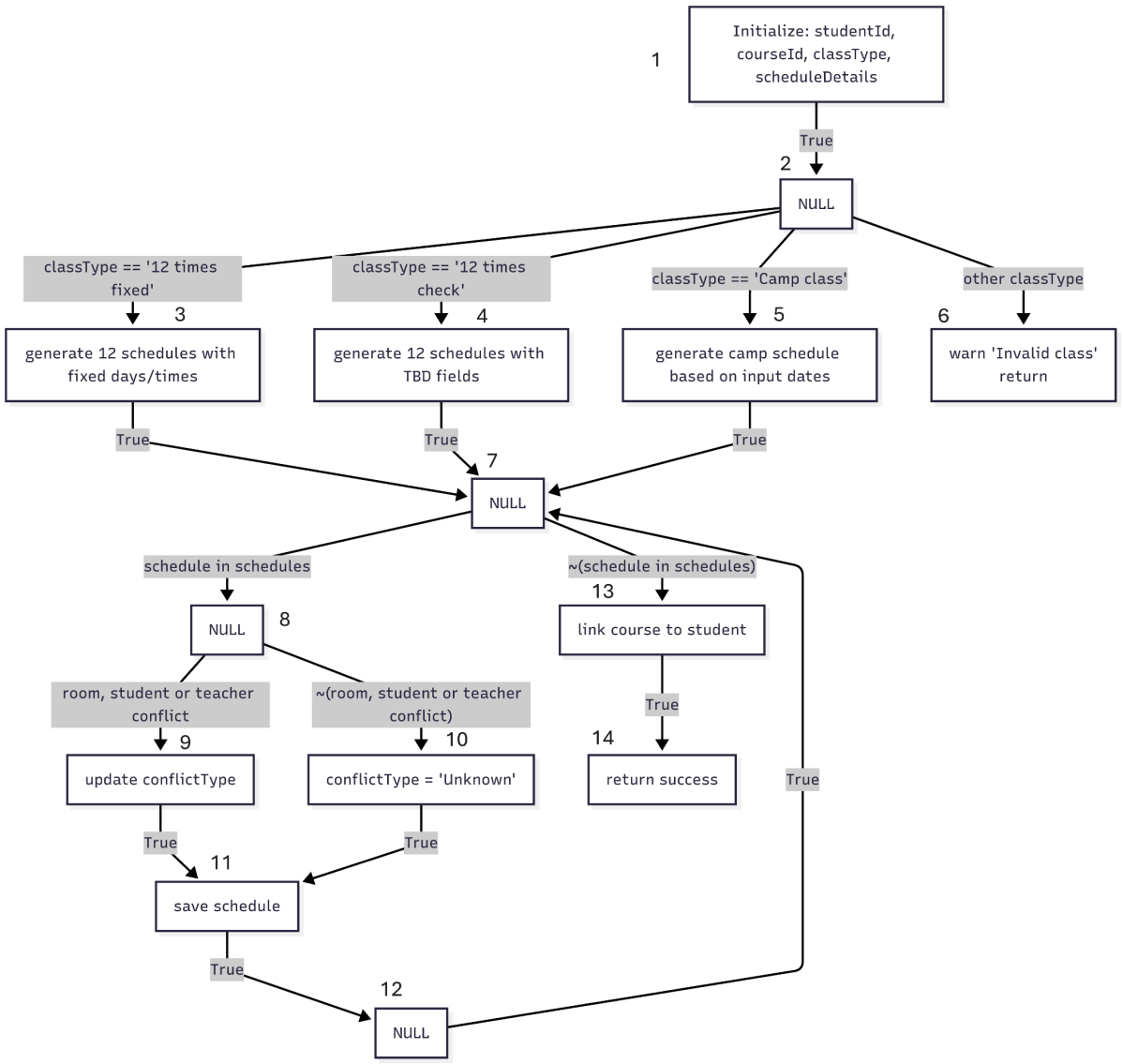


Figure 17 DFG of addCoursePackageToStudent

Predicates and p-use() Set of Edges

Edge (i, j)	Predicate(i, j)	P-Use(i, j)
(1, 2)	True	{}
(2, 3)	classType == '12 times fixed'	{classType}
(2, 4)	classType == '12 times check'	{classType}
(2, 5)	classType == 'camp class'	{classType}
(2, 6)	other classType	{classType}
(3, 7)	True	{}
(4, 7)	True	{}
(5, 7)	True	{}
(6, 12)	True	{}
(7, 8)	schedule in scheduling	{schedule}
(7, 13)	~(schedule in scheduling)	{schedule}
(8, 9)	room, student or teacher conflict	{room, student, teacher}
(8, 10)	~(room, student or teacher conflict)	{room, student, teacher}
(9, 11)	True	{}
(10, 11)	conflictType == 'unknown'	{conflictType}
(10, 14)	conflictType != 'unknown'	{conflictType}
(11, 12)	True	{}
(13, 14)	True	{}
(14, 12)	True	{}
(12, 2)	True	{}

User Management

1.createRegistrar

name, email, password, photo: Defined by input, used in validation and to create registrar.

registrar: Defined after validation, used to save to database, returned.

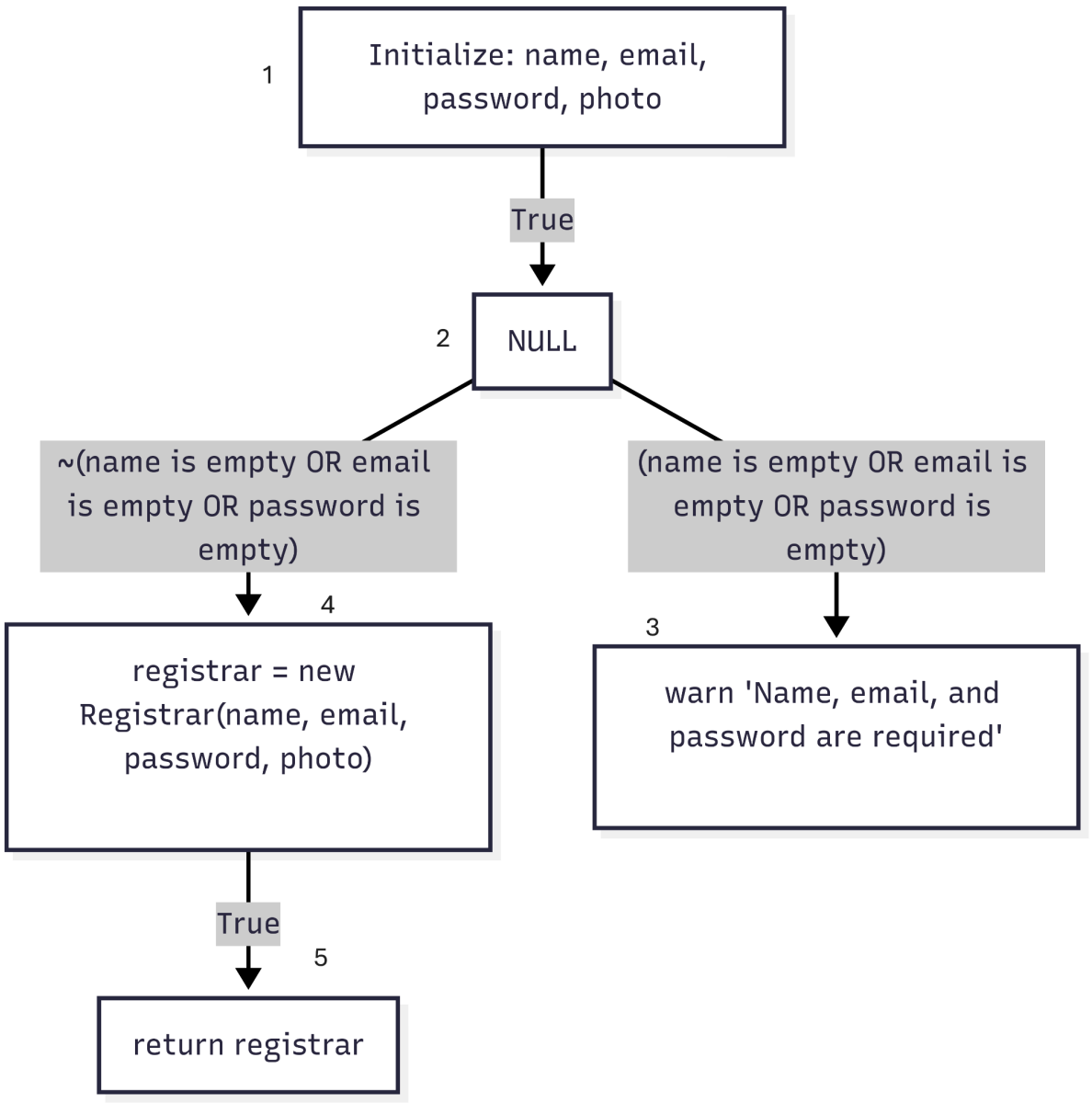


Figure 18 DFG of createRegistrar

Predicates and p-use() Set of Edges

Edge (i, j)	Predicate(i, j)	P-Use(i, j)
(1, 2)	True	{}
(2, 3)	(name is empty OR email is empty OR password is empty)	{name, email, password}
(2, 4)	~(name is empty OR email is empty OR password is empty)	{name, email, password}
(3, 1)	True	{}
(4, 5)	True	{}
(5, 1)	True	{}

2.createStudent

details: Defined by input, used to check required fields.

student: Defined after validation, used to save to database, returned.

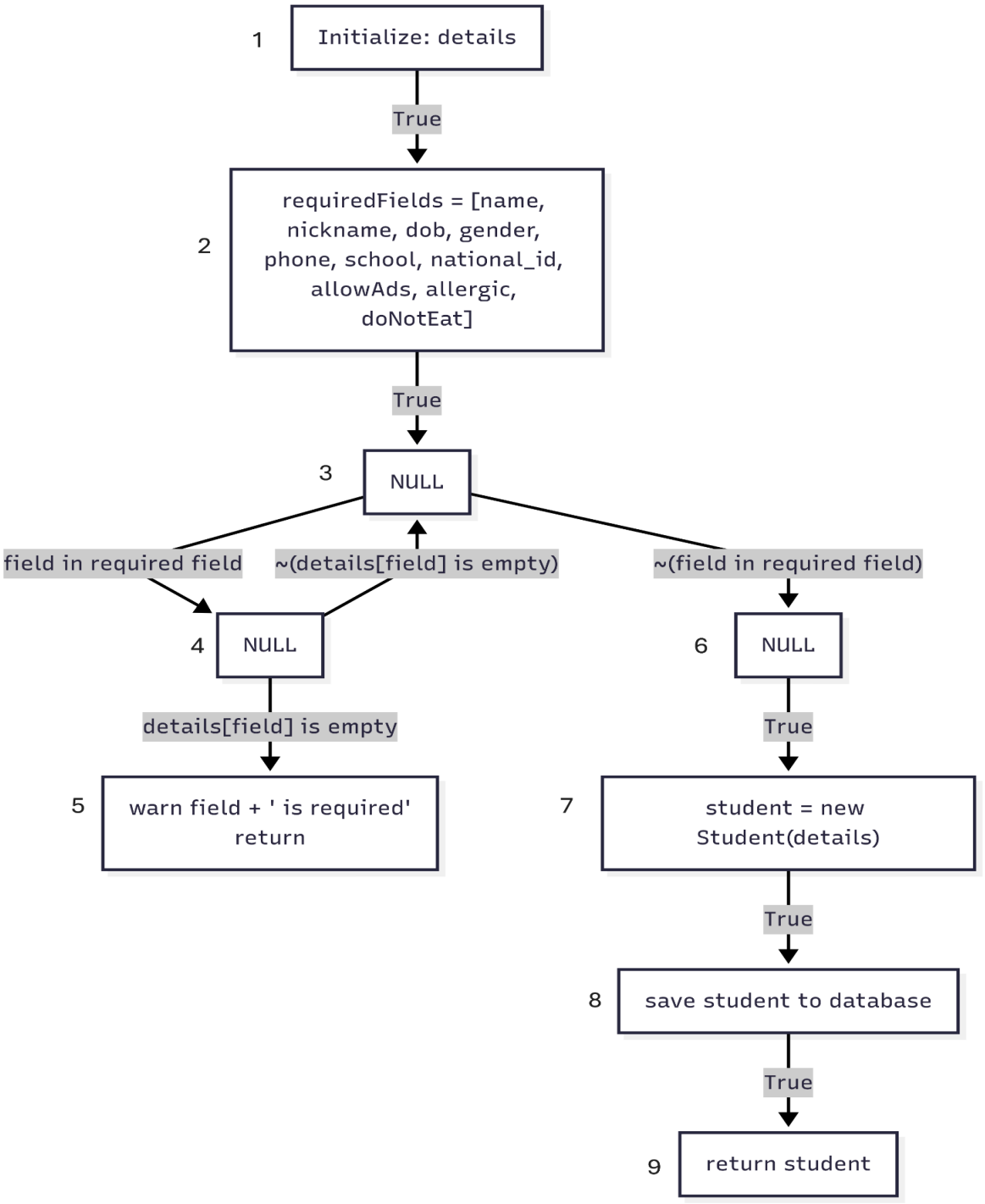


Figure 19 DFG of createStudent

3.createParent

details: Defined by input, used to check required fields.

parent: Defined after validation, used to save to database, returned.

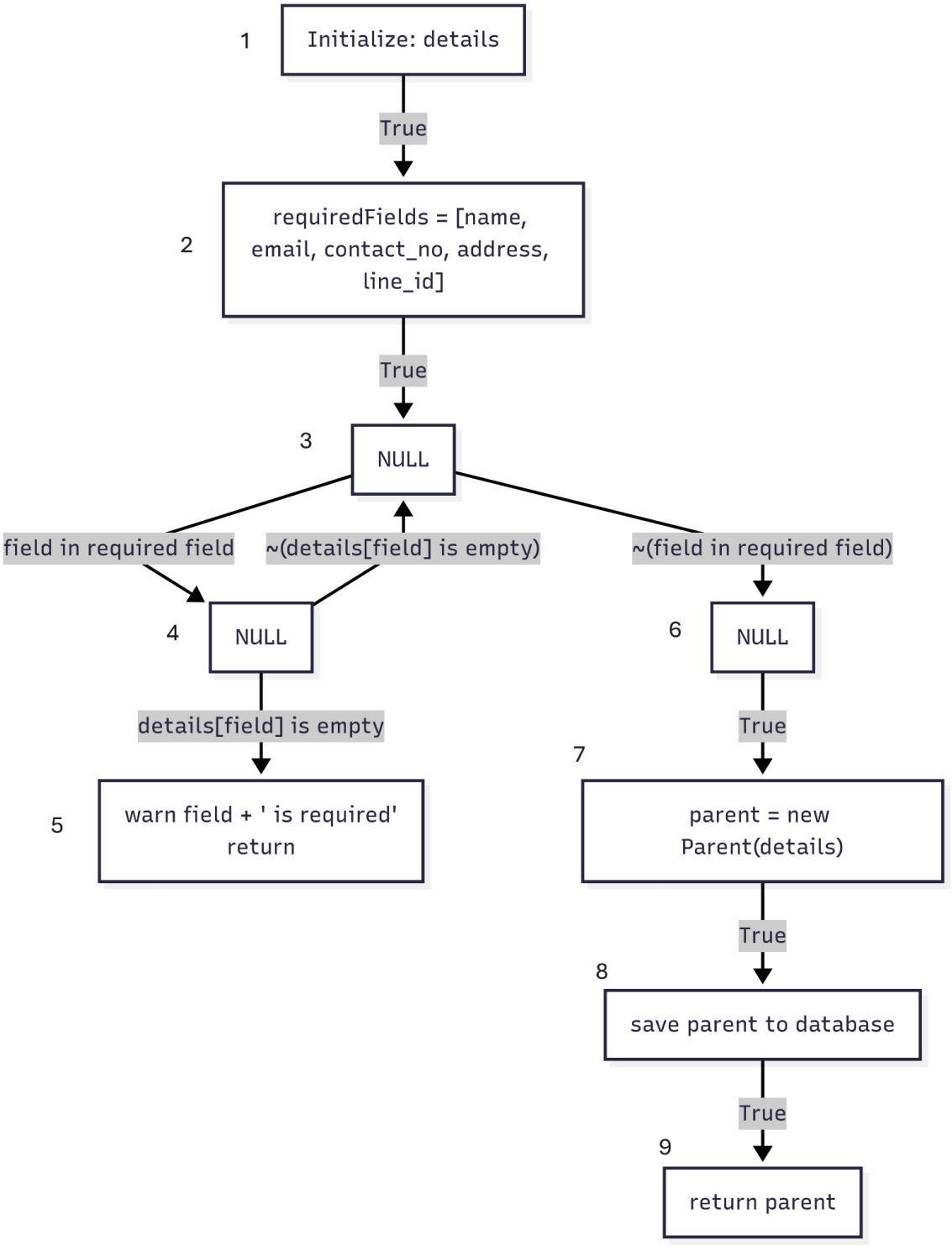


Figure 20 DFD of createParent

Predicates and p-use() Set of Edges

Edge	From Node	To Node	Predicate	P-Use Variables
(1,2)	1	2	True	{}
(2,3)	2	3	True	{name, email, contact_no, address, line_id}
(3,4)	3	4	field in required fields	{field, requiredFields}
(3,6)	3	6	~(field in required fields)	{field, requiredFields}
(4,5)	4	5	details[field] is empty	{details, field}
(4,3)	4	3	~(details[field] is empty)	{details, field}
(6,7)	6	7	True	{}
(7,8)	7	8	True	{details}
(8,9)	8	9	True	{parent}

4.createTeacher

name, email, password, photo, contact_no, line_id, address, confirm_password, courses: Defined by input, used in validation and to create teacher.

teacher: Defined after validation, used to save to database, returned.

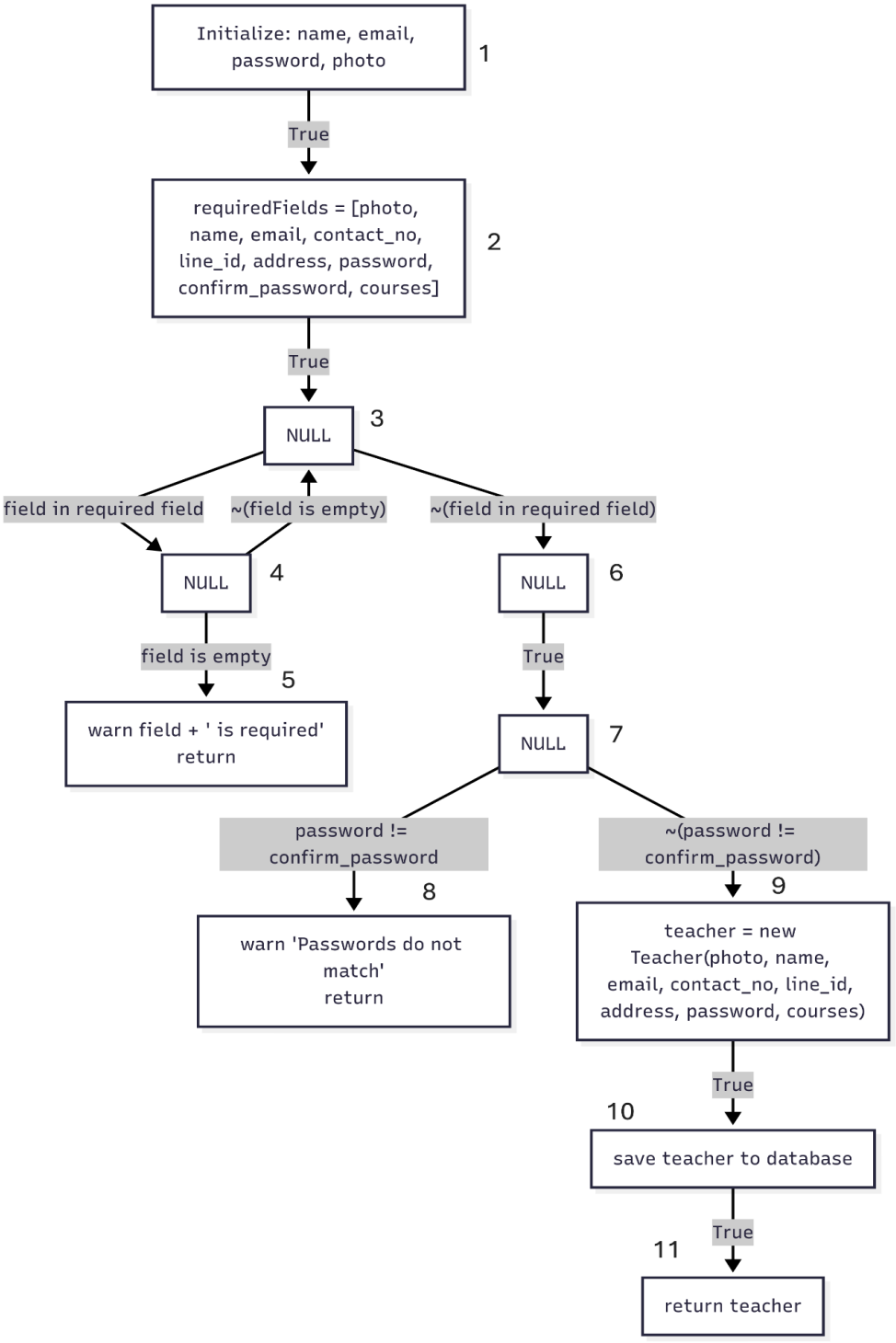


Figure 21 DFG of createTeacher

Predicates and p-use() Set of Edges

Edge	From Node	To Node	Predicate	P-Use Variables
(1,2)	1	2	True	{}
(2,3)	2	3	name is empty OR nickname is empty OR school is empty OR nationalID is empty OR dob is empty OR gender is empty	{name, nickname, school, nationalID, dob, gender}
(2,4)	2	4	~(name is empty OR nickname is empty OR school is empty OR nationalID is empty OR dob is empty OR gender is empty)	{name, nickname, school, nationalID, dob, gender}
(4,5)	4	5	nationalID exists in database	{nationalID}
(4,6)	4	6	~(nationalID exists in database)	{nationalID}
(6,7)	6	7	True	{}
(7,8)	7	8	True	{studentID, name, nickname, school, nationalID, dob, gender, phone, parent, allergic, doNotEat, allowPhotoAds}
(8,9)	8	9	True	{student}

Student Management

1.registerNewStudent

name, nickname, school, nationalID, dob, gender, phone, parent, allergic, doNotEat, allowPhotoAds: Defined by input, used in validation and saving.

studentID: Defined by generateStudentID, used to save student.

student: Defined in saveStudentToDatabase, used to save to database.

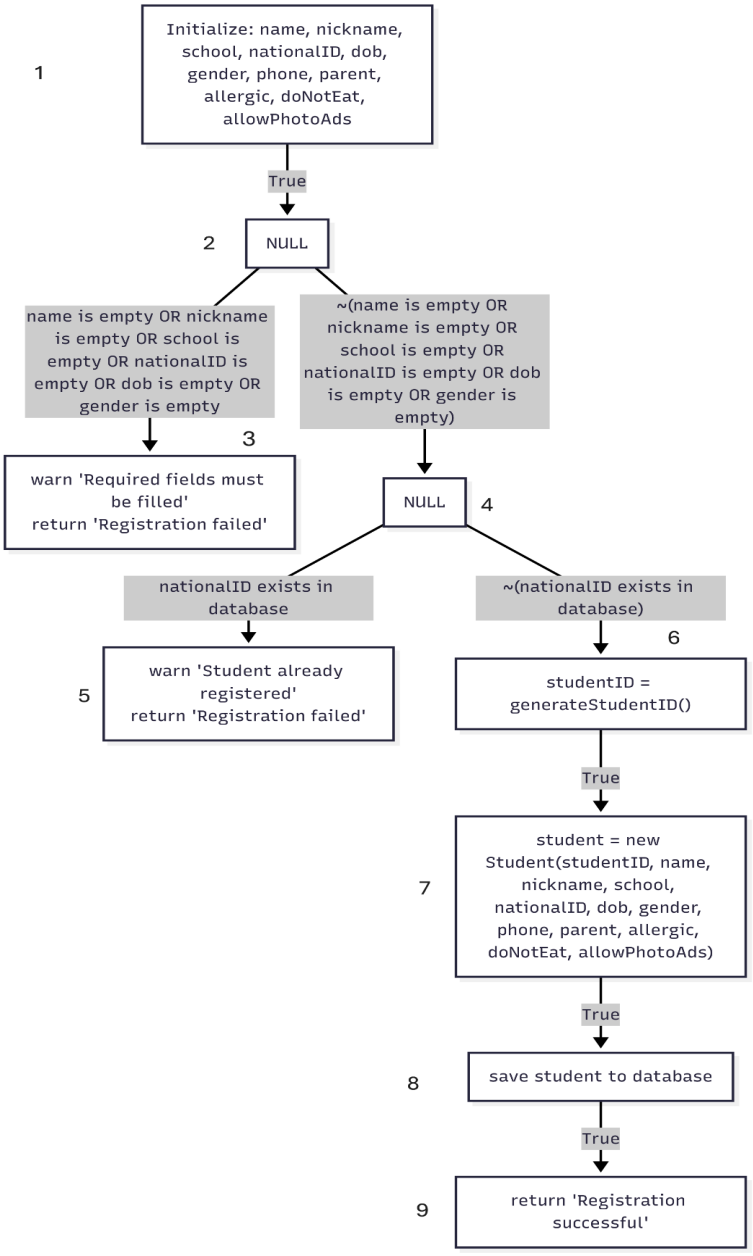


Figure 22 DFG of registerNewStudent

Predicates and p-use() Set of Edges

Edge	From Node	To Node	Predicate	P-Use Variables
(1,2)	1	2	True	{}
(2,3)	2	3	name is empty OR nickname is empty OR school is empty OR nationalID is empty OR dob is empty OR gender is empty	{name, nickname, school, nationalID, dob, gender}
(2,4)	2	4	~(name is empty OR nickname is empty OR school is empty OR nationalID is empty OR dob is empty OR gender is empty)	{name, nickname, school, nationalID, dob, gender}
(4,5)	4	5	nationalID exists in database	{nationalID}
(4,6)	4	6	~(nationalID exists in database)	{nationalID}
(6,7)	6	7	True	{}
(7,8)	7	8	True	{studentID, name, nickname, school, nationalID, dob, gender, phone, parent, allergic, doNotEat, allowPhotoAds}
(8,9)	8	9	True	{student}

2.registerNewParent

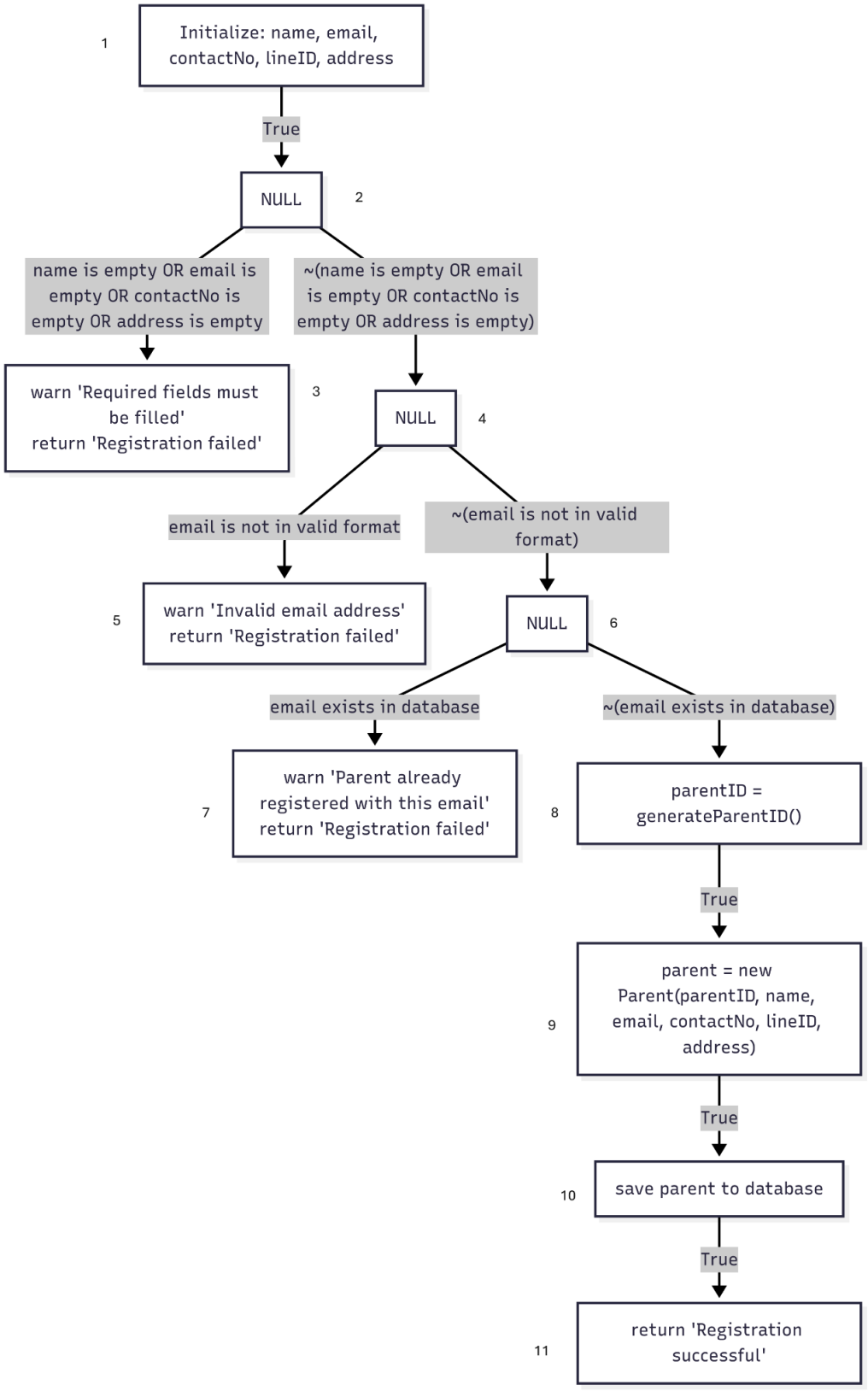


Figure 23 DFG of registerNewParent

Predicates and p-use() Set of Edges

Edge	From Node	To Node	Predicate	P-Use Variables
(1,2)	1	2	True	{}
(2,3)	2	3	name is empty OR email is empty OR contactNo is empty OR address is empty	{name, email, contactNo, address}
(2,4)	2	4	~(name is empty OR email is empty OR contactNo is empty OR address is empty)	{name, email, contactNo, address}
(4,5)	4	5	email is not in valid format	{email}
(4,6)	4	6	~(email is not in valid format)	{email}
(6,7)	6	7	email exists in database	{email}
(6,8)	6	8	~(email exists in database)	{email}
(8,9)	8	9	True	{}
(9,10)	9	10	True	{parentID, name, email, contactNo, lineID, address}
(10,11)	10	11	True	{parent}

Scheduling

1.createSchedule

date, startTime, endTime, additionalData: Defined by input, used in validation and to create schedule.

schedule: Defined after validation, used to save to repository.

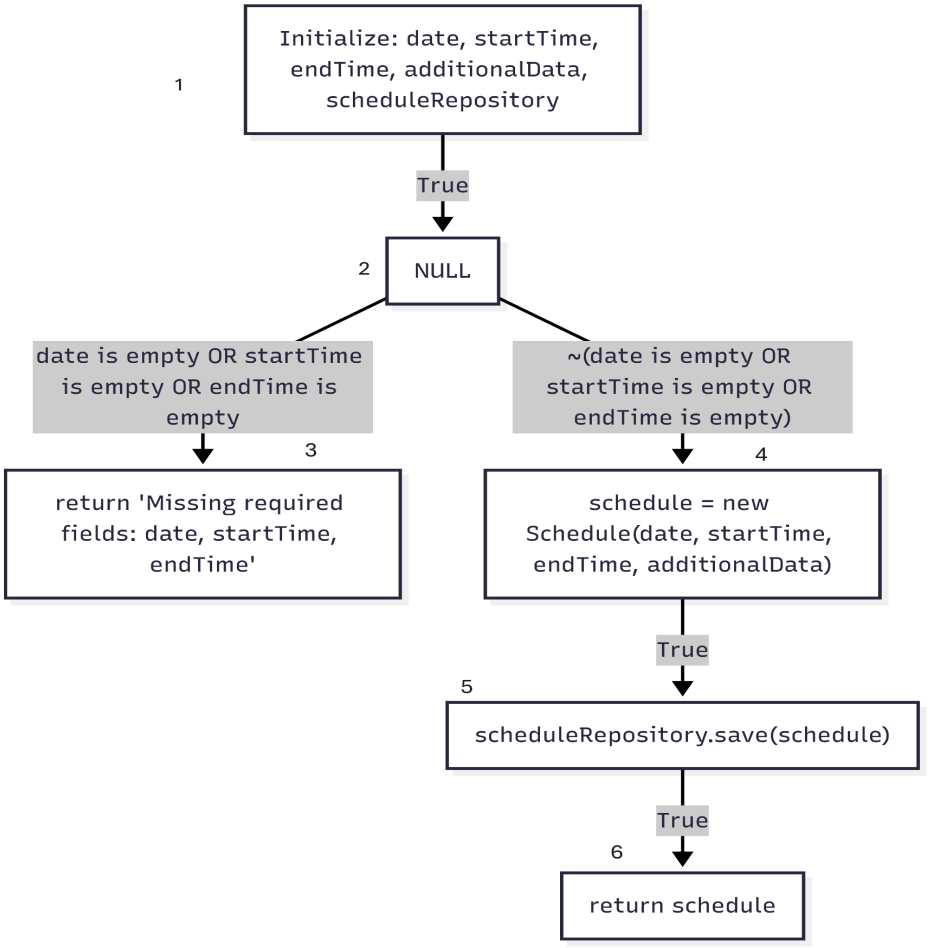


Figure 24 DFG of 1createSchedule

Predicates and p-use() Set of Edges

Edge	From Node	To Node	Predicate	P-Use Variables
(1,2)	1	2	True	{}
(2,3)	2	3	date is empty OR startTime is empty OR endTime is empty	{date, startTime, endTime}
(2,4)	2	4	~(date is empty OR startTime is empty OR endTime is empty)	{date, startTime, endTime}
(4,5)	4	5	True	{date, startTime, endTime, additionalData}
(5,6)	5	6	True	{schedule, scheduleRepository}

2.checkForConflicts

scheduleData: Used to extract fields for conflict checking.

existingSchedule: Defined by repository search, used to build conflict details.

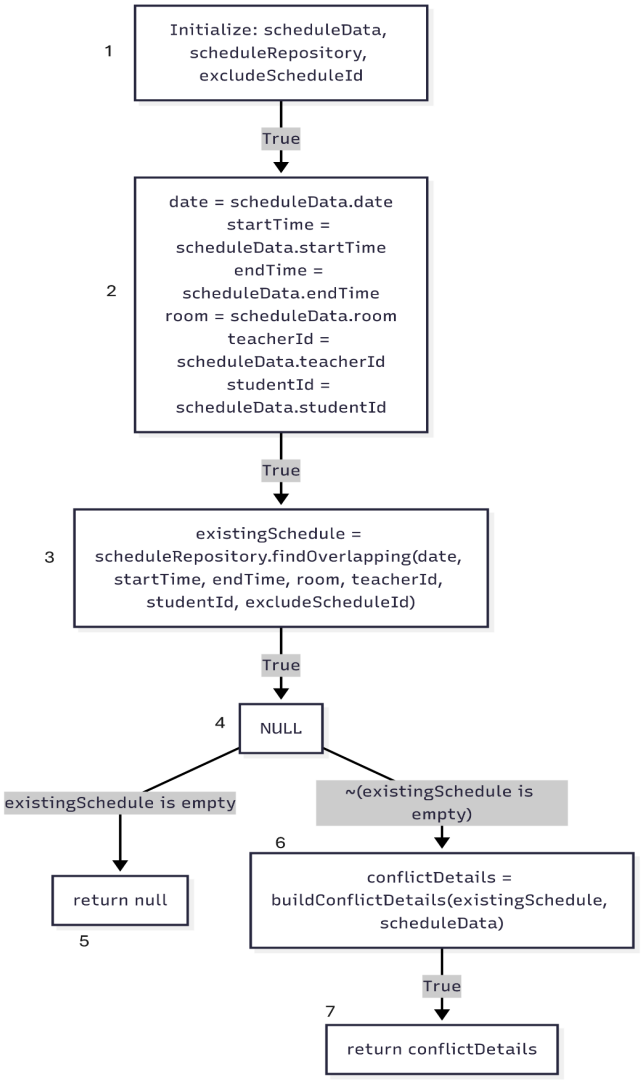


Figure 25 DFG of checkForConflicts

Predicates and p-use() Set of Edges

Edge	From Node	To Node	Predicate	P-Use Variables
(1,2)	1	2	True	{}
(2,3)	2	3	True	{scheduleData}
(3,4)	3	4	True	{date, startTime, endTime, room, teacherId, studentId, excludeScheduleId}
(4,5)	4	5	existingSchedule == null	{existingSchedule}
(4,6)	4	6	existingSchedule != null	{existingSchedule}
(6,7)	6	7	True	{existingSchedule, scheduleData}

Attendance and Feedback

1.logAttendanceAndFeedback

scheduleId, attendanceStatus, feedbackText: Defined by input, used to update schedule.

classInfo: Defined by lookup, used for validation.

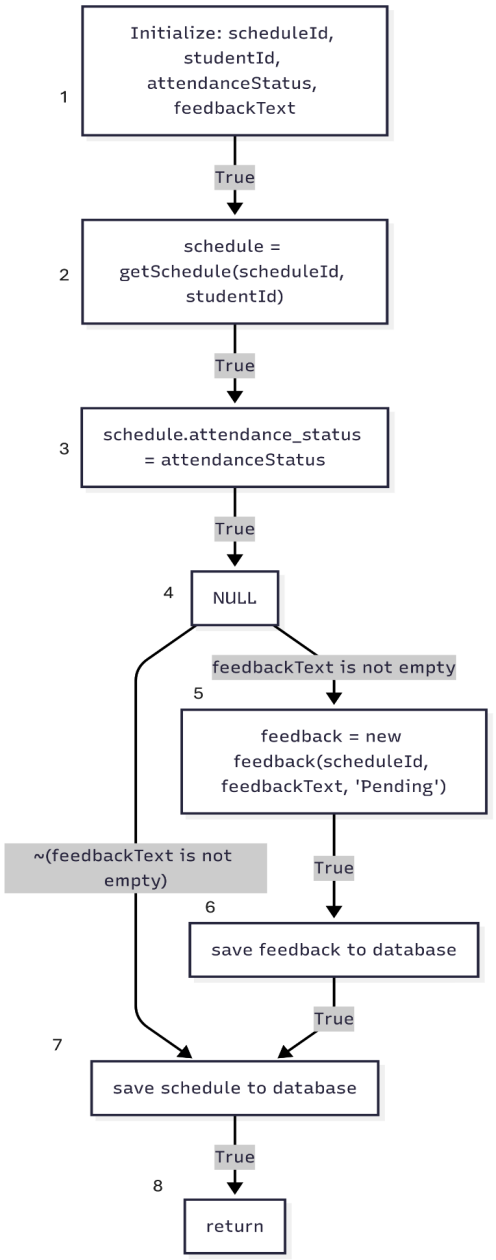


Figure 26 DFG of logAttendanceAndFeedback

Predicates and p-use() Set of Edges

Edge (i, j)	Predicate(i, j)	P-Use(i, j)
(1, 2)	True	{}
(2, 3)	True	{}
(3, 4)	True	{}
(4, 5)	feedbackText is not empty	{feedbackText}
(4, 7)	¬(feedbackText is not empty)	{feedbackText}
(5, 6)	True	{}
(6, 7)	True	{}
(7, 8)	True	{}

2.editFeedback

scheduleId, newFeedbackText: Defined by input, used to update schedule.

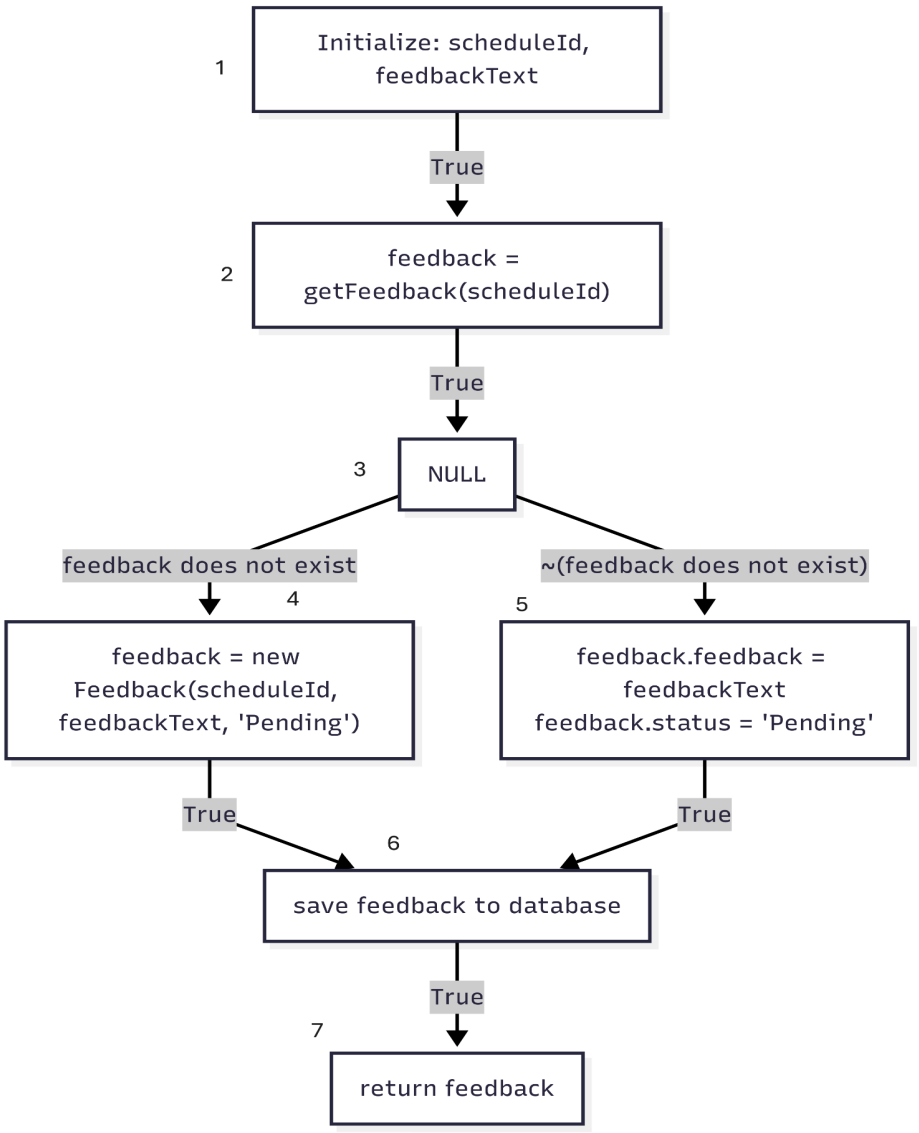


Figure 27 DFG of editFeedback

Predicates and p-use() Set of Edges

Edge (i, j)	Predicate(i, j)	P-Use(i, j)
(1, 2)	True	{}
(2, 3)	True	{}
(3, 4)	feedback does not exist	{feedback}
(3, 5)	¬(feedback does not exist)	{feedback}
(4, 6)	True	{}
(5, 6)	True	{}
(6, 7)	True	{}

Daily Overview

1.showTodaySchedule

todayDate: Defined by system, used to find schedules.

classesToday: Defined by lookup, used to display.

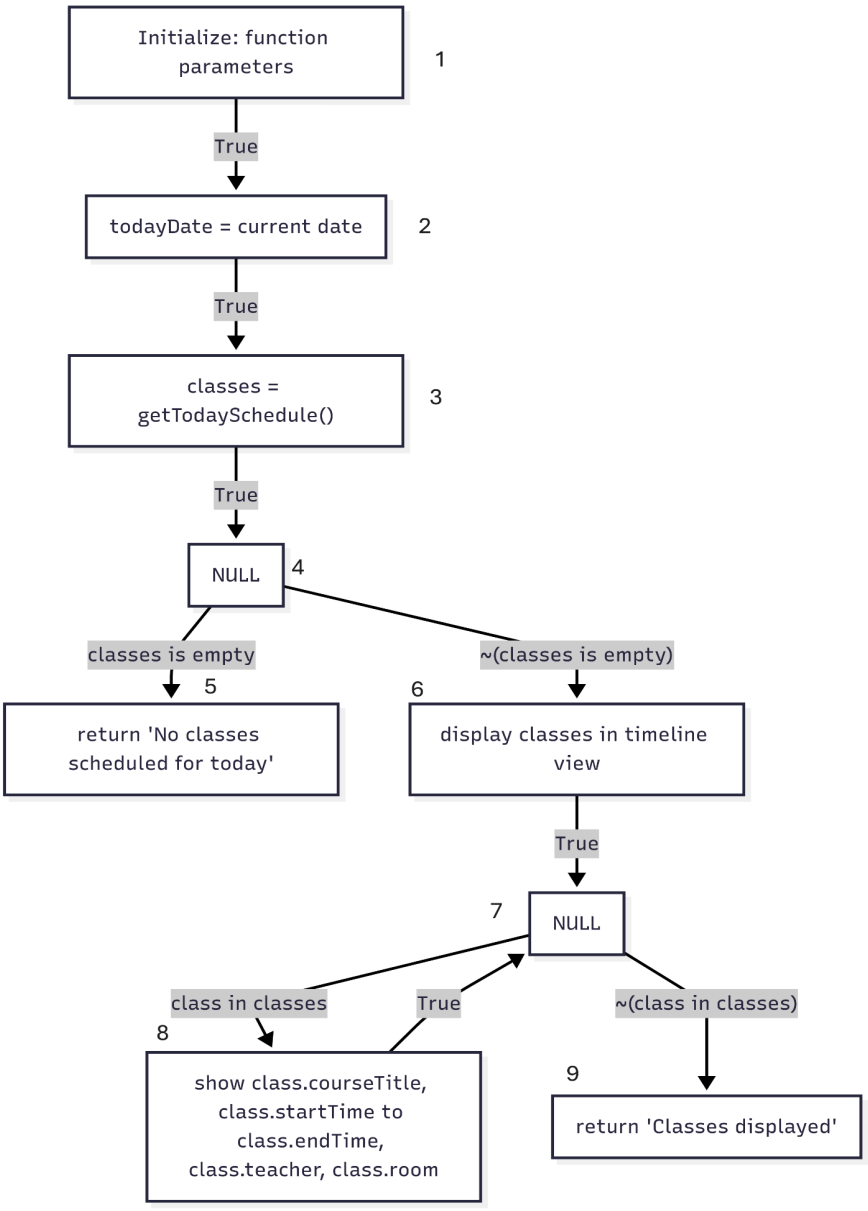


Figure 28 DFG of showTodaySchedule

Predicates and p-use() Set of Edges

Edge (i, j)	Predicate(i, j)	P-Use(i, j)
(1, 2)	True	{}
(2, 3)	True	{}
(3, 4)	True	{}
(4, 5)	classes is empty	{classes}
(4, 6)	¬(classes is empty)	{classes}
(5, exit)	True	{}
(6, 7)	True	{}
(7, 8)	class in classes	{class, classes}
(7, 9)	¬(class in classes)	{class, classes}
(8, 7)	True	{}
(9, exit)	True	{}

2.showAttendingStudents

classId: Defined by input, used to find schedule and students.

students: Defined by lookup, used to display.

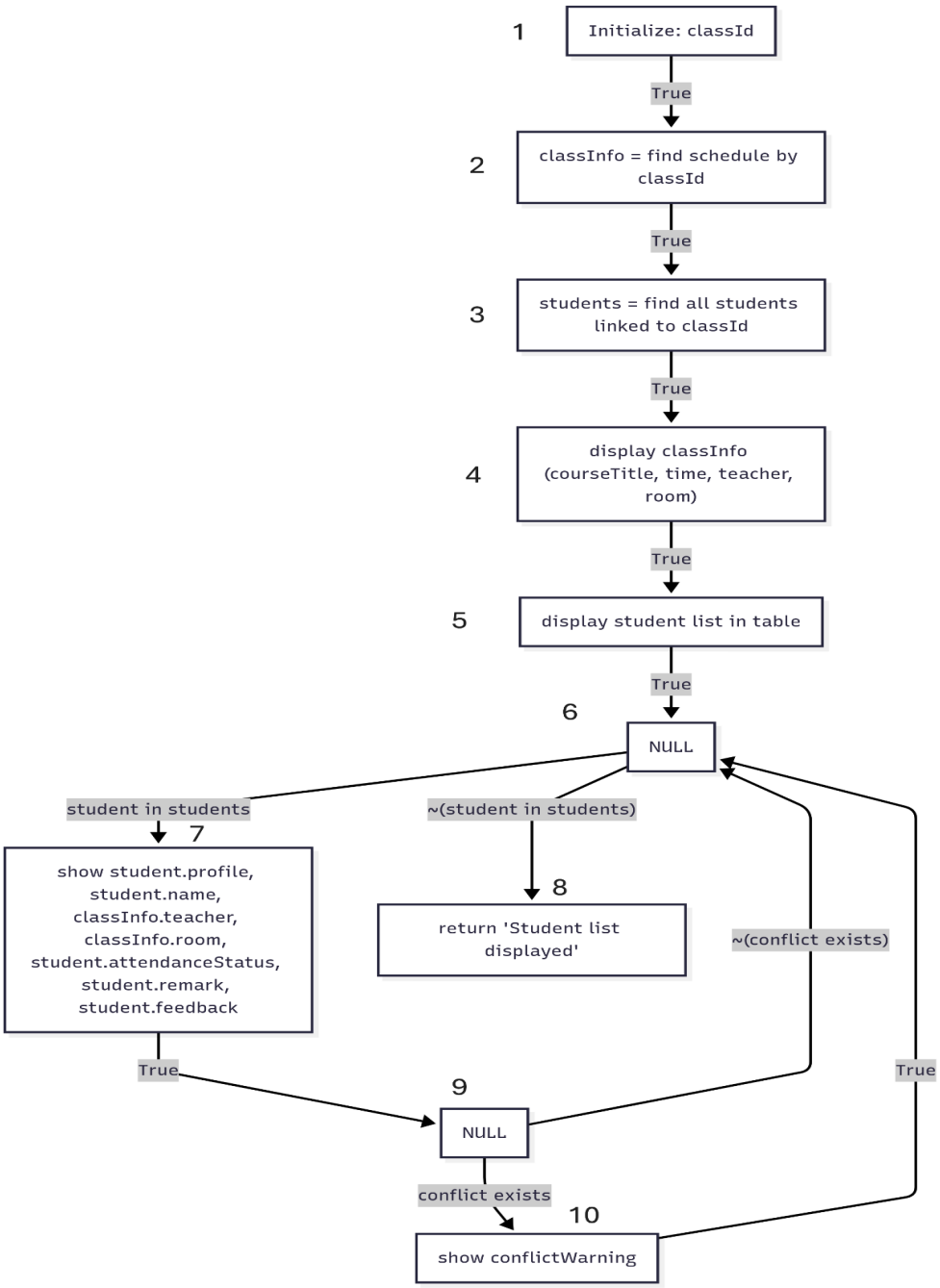


Figure 29 DFG of showAttendingStudents

Predicates and p-use() Set of Edges

Edge (i, j)	Predicate(i, j)	P-Use(i, j)
(1, 2)	True	{}
(2, 3)	True	{}
(3, 4)	True	{}
(4, 5)	True	{}
(5, 6)	True	{}
(6, 7)	student in students	{student, students}
(6, 8)	¬(student in students)	{student, students}
(7, 9)	True	{}
(8, 9)	True	{}
(9, 6)	¬(conflict exists)	{}
(9, 10)	conflict exists	{}
(10, 6)	True	{}

Security

1.login

email, password, role: Defined by input, used in validation and authentication.

userRecord: Defined by lookup, used for authentication and role-based access.

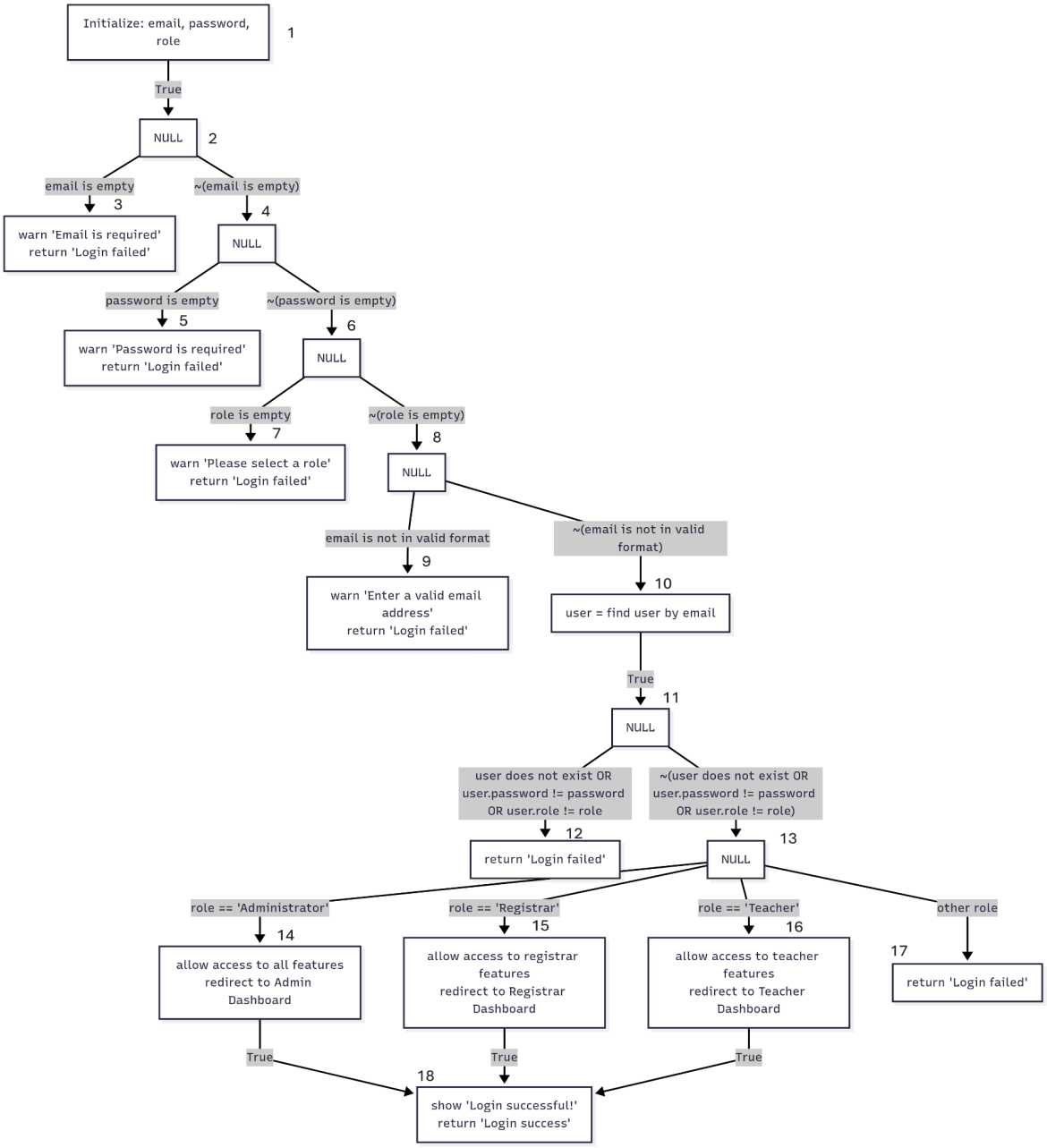


Figure 30 DFG of login

Predicates and p-use() Set of Edges

Edge (i, j)	Predicate(i, j)	P-Use(i, j)
(2, 3)	email is empty	{email}
(2, 4)	¬(email is empty)	{email}
(4, 5)	password is empty	{password}
(4, 6)	¬(password is empty)	{password}
(6, 7)	role is empty	{role}
(6, 8)	¬(role is empty)	{role}
(8, 9)	email is not in valid format	{email}
(8, 10)	¬(email is not in valid format)	{email}
(11, 12)	user does not exist OR user.password != password OR user.role != role	{user, password, role}
(11, 13)	¬(user does not exist OR user.password != password OR user.role != role)	{user, password, role}
(13, 14)	role == 'Administrator'	{role}
(13, 15)	role == 'Registrar'	{role}
(13, 16)	role == 'Teacher'	{role}
(13, 17)	other role	{role}

Domain Testing

Domain testing is a black box testing technique that focuses on testing input domains and boundary values without knowledge of internal implementation. It involves dividing the input space into equivalence classes, testing boundary values at the edges of valid ranges, validating invalid inputs for proper error handling, and selecting representative values from each partition to ensure comprehensive coverage while minimizing test cases. This approach treats functions as "black boxes" and is ideal for validating requirements and ensuring robust handling of edge cases.

createCourse

Objective: Verify course creation handles boundary values and invalid domains for input fields.

Steps:

1. Test with empty string for courseTitle (boundary).
2. Test with maximum length string for courseTitle (boundary).
3. Test with special characters in courseTitle.
4. Test with valid age range "5-10".
5. Test with invalid age range "abc" (invalid domain).
6. Test with medium as empty string (boundary).
7. Test with medium as valid value "online".

Expected Result:

- Steps 1, 5, 6: Function returns None and warns about required fields.
- Steps 2, 3, 4, 7: Function creates course successfully.

createStudent

Objective: Verify student creation handles boundary values for required fields.

Steps:

1. Test with empty name (boundary).
2. Test with name containing 1 character (boundary).
3. Test with name containing maximum allowed characters (boundary).
4. Test with valid phone number format.
5. Test with invalid phone number format (invalid domain).
6. Test with future date for date of birth (invalid domain).
7. Test with valid past date for date of birth.
8. Test with national_id as empty string (boundary).
9. Test with national_id containing special characters (invalid domain).

Expected Result:

- Steps 1, 5, 6, 8, 9: Function returns None and warns about invalid input.
- Steps 2, 3, 4, 7: Function creates student successfully.

createTeacher

Objective: Verify teacher creation handles boundary values for password validation.

Steps:

1. Test with password length = 0 (boundary).
2. Test with password length = 1 (boundary).
3. Test with password length = 8 (minimum valid boundary).
4. Test with password length = 50 (maximum valid boundary).
5. Test with matching passwords.
6. Test with passwords differing by 1 character (boundary).
7. Test with completely different passwords.
8. Test with email in invalid format (invalid domain).
9. Test with email in valid format.

Expected Result:

- Steps 1, 2, 6, 7, 8: Function returns None and warns about invalid input.
- Steps 3, 4, 5, 9: Function creates teacher successfully.

generateStudentID

Objective: Verify student ID generation handles boundary values for year, month, and sequence.

Steps:

1. Test with year = 00 (boundary).
2. Test with year = 99 (boundary).
3. Test with month = 01 (boundary).
4. Test with month = 12 (boundary).
5. Test with sequence = 001 (boundary).
6. Test with sequence = 999 (boundary).
7. Test ID uniqueness by generating multiple IDs.

Expected Result:

- All steps: Function returns valid string ID in correct format.
- Step 7: All generated IDs should be unique.

login

Objective: Verify login handles boundary values and invalid domains for authentication.

Steps:

1. Test with email as empty string (boundary).
2. Test with email containing only "@" symbol (invalid domain).
3. Test with valid email format.

4. Test with password as empty string (boundary).
5. Test with password containing only spaces (boundary).
6. Test with correct password.
7. Test with role as empty string (boundary).
8. Test with invalid role "InvalidRole" (invalid domain).
9. Test with valid roles: "Administrator", "Registrar", "Teacher".

Expected Result:

- Steps 1, 2, 4, 5, 7, 8: Function returns "Login failed".
- Steps 3, 6, 9: Function proceeds with authentication check.

checkForConflicts

Objective: Verify conflict detection handles boundary values for time and date ranges.

Steps:

1. Test with start time = end time (boundary).
2. Test with start time > end time (invalid domain).
3. Test with overlapping time by 1 minute (boundary).
4. Test with non-overlapping time by 1 minute (boundary).
5. Test with same date, different rooms.
6. Test with same date, same room, same time (conflict).

7. Test with different dates, same time and room.

Expected Result:

- Steps 1, 2: Function handles invalid time range appropriately.
- Step 3, 6: Function detects conflict.
- Steps 4, 5, 7: Function returns no conflict.

validateClassTypeData

Objective: Verify class type validation handles boundary values for different class modes.

Steps:

1. Test "12 times fixed" with 0 days selected (boundary).
2. Test "12 times fixed" with 1 day selected (boundary).
3. Test "12 times fixed" with 7 days selected (boundary).
4. Test "Camp class" with 0 dates (boundary).
5. Test "Camp class" with 4 dates (boundary - 1 less than required).
6. Test "Camp class" with 5 dates (exact requirement).
7. Test "Camp class" with 6 dates (boundary + 1 more than required).
8. Test with invalid class mode "InvalidMode" (invalid domain).

Expected Result:

- Steps 1, 4, 5: Function returns invalid with appropriate error message.
- Steps 2, 3, 6: Function returns valid.
- Step 7: Function behavior depends on business rules.
- Step 8: Function handles invalid mode appropriately.

Integration Testing

Integration testing is a software testing technique that verifies the interfaces and interactions between integrated software modules, focusing on detecting interface defects, data flow issues, and interaction problems that may not be apparent during individual unit testing. This testing approach is conducted manually by systematically combining tested modules into larger subsystems, testing data passing between components, validating end-to-end workflows, and verifying error handling across module boundaries. Testers execute sequential function calls with real test data to ensure the integrated system produces expected results, covering both successful integration scenarios and failure conditions where modules must properly handle errors from dependent components.

Course and Student Integration

Objective: Verify that course creation and student enrollment work together correctly.

Steps:

1. Create a new course using createCourse function.
2. Create a new student using createStudent function.

3. Add the course to the student using addCourseToStudent function.
4. Verify the course appears in the student's enrolled courses.
5. Verify the student appears in the course's enrolled students list.

Expected Result:

- All steps complete successfully with proper data linking between course and student records.

Student Registration and Parent Association

Objective: Verify that student and parent creation integrate properly with database relationships.

Steps:

1. Create a new parent using createParent function.
2. Create a new student with the parent's ID using createStudent function.
3. Verify the parent-student relationship is established in the database.
4. Query parent record and verify student is listed as child.
5. Query student record and verify parent information is correct.

Expected Result:

- Parent-student relationship is properly established and queryable from both directions.

Teacher Creation and Course Assignment

Objective: Verify that teacher creation and course assignment integration works correctly.

Steps:

1. Create a new course using createCourse function.
2. Create a new teacher using createTeacher function with the course assigned.
3. Verify the teacher is assigned to the course.
4. Verify the course shows the teacher as instructor.
5. Create a schedule with the teacher and course.

Expected Result:

Teacher-course relationship is established and schedule creation uses correct associations.

Schedule Creation and Conflict Detection

Objective: Verify that schedule creation integrates properly with conflict detection system.

Steps:

1. Create a teacher using createTeacher function.
2. Create a student using createStudent function.
3. Create a schedule using createSchedule with specific room, teacher, and time.
4. Attempt to create another schedule with same room, teacher, and overlapping time.
5. Verify that checkForConflicts function detects the conflict.
6. Verify both schedules show warning.

Expected Result:

- Conflict detection system properly integrates with schedule creation to prevent double-booking.

Authentication and Role-Based Access

Objective: Verify that login authentication integrates with role-based access control.

Steps:

1. Create a registrar using createRegistrar function.
2. Login using the registrar credentials with login function.
3. Attempt to access admin-only functions (should fail).
4. Attempt to access registrar functions (should succeed).
5. Logout and login as different role.
6. Verify role-specific access permissions.

Expected Result:

- Authentication integrates with authorization to properly restrict access based on user roles.

Student ID Generation and Database Storage

Objective: Verify that student ID generation integrates with database storage and uniqueness constraints.

Steps:

1. Generate multiple student IDs using generateStudentID function.
2. Create students using createStudent with generated IDs.
3. Verify all IDs are unique in the database.

4. Attempt to create student with duplicate ID (should fail).
5. Verify database maintains referential integrity.

Expected Result:

- ID generation integrates with database to ensure uniqueness and data integrity.

Schedule and Attendance Integration

Objective: Verify that schedule creation integrates with attendance logging system.

Steps:

1. Create a complete class setup (course, teacher, student, schedule).
2. Use logAttendanceAndFeedback to mark attendance for the scheduled class.
3. Verify attendance is recorded against the correct schedule.
4. Edit feedback using editFeedback function.
5. Verify feedback updates are linked to the correct schedule and student.

Expected Result:

- Schedule system integrates with attendance to maintain accurate records.

Daily Overview and Data Aggregation

Objective: Verify that daily overview functions integrate with all data sources.

Steps:

1. Create multiple schedules for today's date.
2. Create students and assign them to today's classes.
3. Use showTodaySchedule to display today's classes.
4. Use showAttendingStudents for each class.

5. Verify all data is correctly aggregated and displayed.
6. Mark some attendance and verify updates appear in overview.

Expected Result:

- Daily overview functions properly integrate with schedule, student, and attendance data.

Course Package

Objective: Verify that course package assignment integrates with student records

Steps:

1. Create a student using createStudent function.
2. Add a course package using addPackageToStudent function.
3. Create schedules for the package classes.
4. Verify package details are correctly stored.
5. Verify remaining classes are tracked correctly.
6. Simulate class attendance and verify package usage updates.

Expected Result:

- Course package system integrates with student records and class tracking.

End-to-End Registration Workflow

Objective: Verify complete student registration workflow integration.

Steps:

1. Create parent using registerNewParent function.
2. Create student using registerNewStudent function with parent association.

3. Add course package to student.

4. Create schedule entries for the student's classes.

5. Verify all data relationships are correct across all systems.

6. Test data retrieval from different system components.

Expected Result:

- Complete registration workflow integrates all components seamlessly with proper data relationships.

Use case diagram

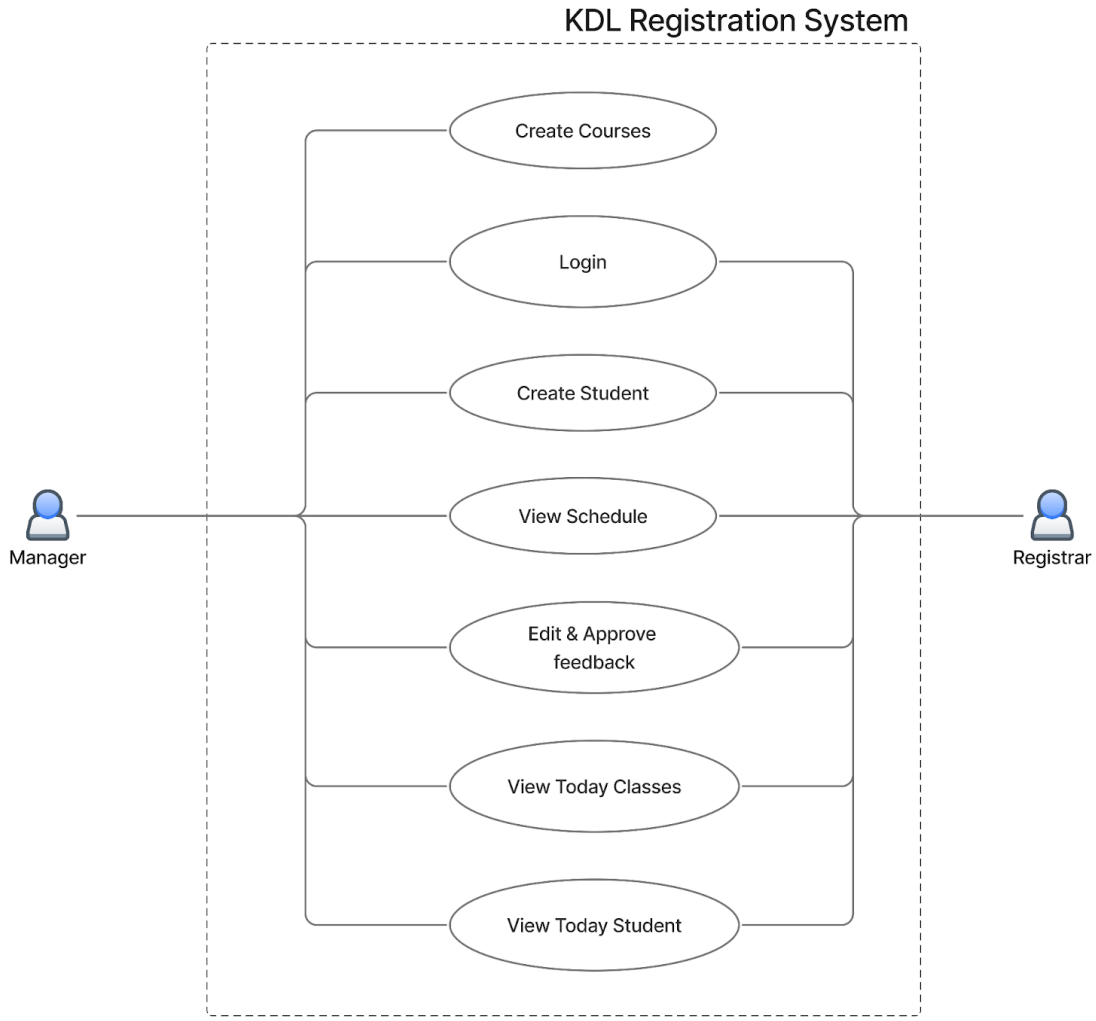


Figure 31 Use Case Diagram 1

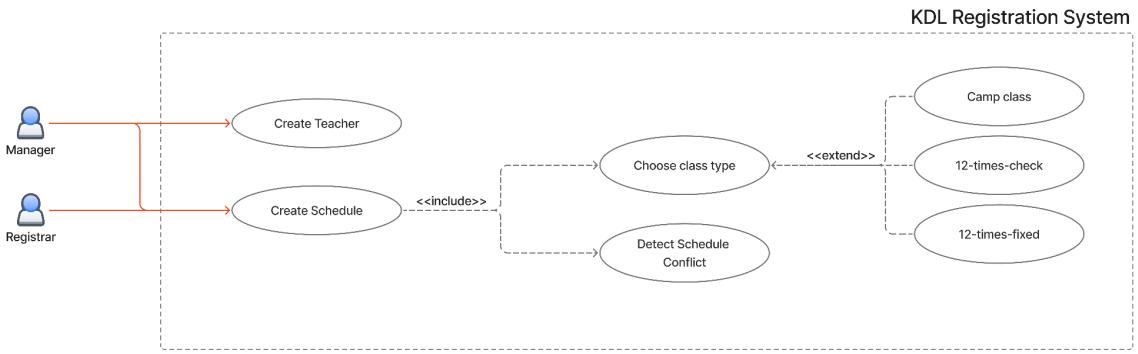


Figure 32 Use Case Diagram 2

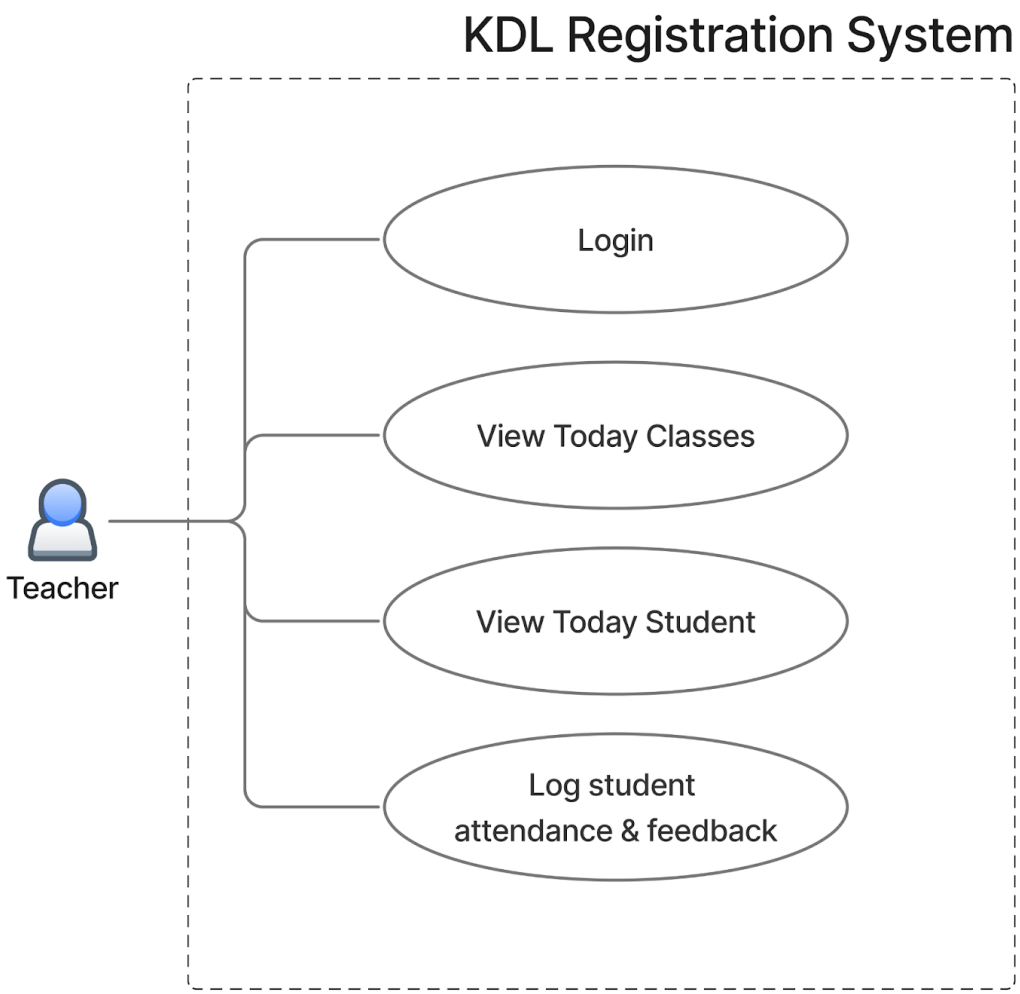


Figure 33 Use Case Diagram 3

User Acceptance Testing

UAT (User Acceptance Testing) for Kiddee Lab is done to ensure the product meets the needs of its end users, aligns with the specified requirements, and functions properly in real-world scenarios. It helps identify any issues, validate compliance, and improve the user experience before the system goes live. UAT ensures that Kiddee Lab is intuitive, user-friendly, and ready for production, minimizing risks of post-launch problems.

Coverage matrix

Test by	Test Case Idendifier	Requirement Identifier												
		mgt001	mgt002	std001	std002	sked001	sked002	sked003	sked004	atn001	atn002	util001	util002	sec001
Mod (Manager)	1	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☑
	2	☑	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
	3	☐	☑	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Penguine (Registrar)	4	☐	☐	☑	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
	5	☐	☐	☐	☐	☑	☑	☑	☐	☐	☐	☐	☐	☐
	6	☐	☐	☐	☑	☐	☐	☐	☐	☐	☐	☐	☐	☐
	7	☐	☐	☐	☐	☑	☑	☑	☐	☐	☐	☑	☑	☐
	8	☐	☐	☐	☐	☑	☑	☑	☐	☐	☐	☐	☐	☐
	9	☐	☐	☐	☐	☐	☐	☐	☐	☐	☑	☑	☑	☐
	10	☐	☐	☐	☐	☐	☐	☐	☑	☐	☐	☐	☐	☐
Flexi (Teacher)	11	☐	☐	☐	☐	☐	☐	☐	☐	☑	☐	☑	☑	☐

Figure 34 Coverage matrix

Test Plan							
Program ID:			Version number:				
Tester:	Mod		Date designed	7 Sep 2025	Date conducted	8 Sep 2025	
Result:	TRUE	Passed			Pre-condition	All users' account is created.	
Test ID:	3	Requirement addressed:	sec001 The system shall implement role base access control for authentication.				
Objective:	Each user can only use the functions they have permission to.						

Test cases		
Interface ID	Expected result	actual result
Admin	Today, Schedule, Courses, New Courses, Add Course Package, Students, New Students, Teachers, New Teacher, Parents, New Parents, Registrar, New Registrar, Fee & Discount, New Discount/Fees, Feedback, Create. Invoice, Invoices & Receipts	as expected result.
Registrar	Today, Schedule, Courses, Add Course Package, Students, New Students, Teachers, New Teacher, Parents, New Parents, Feedback, Create. Invoice, Invoices & Receipts	as expected result.
Teacher	Today, My Schedule	as expected result.

Script

Test Plan							
Program ID:			Version number:				
Tester:	Mod		Date designed	6 Sep 2025	Date conducted	8 Sep 2025	
Result:	TRUE	Passed			Pre-condition		
Test ID:	2	Requirement addressed:	mgt001: The system shall provide the manager with the ability to create new courses.				
Objective:	New course can be created without fail.						
Test cases							
Interface ID	Data Field	Value Entered	Expected result	actual result			
Create New Course	Course Title	Test Course	Test Course with details is created.	as expected result.			
	Description	Test Description					
	Age Range	5-6 yrs					
	Learning Medium	iPad					
Create New Course	Course Title		Warn user to input required fields such as course title, description, age range and learning medium	as expected result.			
	Description						
	Age Range						
	Learning Medium						
Script							

Test Plan							
Program ID:			Version number:				
Tester:	Mod		Date designed	6 Sep 2025	Date conducted	8 Sep 2025	
Result:	TRUE	Passed			Pre-condition		
Test ID:	3	Requirement addressed:	mgt002: The system shall provide the manager with the ability to create new registrar.				
Objective:	To create new registrar account and the registrar can login using the cretentials.						

Test cases				
Interface ID	Data Field	Value Entered	Expected result	actual result
New Registrar	Photo	image.png	Test Registrar account is created.	as expected result.
	Name	Test Registrar		
	Email	testregistrar@test.com		
	Password	password123		
New Registrar	Photo		Warn user to input required fields such as name, email and password.	as expected result.
	Name			
	Email			
	Password			

Script

Test Plan							
Program ID:			Version number:				
Tester:	Penguine		Date designed	6 Sep 2025	Date conducted	8 Sep 2025	
Result:	TRUE	Passed			Pre-condition	Related parent should be created.	
Test ID:	4	Requirement addressed:	std001 The system shall provide the registrar with the ability to register new students.				
Objective:	Registar can create new student.						
Test cases							
Interface ID	Data Field	Value Entered	Expected result	actual result			
New Student	Photo	image.png	Test Student with student ID 2025090001 is created with details.	as expected result.			
	Name	Test Student					
	Nickname	test student					
	DOB	12 March 2025					
	National ID	1234567890123					
	Gender	Male					
	Phone	1234567890					
	School	ABC					
	Allergic						
	Do Not Eat						
	Parent	Test Parent					
	Allow to use photo for ads	FALSE					
New Student	Photo		Warn user to input required fields such as	as expected result.			
	Name						

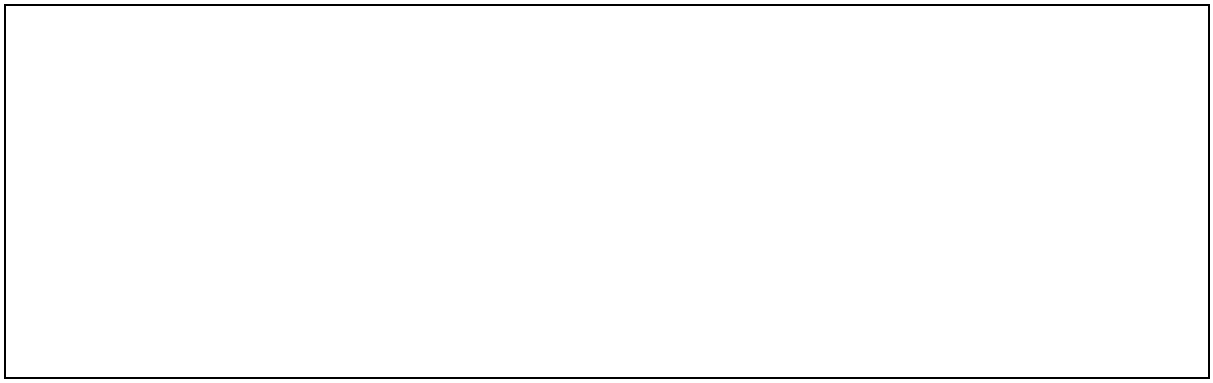
	Nickname		name, nickname, dob, gender, phone and school.	
	DOB			
	National ID			
	Gender			
	Phone			
	School			
	Allergic			
	Do Not Eat			
	Parent			
	Allow to use photo for ads	FALSE		
Script				

Test Plan							
Program ID:			Version number:				
Tester:	Penguine		Date designed	6 Sep 2025	Date conducted	8 Sep 2025	
Result:	TRUE	Passed			Pre-condition	The course must be created.	
Test ID:	5	Requirement addressed:	sked001 The system shall provide the register with the ability to view existing schedules as well as create new schedules. sked002 The system shall be able to show warning if the schedule has conflict. sked003 The system shall support the following class types: 12 times fixed,12 times check ,Camp class				
Objective:	Testing 12 time fixed class types. New courses can be added to existing student. Three class types schedule are supported. Warning must be show accordingly.						

Test cases				
Interface ID	Data Field	Value Entered	Expected result	actual result
Students >student >add course	target student	A	In Confirm Class Schedule, a list of class schedule for Tinkamo TInker Intermediate II for the target student A is created for every Mon, Tue, Wed 9:00 to 12:00 with Tinkamo Teacher at Tinkamo room with no conflict warning. After Confirm Tinkamo Tinkerer Intermediate II is added to the target student.	as expected result.
	Course Name	Tinkamo Tinkerer Intermediate II		
	Class Type	12 times fixed		
	Select Days	Mon, Tue, Wed		
	Start Time	09:00		
	End Time	12:00		
	Teacher	Tinkamo Teacher		
	Room	Tinkamo		
	Remark			
Students >student >add course	target student	B	In Confirm Class Schedule, a list of class schedule for Tinkamo TInker Intermediate II	as expected result.
	Course Name	Tinkamo Tinkerer Intermediate II		

	Class Type	12 times fixed	for the target student B is created for every Wed, Thurs, Fri 11:00 to 14:00 with Tinkamo Teacher at Tinkamo room with room and teacher conflict warning. After changing room, room conflict waring is resolved. After changing teacher, teacher conflict warning is resolved. After Confirm Tinkamo Tinkerer Intermediate II is added to the target student.	
	Select Days	Wed, Thurs, Fri		
	Start Time	11:00		
	End Time	14:00		
	Teacher	Tinkamo Teacher		
	Room	Tinkamo		
	Remark			
Script				

Test Plan							
Program ID:			Version number:				
Tester:	Penguine		Date designed	6 Sep 2025	Date conducted	8 Sep 2025	
Result:	TRUE	Passed			Pre-condition		
Test ID:	6	Requirement addressed:	std002 The system shall provide the registrar with the ability to register new parents.				
Objective:	To create new parent.						
Test cases							
Interface ID	Data Field	Value Entered	Expected result	actual result			
New Parent	Photo	image.png	Test parent is created.	as expected result.			
	Name	Test Parent					
	Email	testparent@test.com					
	Contact No	1234567890					
	Line ID	1234567890					
	Address	test address					
New Parent	Photo		Warn user to input required fields such as name, email, contact no and address	as expected result.			
	Name						
	Email						
	Contact No						
	Line ID						
	Address						
Script							

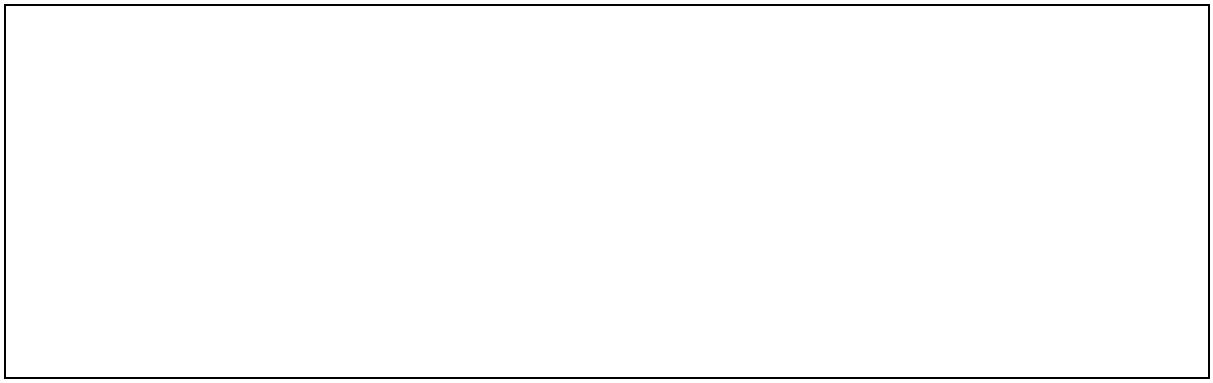


Test Plan							
Program ID:			Version number:				
Tester:	Penguine		Date designed	6 Sep 2025	Date conducted	8 Sep 2025	
Result:	TRUE	Passed			Pre-condition	The course must be created.	
Test ID:	7	Requirement addressed:	sked001 The system shall provide the register with the ability to view existing schedules as well as create new schedules. sked002 The system shall be able to show warning if the schedule has conflict. sked003 The system shall support the following class types: 12 times fixed,12 times check ,Camp class util001 The system shall display the classes scheduled for today. util002 The system shall display the list of students attending today's class.				
Objective:	Testing 12 time check class types. Modify one schedule to show on Today feature.						

Test cases				
Interface ID	Data Field	Value Entered	Expected result	actual result
Students >student >add course	target student	C	In Confirm Class Schedule, a list of class schedule for Fun Coding with mBot Beginner for target student C is created for 12 times with the status of TBD (To be determined) in the following fields: Date, Time, Teacher, Room. After Confirm Fun Coding with mBot Beginner is added to the target student. After Modifying one schedule to match date of today, the class schedule is displayed in	as expected result.
	Course Name	Fun Coding with mBot Beginner		
	Class Type	12 times check		

			Today feature. Attending student list == Target Student C.	
Script				

Test Plan							
Program ID:			Version number:				
Tester:	Penguine		Date designed	7 Sep 2025	Date conducted	8 Sep 2025	
Result:	TRUE	Passed			Pre-condition	The course must be created.	
Test ID:	8	Requirement addressed:	sked001 The system shall provide the register with the ability to view existing schedules as well as create new schedules. sked002 The system shall be able to show warning if the schedule has conflict. sked003 The system shall support the following class types: 12 times fixed,12 times check ,Camp class				
Objective:	Testing camp class types. New courses can be added to existing student. Three class types schedule are supported. Warning must be show accordingly.						
Test cases							
Interface ID	Data Field	Value Entered			Expected result		actual result
Students >student >add course	target student	D			In Confirm Class Schedule, a list of class schedule for Tinkamo Tinkerer Beginner for the target student D is created at the input Date from 10:00 to 4:00 with Tinkamo Teacher at Tinkamo room with no conflict warning. After Confirm Tinkamo Tinkerer Begineer is added to the target student.		as expected resutl.
	Course Name	Tinkamo Tinkerer Begineer					
	Class Type	5 Day Camp					
	Select Date	Any 5 Days					
	Start Time	09:00					
	End Time	16:00					
	Teacher	Tinkamo Teacher					
	Room	Tinkamo					
	Remark						
Script							



Test Plan							
Program ID:			Version number:				
Tester:	Penguine		Date designed	7 Sep 2025	Date conducted	8 Sep 2025	
Result:	TRUE	Passed			Pre-condition	Schedule and feedback must exit.	
Test ID:	9	Requirement addressed:	atn002 The system shall provide the registrar with the ability to edit student feedback. util001 The system shall display the classes scheduled for today. util002 The system shall display the list of students attending today's class.				
Objective:	Today's schedule are displayed at today ferature. Students that will be attending today's class are listed according to the course. Teacher can check attendance for each student and add feedback. After feedback is appended, the registrar can edit and/or approve feedback.						
Test cases							
Interface ID	Data Field		Value Entered		Expected result		actual result
Registrar UI > Feedback	target feedback		Student is a slow learner.		The targeted student's feedback is changed to "Student need more time to study and should study at home too.". The concerned feedback disappears.		as expected result.
	Feedback		Student need more time to study and should study at home too.				
	Save & Approve		TRUE				
Script							

Test Plan							
Program ID:			Version number:				
Tester:	Penguine		Date designed	7 Sep 2025	Date conducted	8 Sep 2025	
Result:	TRUE	Passed			Pre-condition	The course must be created.	
Test ID:	10	Requirement addressed:	sked004 The system shall provide the registrar with the ability to add package course to a student.				
Objective:	Add package course to a student and create invoice for package course.						

Test cases				
Interface ID	Data Field	Value Entered	Expected result	actual result
Courses	target student	Test Student	Two TBC is added to target student.	as expected result.
	Course Package	2 Courses Package		

Script

Test Plan							
Program ID:			Version number:				
Tester:	Flexi		Date designed	7 Sep 2025	Date conducted	8 Sep 2025	
Result:	TRUE	Passed			Pre-condition	Schedule must exit.	
Test ID:	11	Requirement addressed:	atn001 The system shall provide the teacher with the ability to log student attendance and feedback. util001 The system shall display the classes scheduled for today. util002 The system shall display the list of students attending today's class.				
Objective:	Today's schedule are displayed at today's feature. Students that will be attending today's class are listed according to the course. Teacher can check attendance for each student and add feedback. After feedback is appended, the registrar can edit and/or approve feedback.						
Test cases							
Interface ID	Data Field		Value Entered		Expected result	actual result	
Teacher UI > Today	target student		Any student that is listed.		The targeted student's attendance is changed to "Completed". Feedback is appended and ready for approval.	as expected result.	
	Attendance		Completed				
	Feedback		Student is a slow learner.				
Script							

Access Control List

Users are granted access only to the pages and features for which they have appropriate permissions. Below is the Access Control List (ACL) outlining the specific access rights.

Page	Feature	Manger	Registrar	Teacher
Login Page	All	☑	☑	☑
Today	Check Schedule	☑	☑	☑
	Check Attendance	☑	☑	☑
	Add Feedback	☐	☐	☑
Schedule	All	☑	☑	☑
Courses	All features except	☑	☑	☐
	Create new course	☑	☐	☐
Students	All	☑	☑	☐
Teacher	All	☑	☑	☐
Parents	All	☑	☑	☐
Registrar	All	☑	☐	☐
Feedback	All	☑	☑	☐

User Interface & User Experience

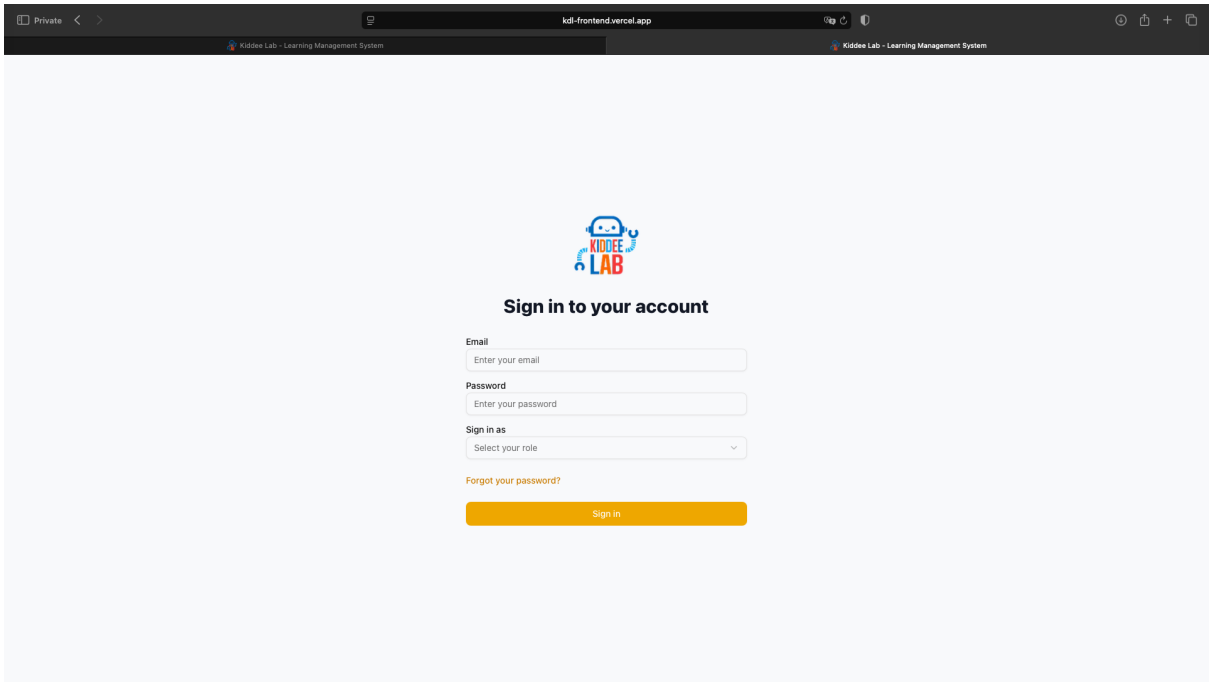
The system includes validation mechanisms to ensure all required fields are completed correctly. If any input is missing or in an incorrect format, the system will display an appropriate error message.

To enhance readability and avoid redundancy, user interfaces that are identical or similar across roles will be presented only once in this document, provided users have access to the same functionality.

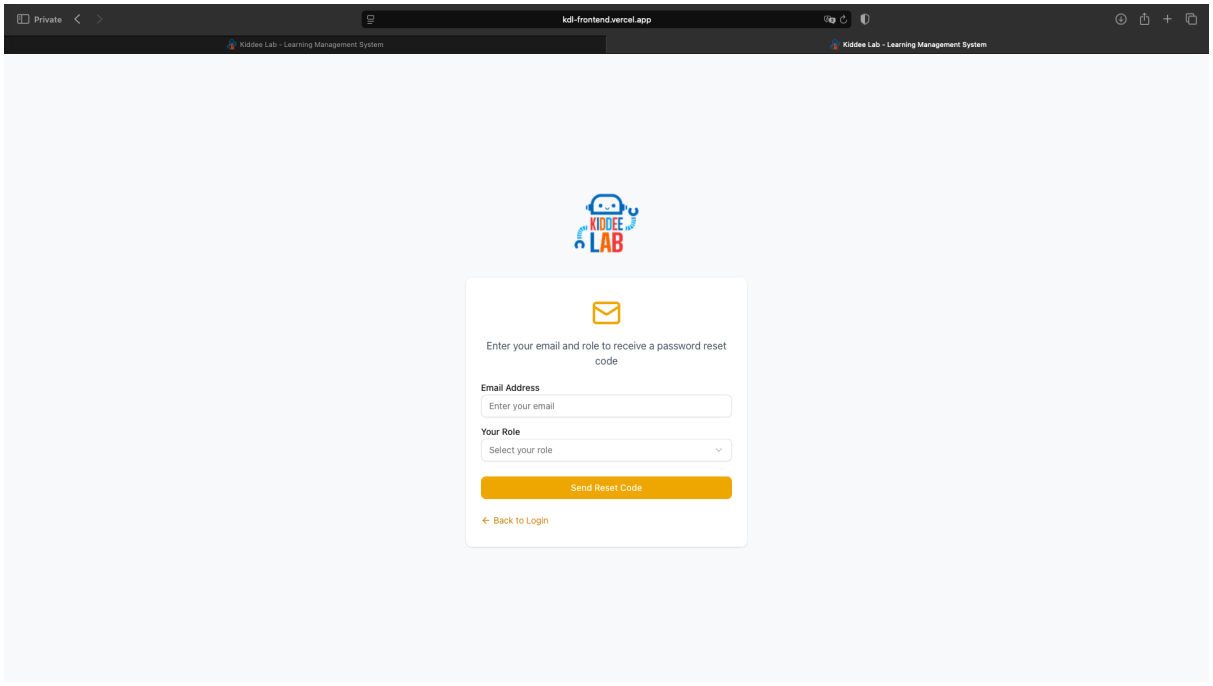
Log in Page

Once users access the system via www.registrar.kiddeelab.co.th, they are required to sign in based on their assigned roles. A "Forgot Password" feature is also available for users who have forgotten or wish to reset their passwords.

Sign in page screen



Forgot password screen

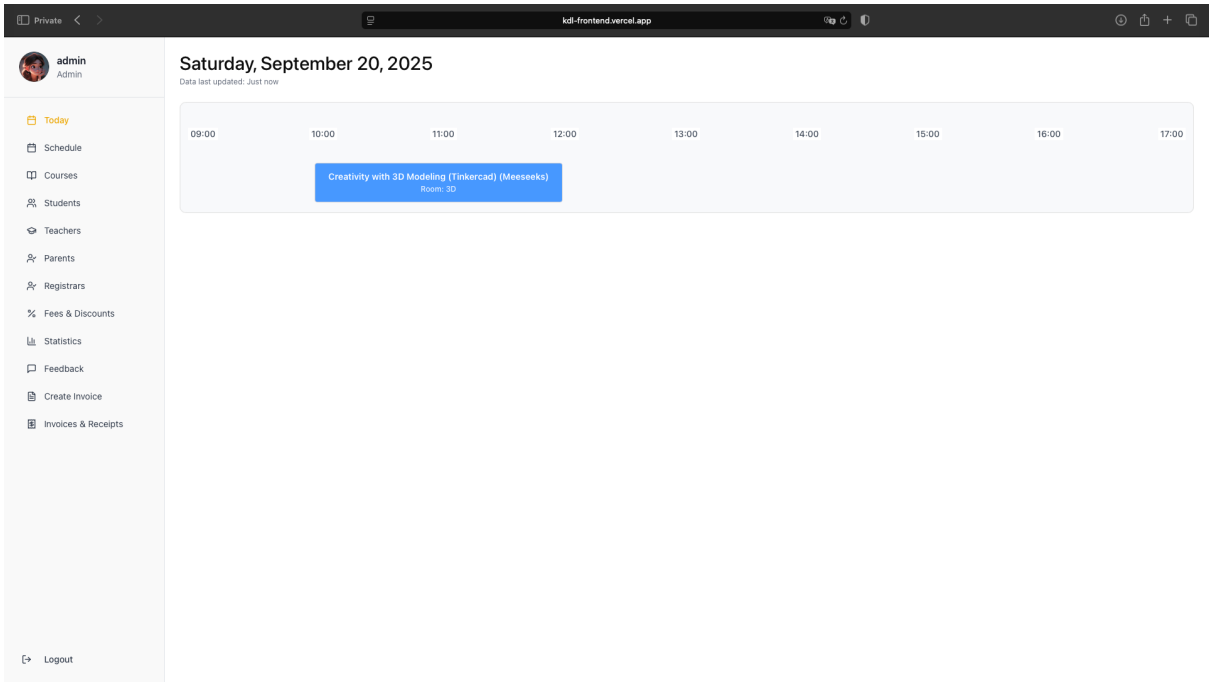


Manager UI

Today Page

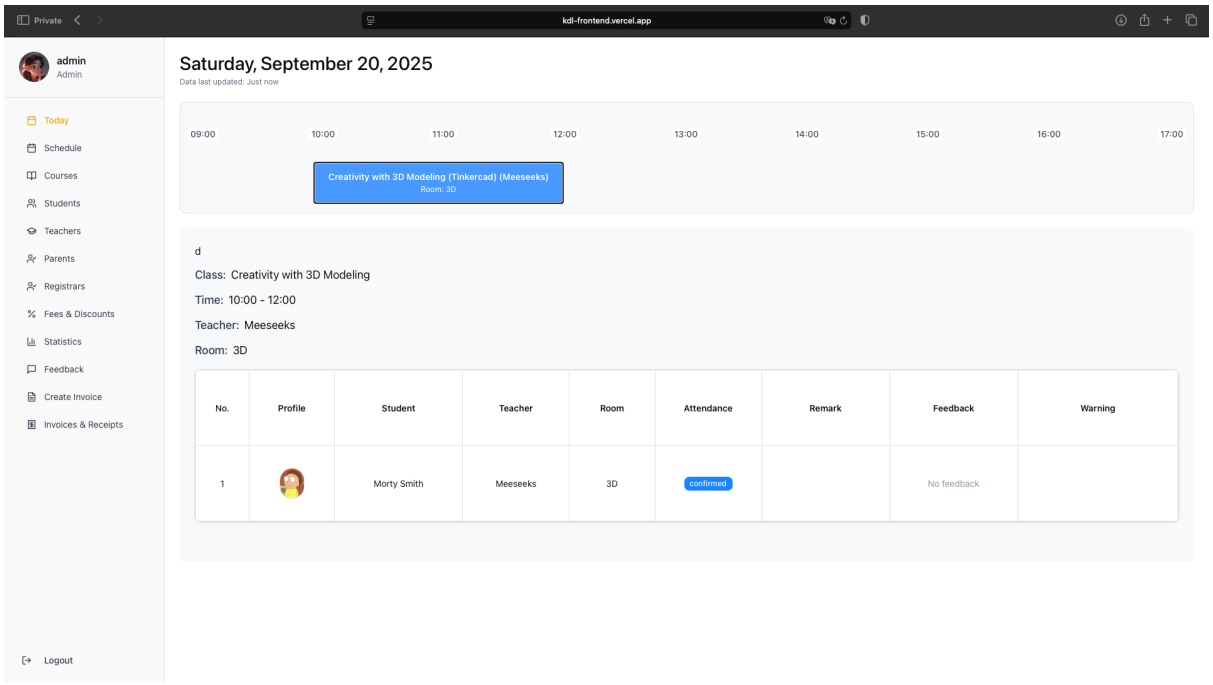
This is the default landing page for managers upon signing in to the system.

Initial Today Screen

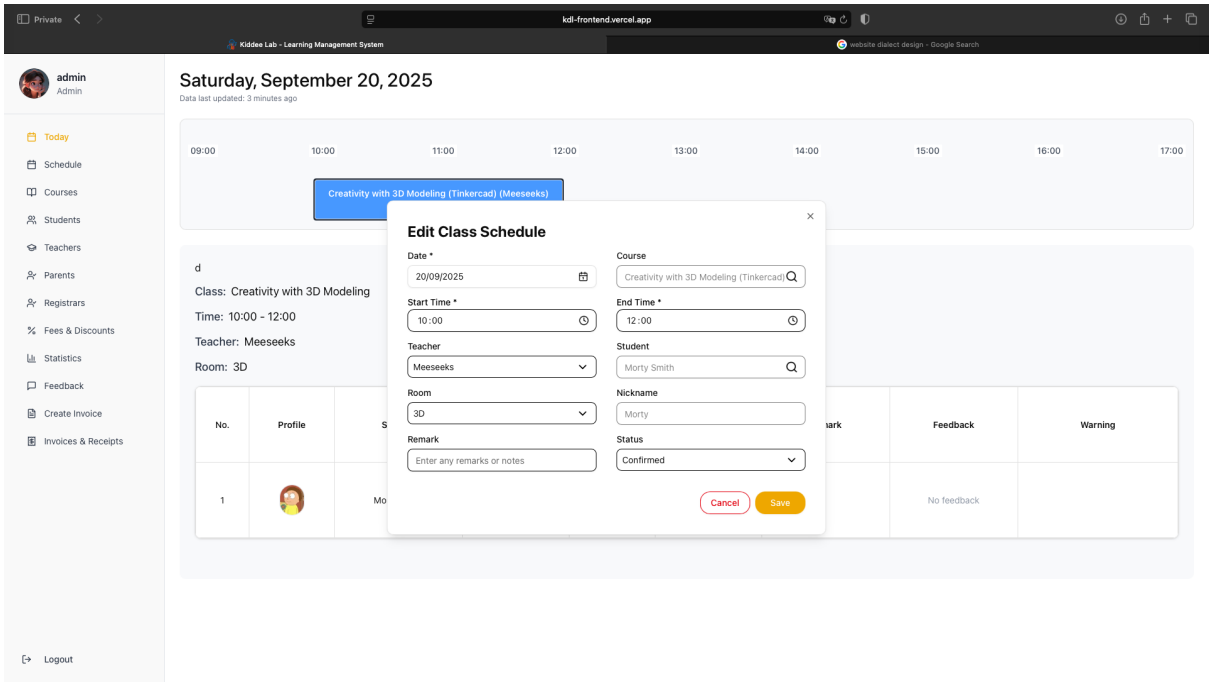


The **Today** feature displays all classes scheduled for the current day. By clicking on a specific class, the manager can view detailed information including the course name, assigned teacher, classroom, time, and the list of students expected to attend. By double-clicking a student's row, the manager can edit the student's details.

Today screen with class details



Editing class schedule in today screen

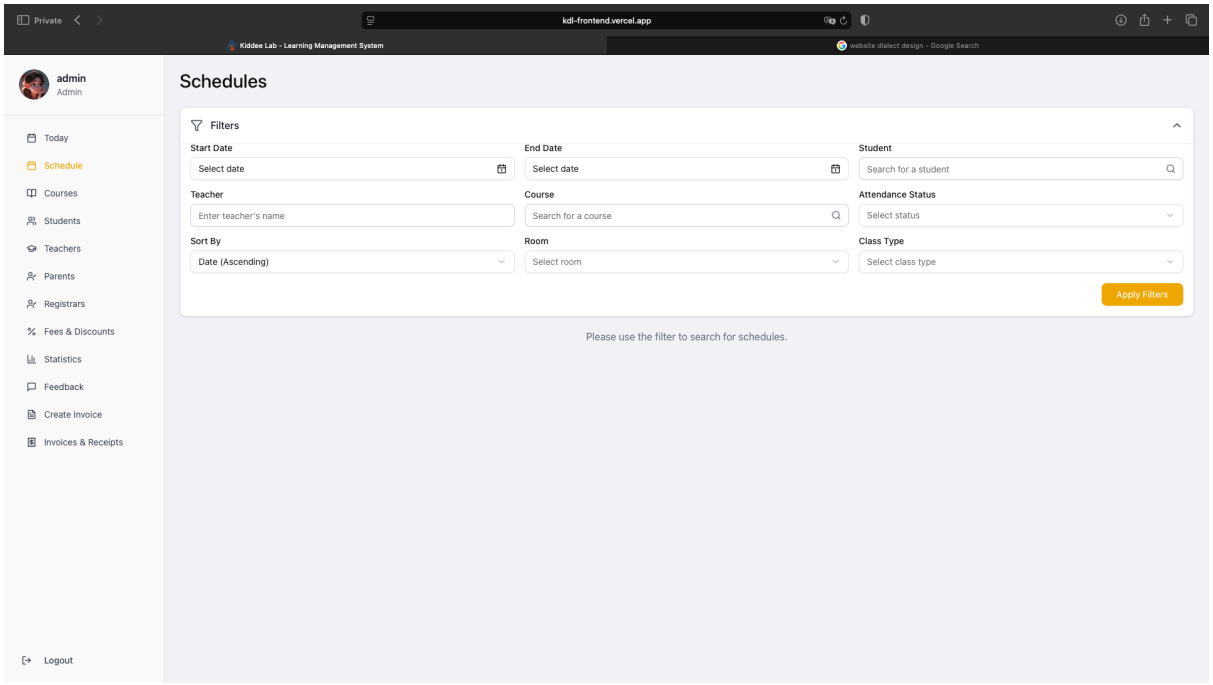


Schedules Page

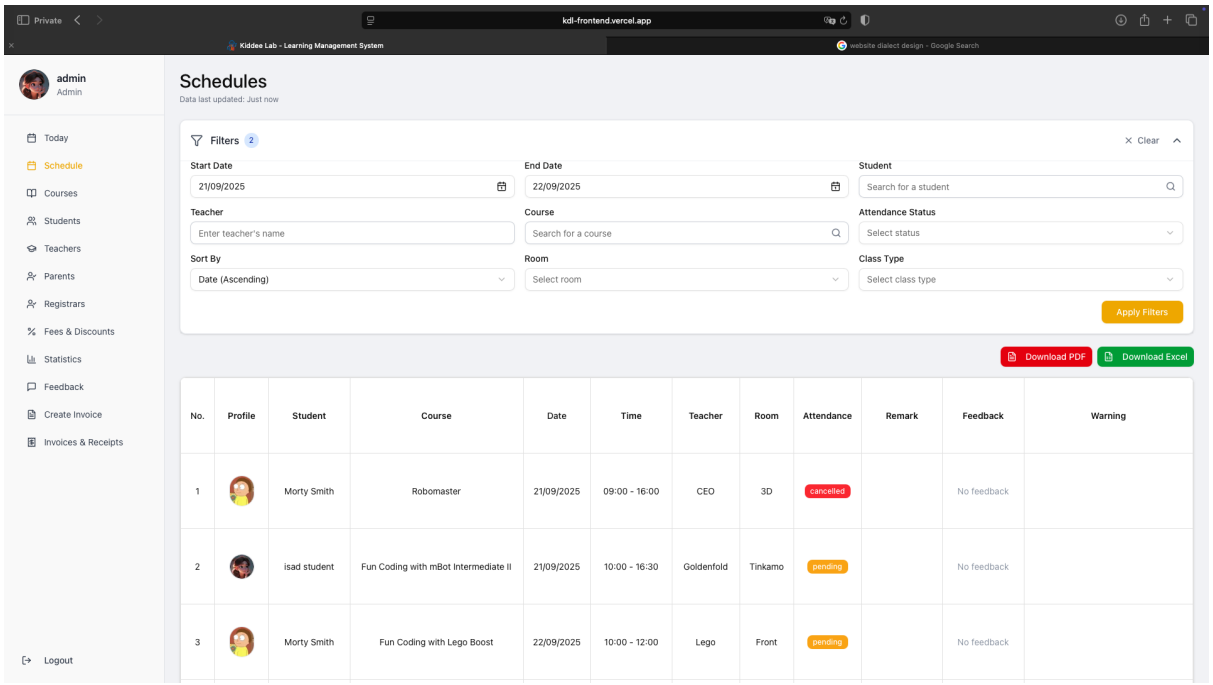
On the **Schedules** page, managers can use the filtering function to search and narrow down student schedules. Available filters include start date, end date, student name, teacher, course, attendance status, sort order, room, and class type.

By double-clicking a schedule row, the manager can access and edit the associated student's details.

Initial Schedules Screen



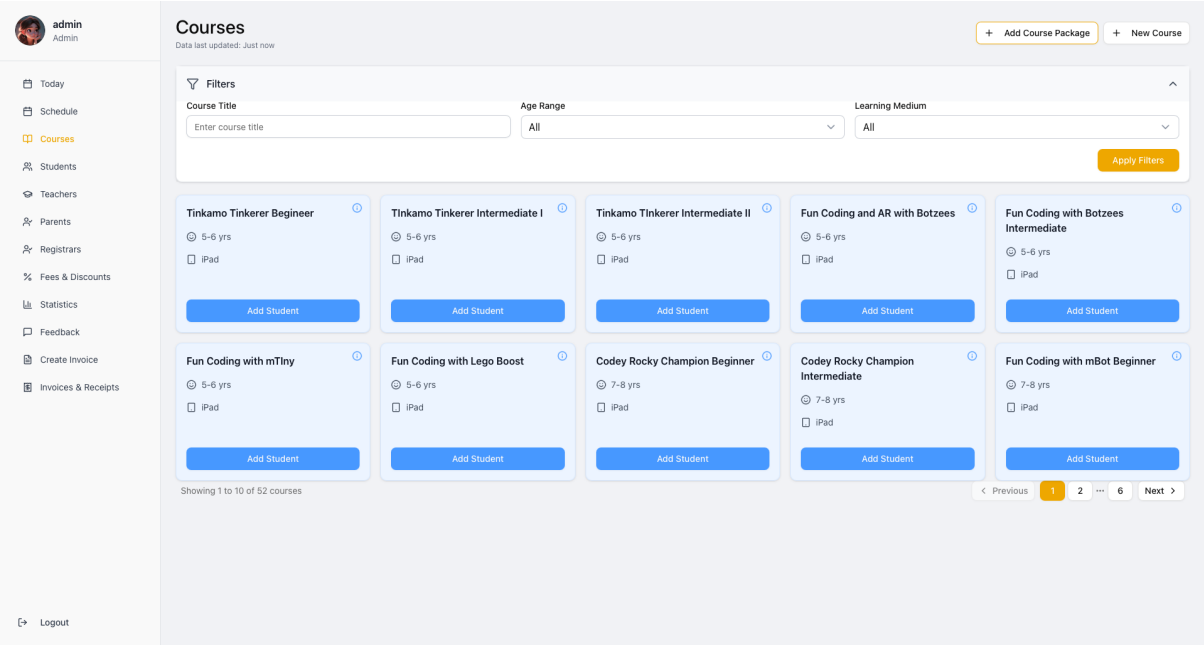
The following is an example of filtering schedules using the start date and end date.



Courses Page

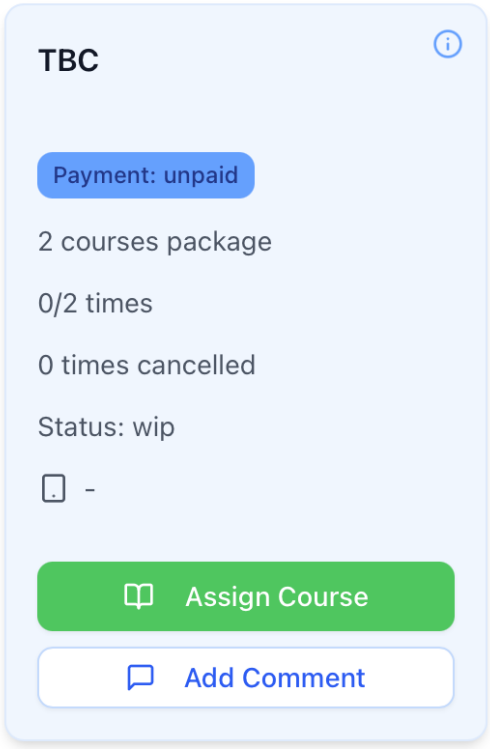
On the **Courses** page, the manager can view all courses currently offered by KDL. New courses can be created by entering details such as the course title, description, recommended age range, and learning medium. Additionally, the manager can directly add students to a course from this page.

Course Screen

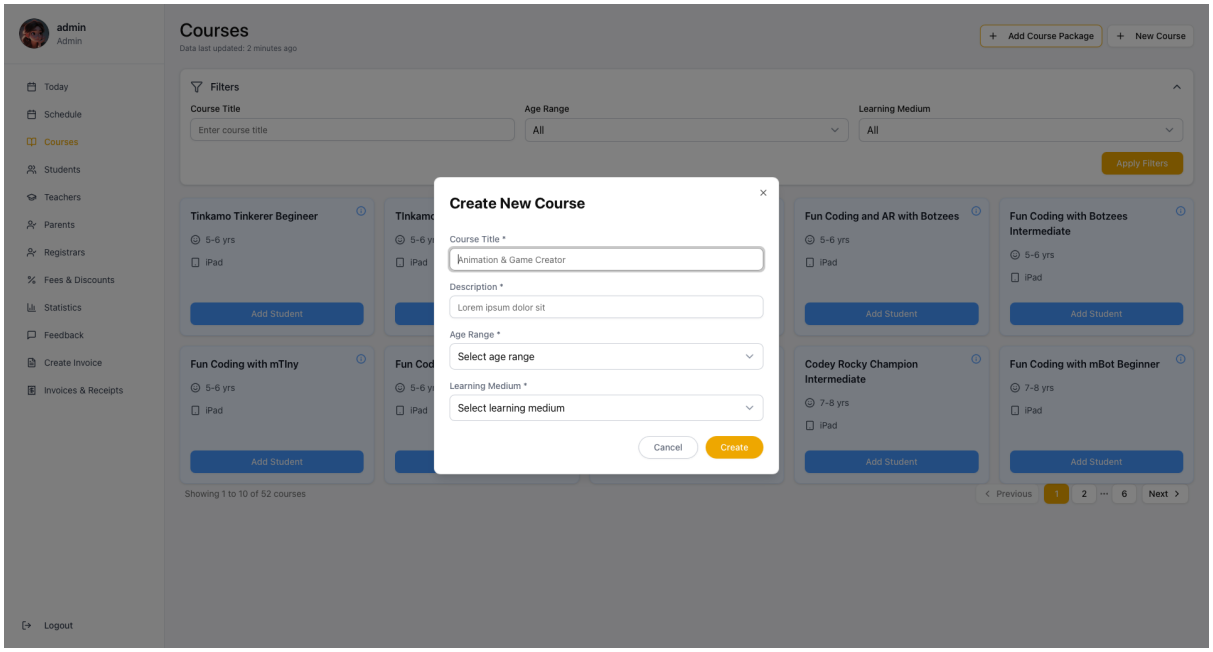


The manager can assign course packages to students. To streamline this process, when searching for a student, the manager can enter the student's name, nickname, or student ID. Once one of these fields is entered, the remaining fields will be automatically populated. After successfully adding a course package, a **TBC (To Be Confirmed)** course will be assigned to the student. This TBC course allows the course title to be specified at a later time. The purpose of the course package is to enable parents to purchase multiple courses in bulk at a discounted price. The procedure for assigning the actual course title is the same as when adding a new course to a student.

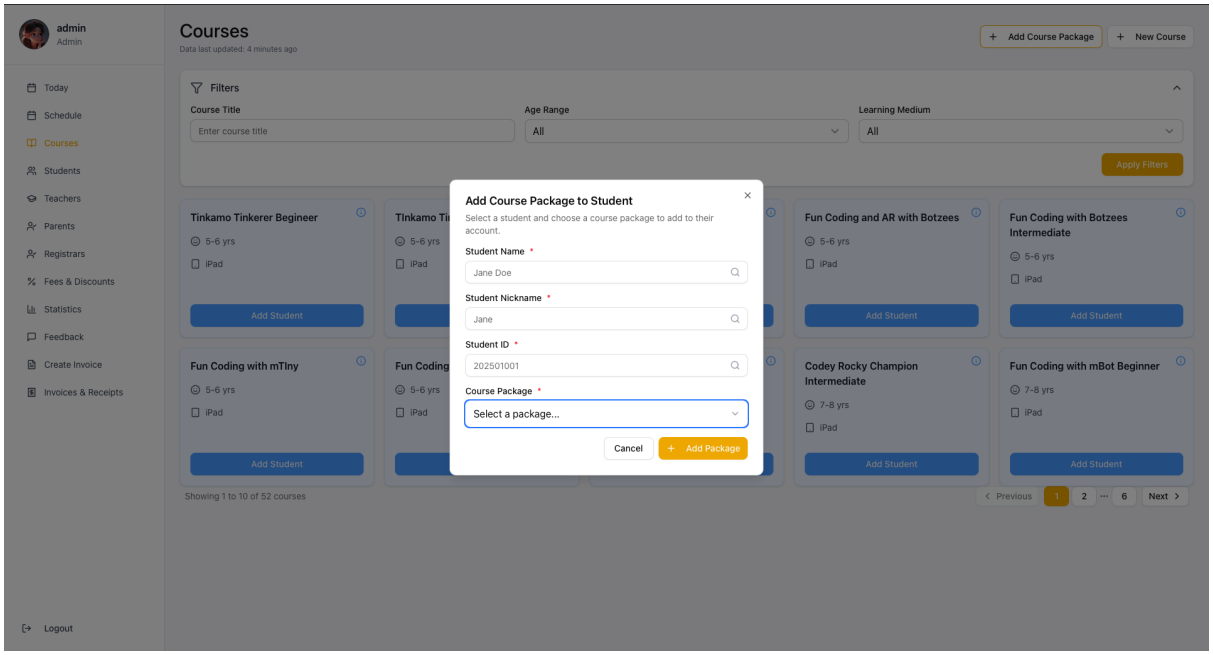
TBC course



Create new course



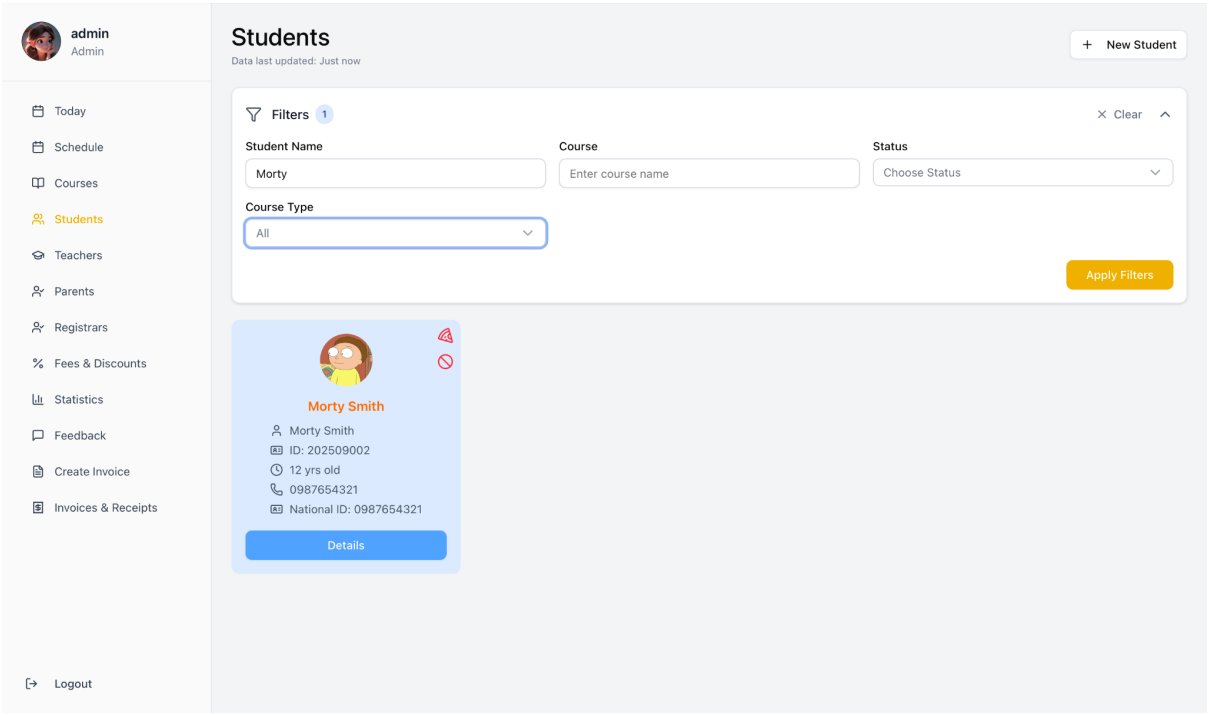
Add Course Package to Student



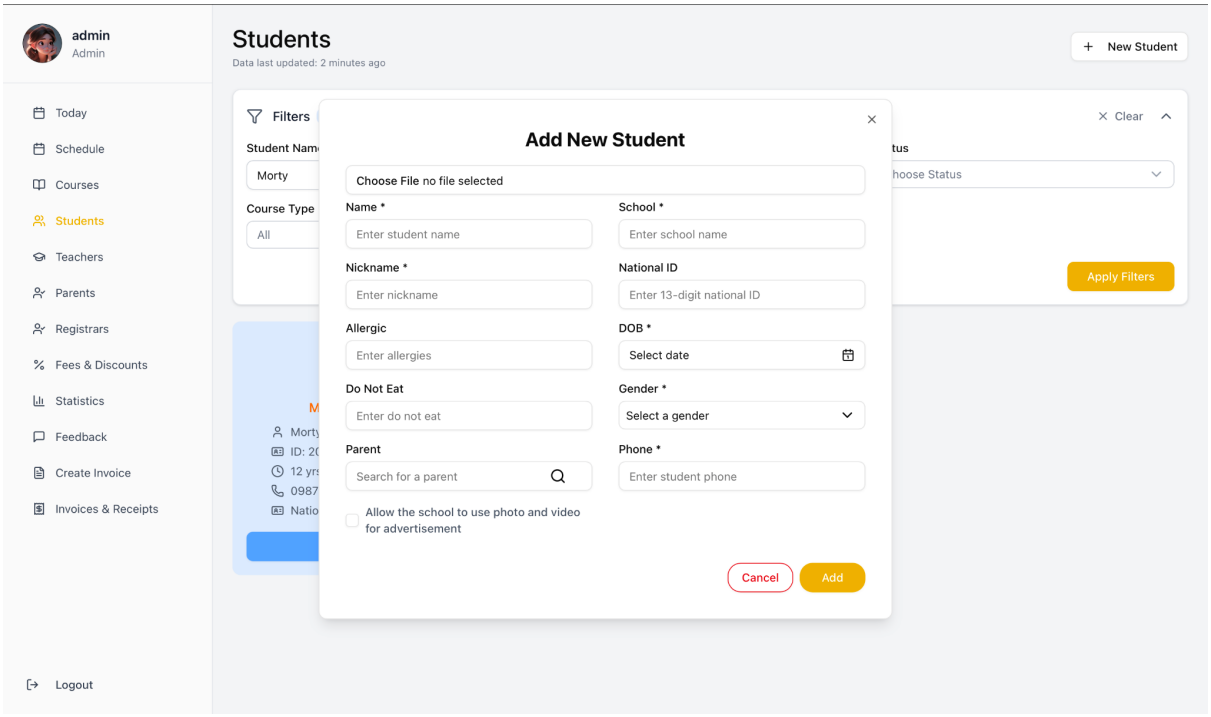
Students Page

On the **Students** page, the manager can view a list of all students. New students can be added by entering details such as photo, name, nickname, allergies, dietary restrictions, school, national ID, date of birth, gender, phone number, parent information, and permission for the use of photos or videos for advertising purposes. Fields marked with an asterisk (*) are required. If the manager enters invalid or incomplete information in these fields, appropriate error messages will be displayed.

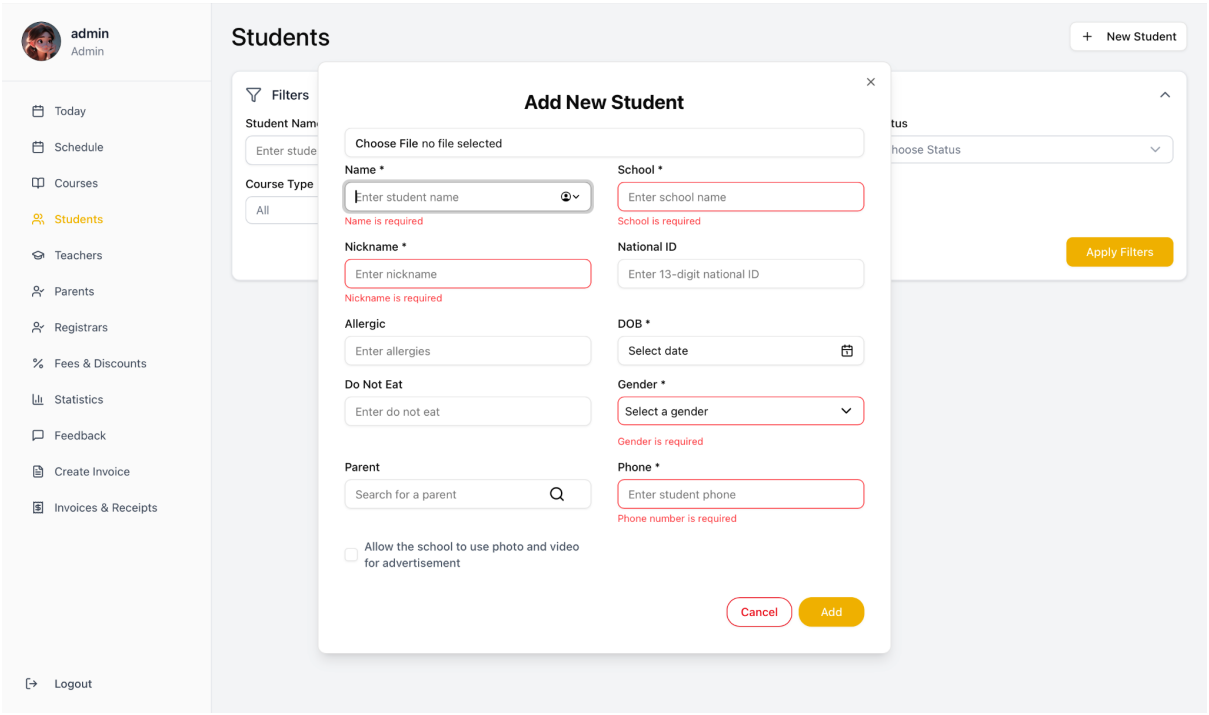
Student page with list of student



Register New Student



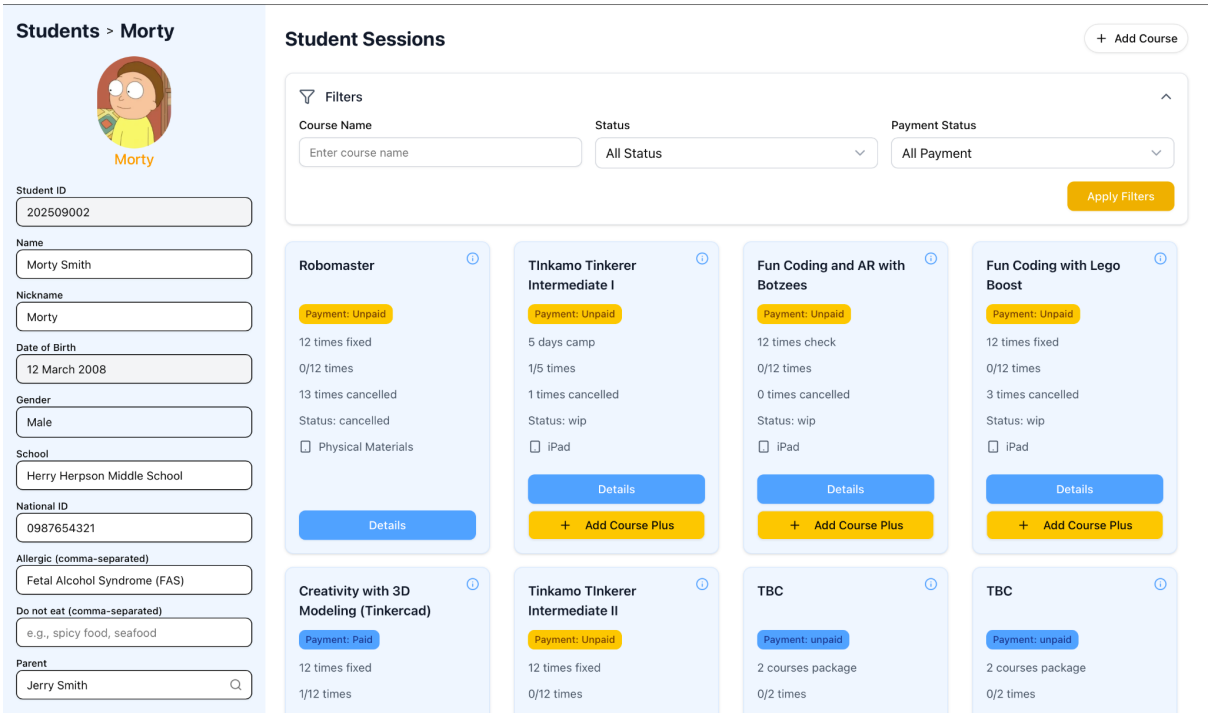
Display error if the required files are not filled.



The manager can add courses to a student by clicking the **Add Course** button.

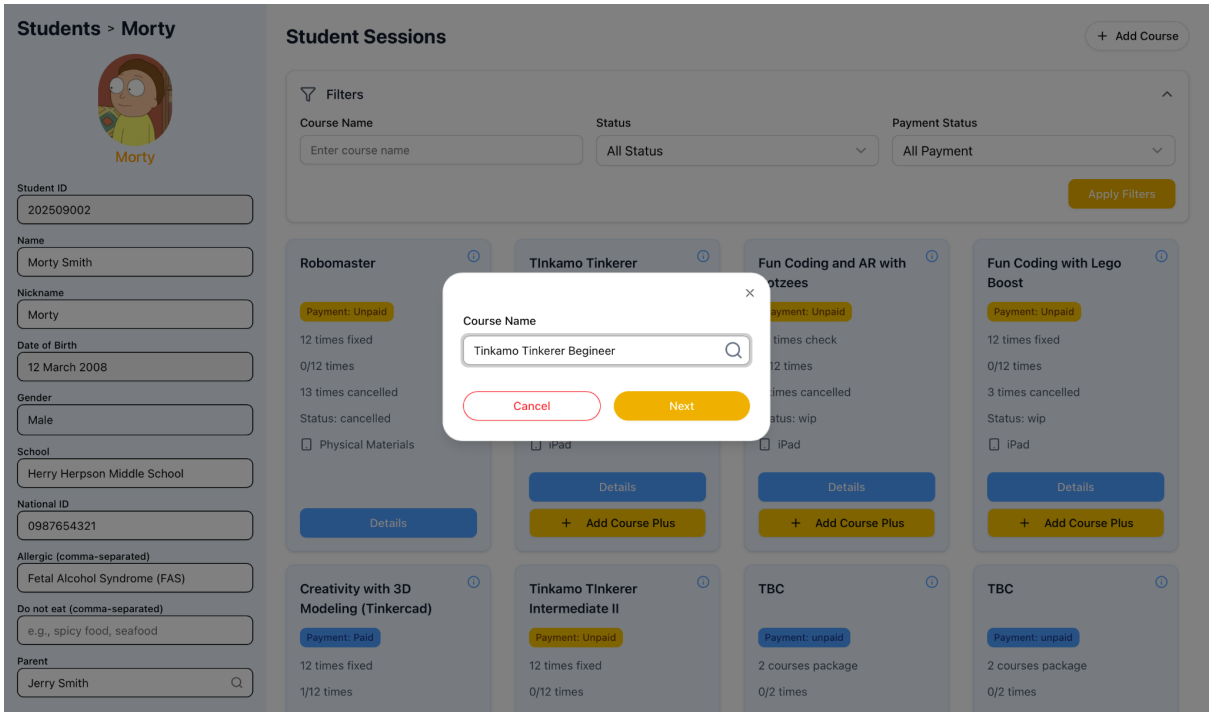
By selecting **Details**, the manager can view the student's personal information as well as their course history.

Student personal detail



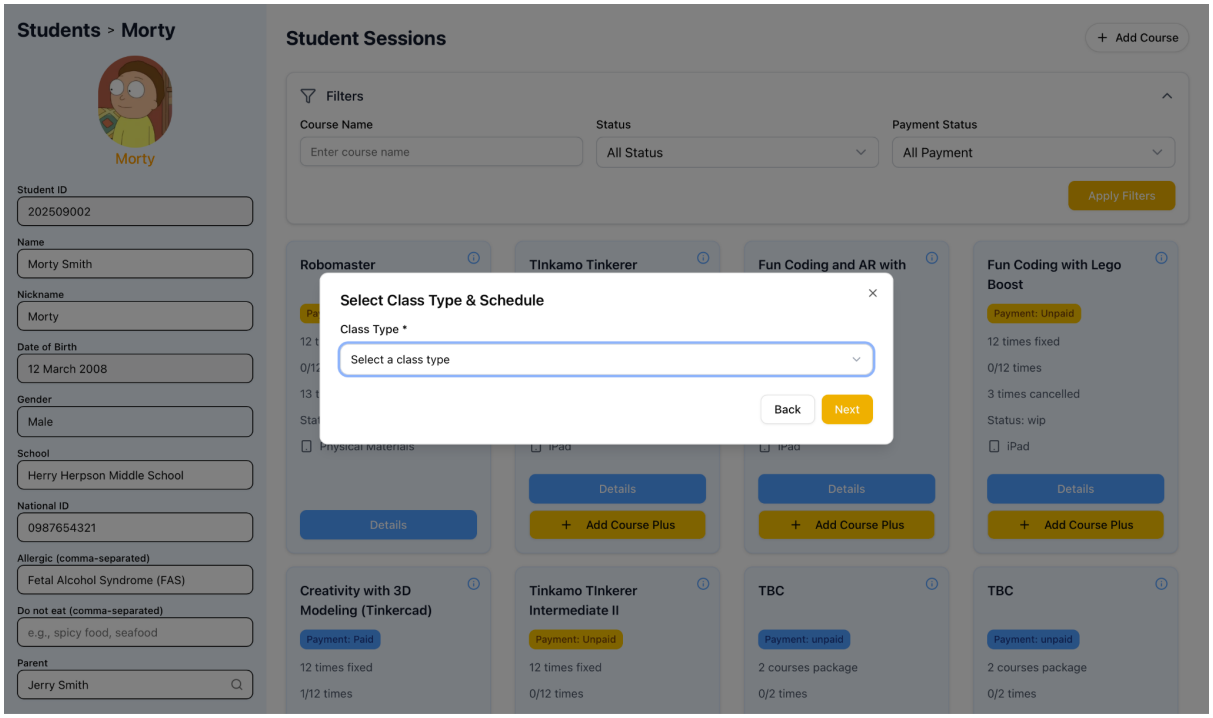
To ensure errors are promptly addressed and users are clearly informed, the system divides the course addition process into smaller, manageable steps. After clicking **Add Course**, the manager must enter a course name that already exists in the system. Once the user types at least three characters, the system will begin suggesting matching course names. The user can only proceed to the next step after entering valid data.

Student enrolls in a course



The next step is to select the class type. To prevent invalid entries, a dropdown list is provided from which the user can choose. The available options include **12 Times Fixed**, **12 Times Check**, **5 Days Camp Class**, and others. The user selects one option and proceeds. Each class type follows its own specific procedure.

Selecting class type

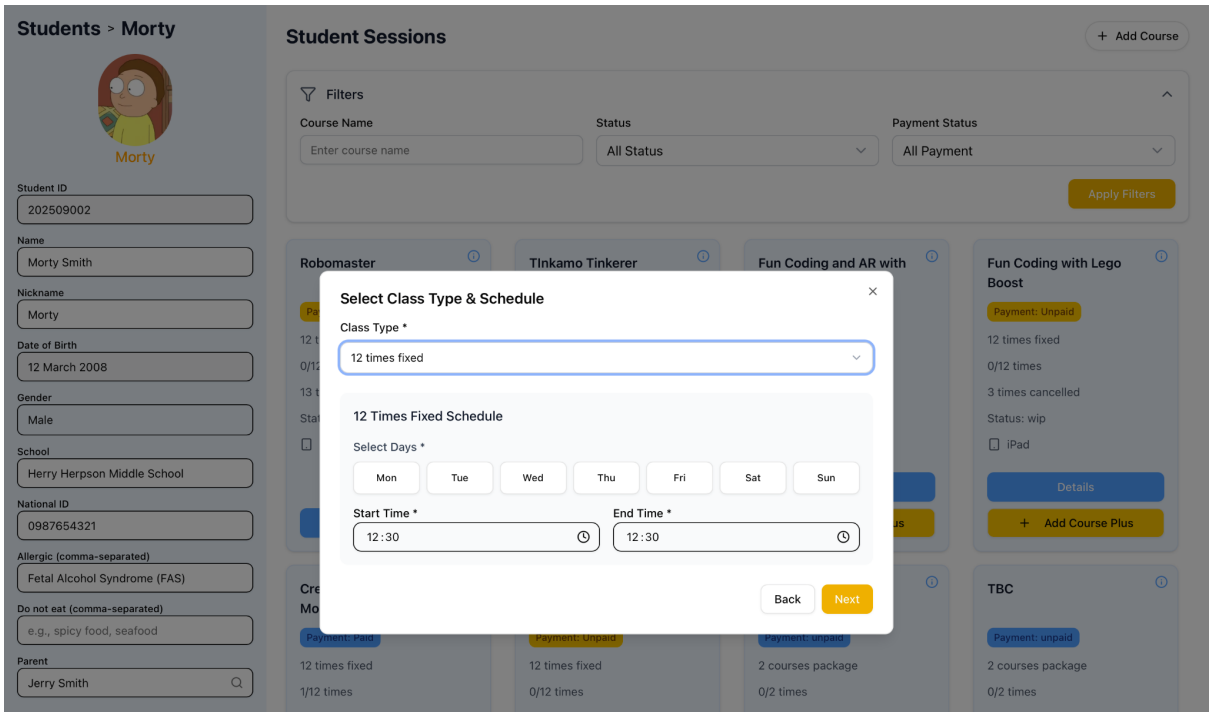


12 Times Fixed

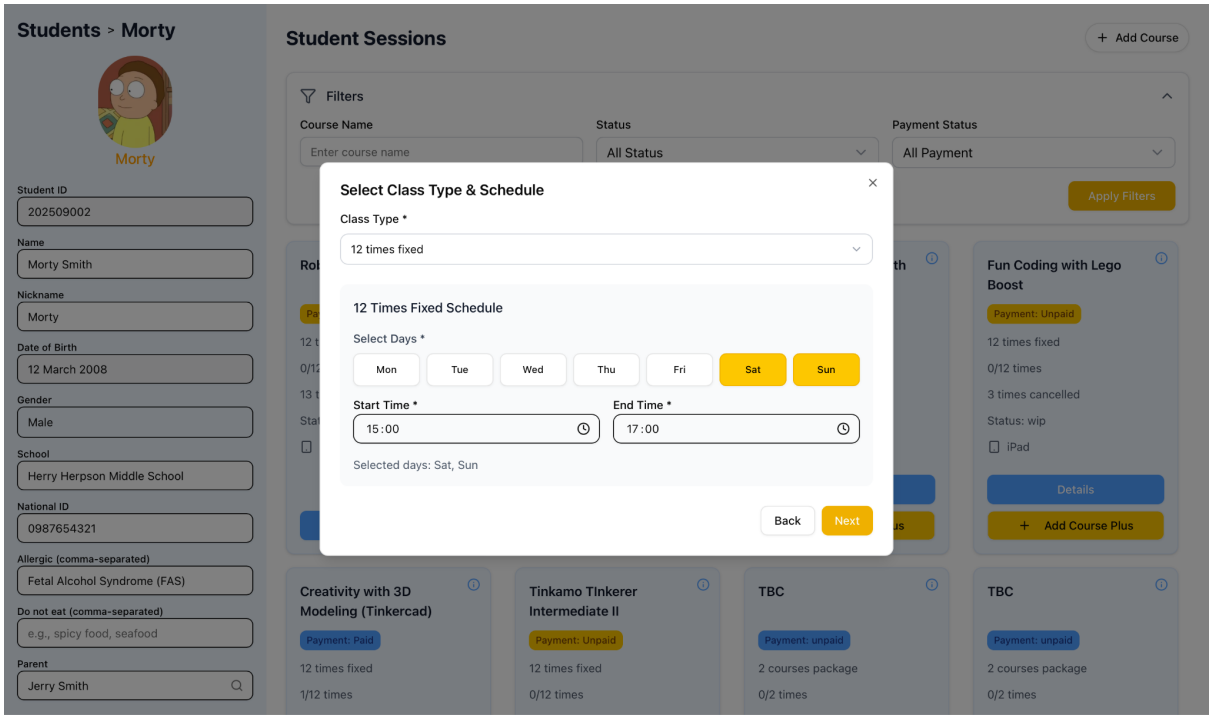
In the **12 Times Fixed** class type, the user can select the specific days and times the student will attend. Afterward, the user chooses the teacher and room. The system includes a mechanism that filters and displays only the teachers qualified to teach the selected subject, preventing the selection of an inappropriate teacher.

For example, a student may be scheduled to attend classes on Saturdays and Sundays from 3 PM to 5 PM with Teacher Meeseeks in the room named "Small."

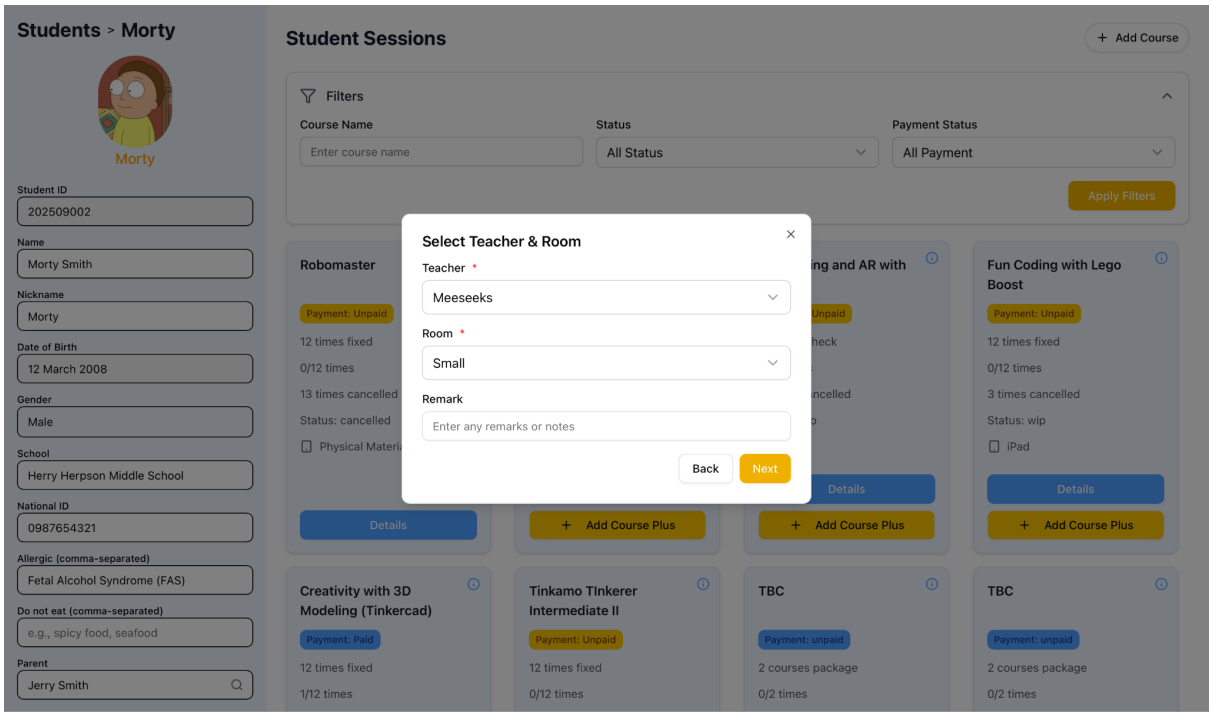
Class type as 12 times fixed screen



Class type as 12 times fixed with example input screen

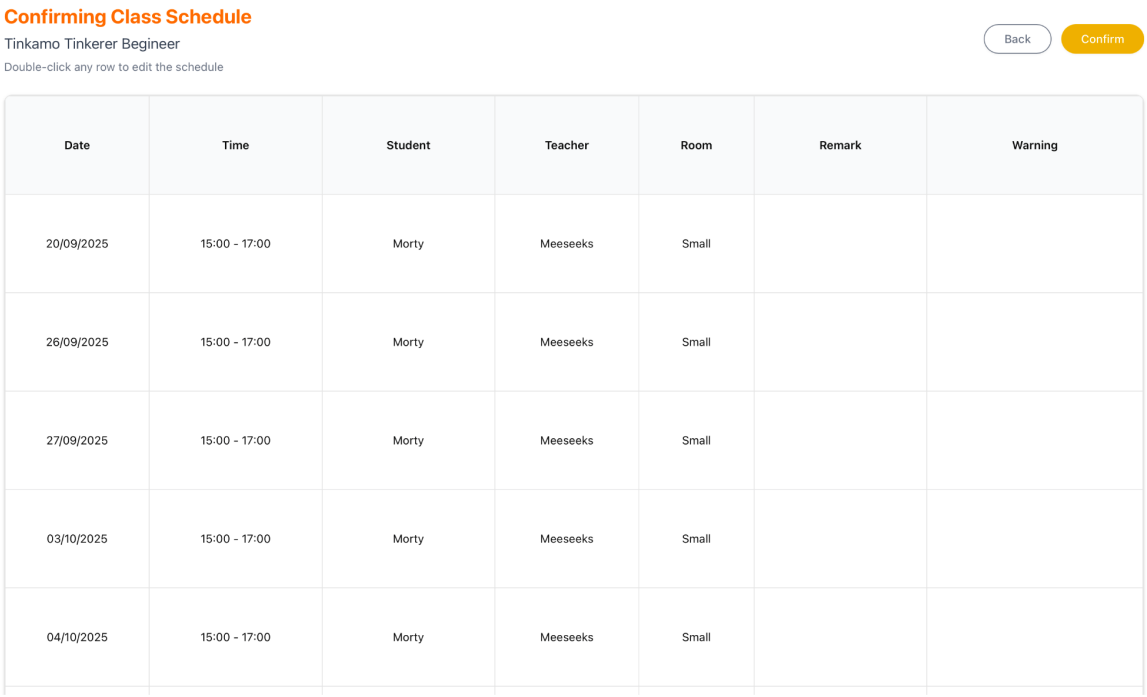


Selecting teacher and room

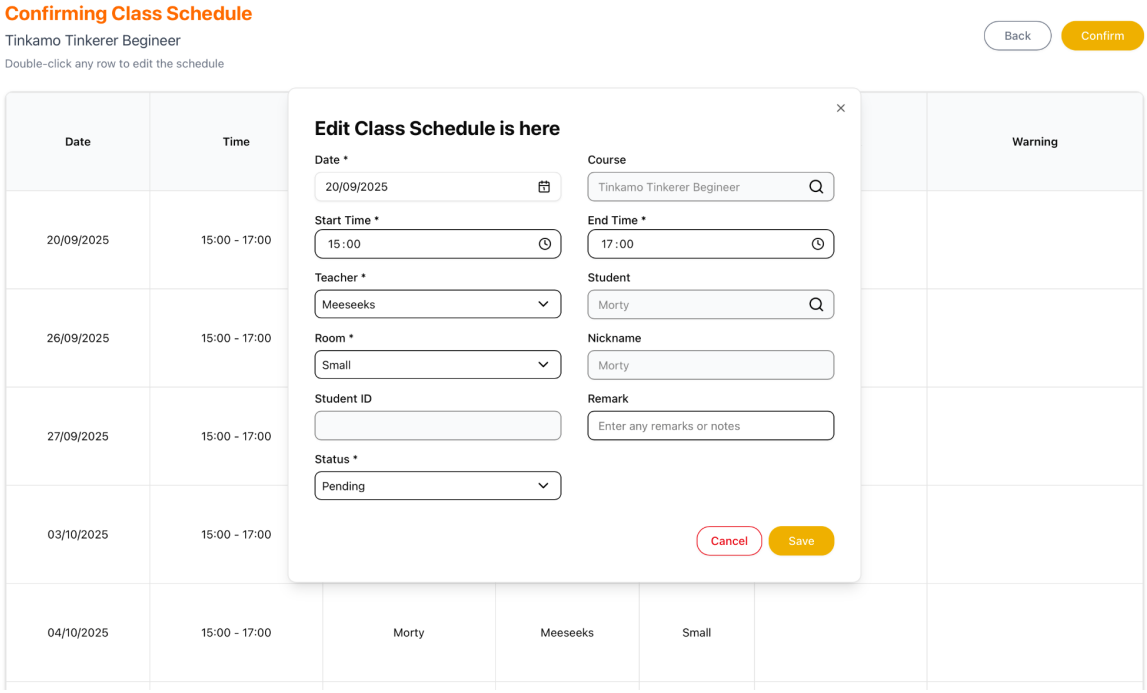


The final step before saving the schedule to the database is **Confirming the Class Schedule**. At this stage, the system allows the user to verify whether there are any scheduling conflicts. The schedule is generated based on the previously entered information. If a conflict is detected, the reason will be displayed in a warning column. The user can double-click any row to edit the details. To save the schedule and add the course to the student's record, the user must click **Confirm**.

Confirm Schedule Screen



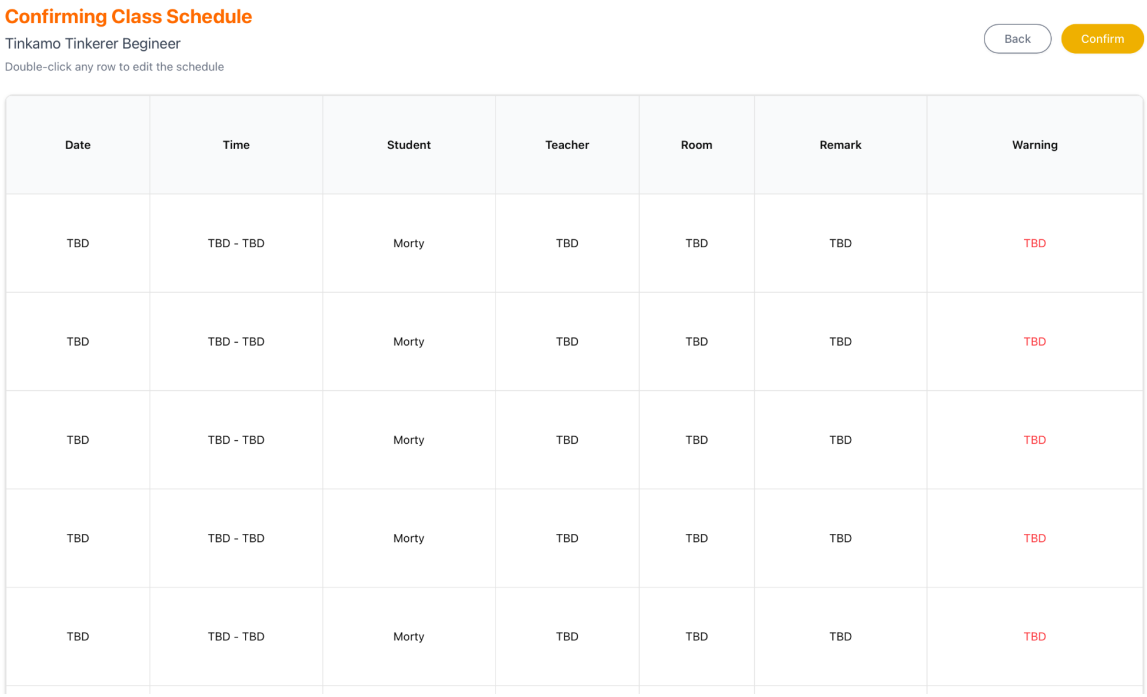
Editing each schedule in Confirming Schedule Screen



12 Times Check

In the **12 Times Check** class type, the user does not need to specify the day, time, teacher, or room in advance. Instead, the student informs KDL of their availability on an ongoing basis, allowing KDL to assign an available teacher and room accordingly.

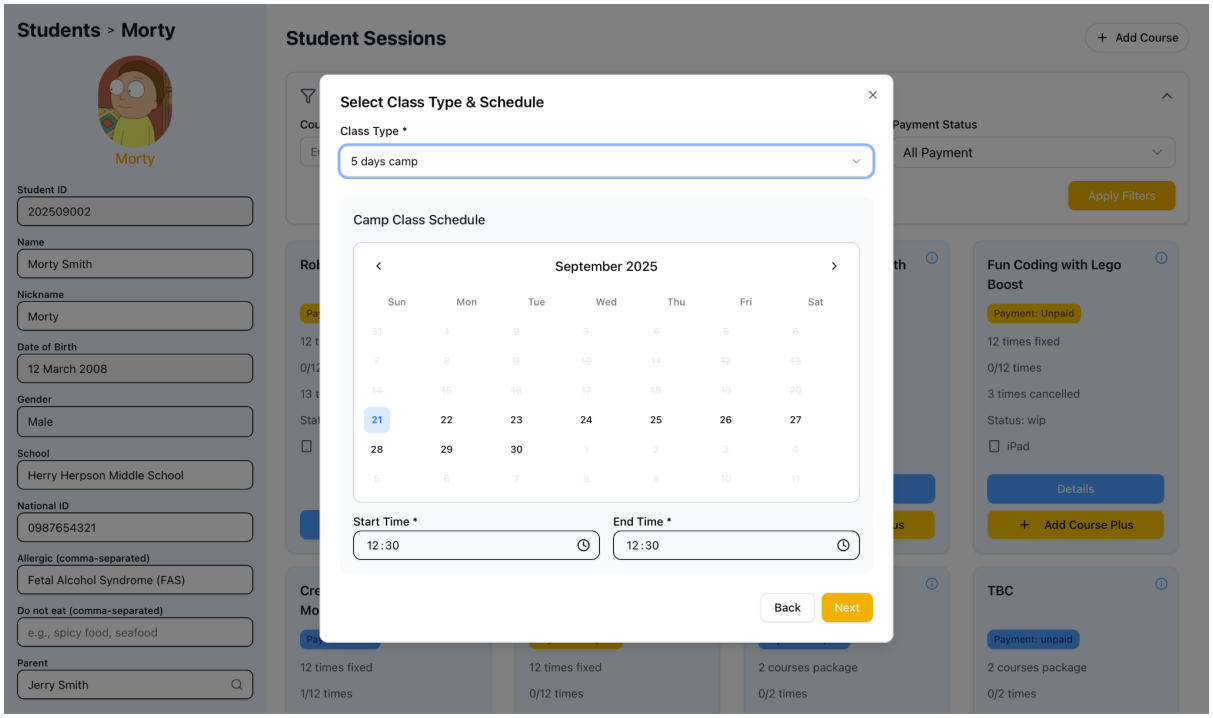
Confirming Schedule for 12 times check as class type



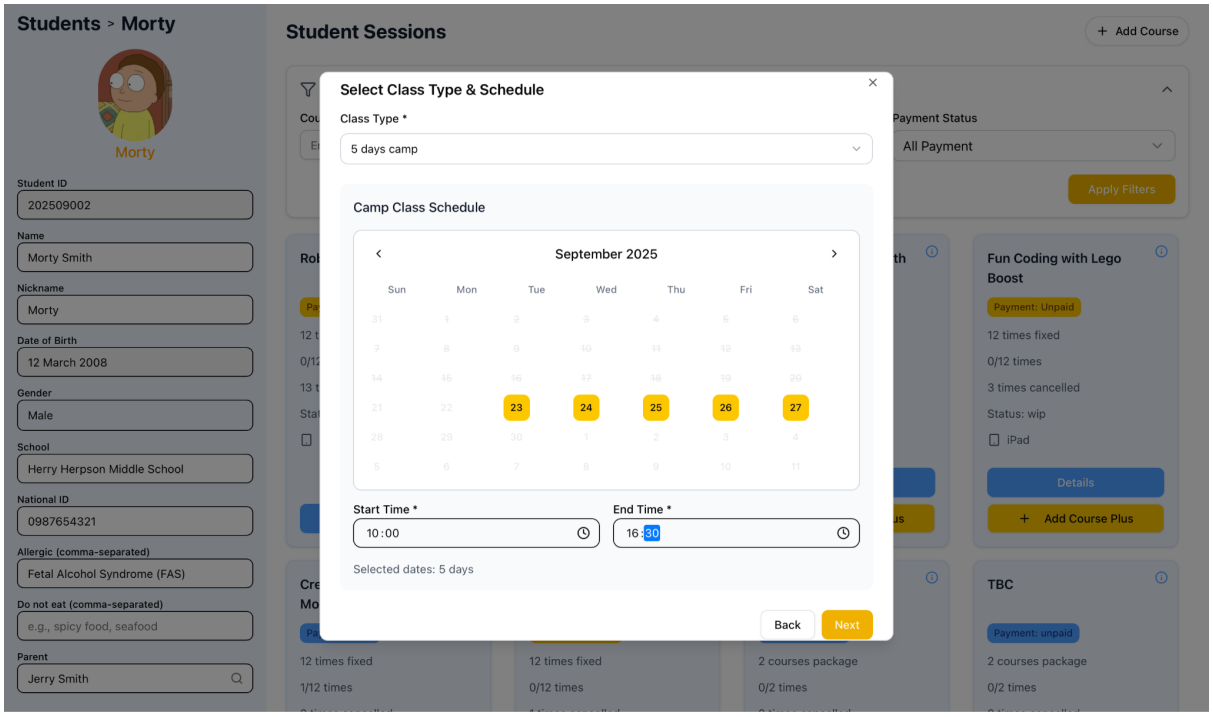
Camp class

In the **Camp Class** type, the user selects the dates and times the student will attend the class. After entering the necessary information, the user can confirm the schedule following the same procedure as in the **12 Times Fixed** class type.

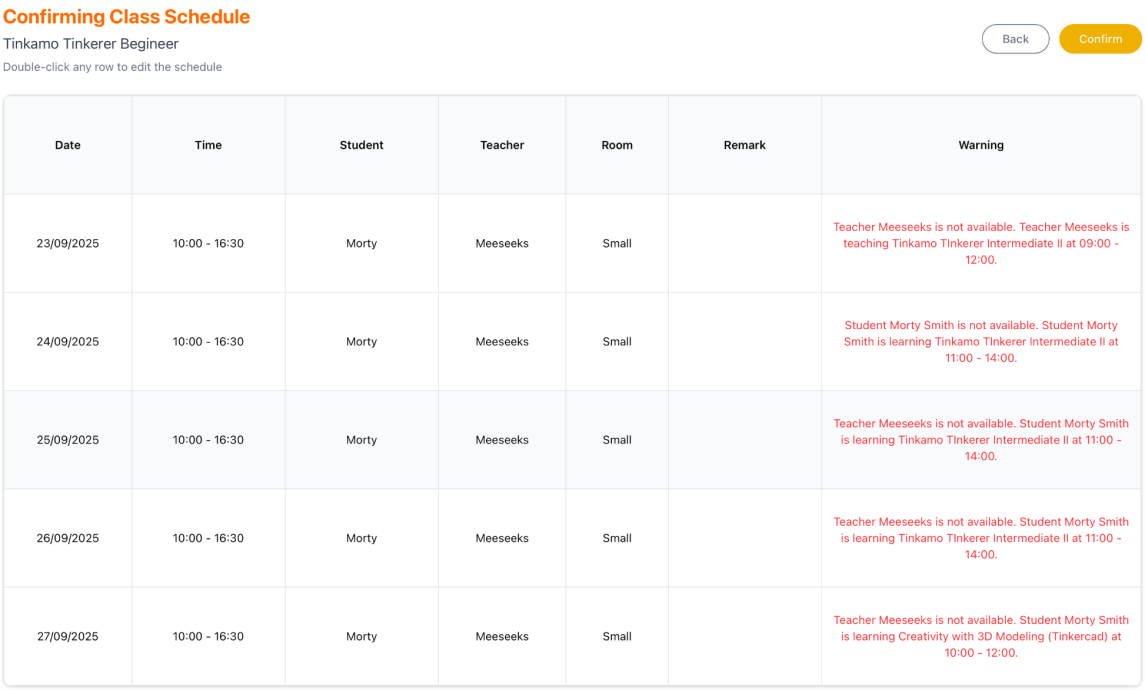
Selecting Camp class as class type screen



Selecting Camp class as class type with example input screen



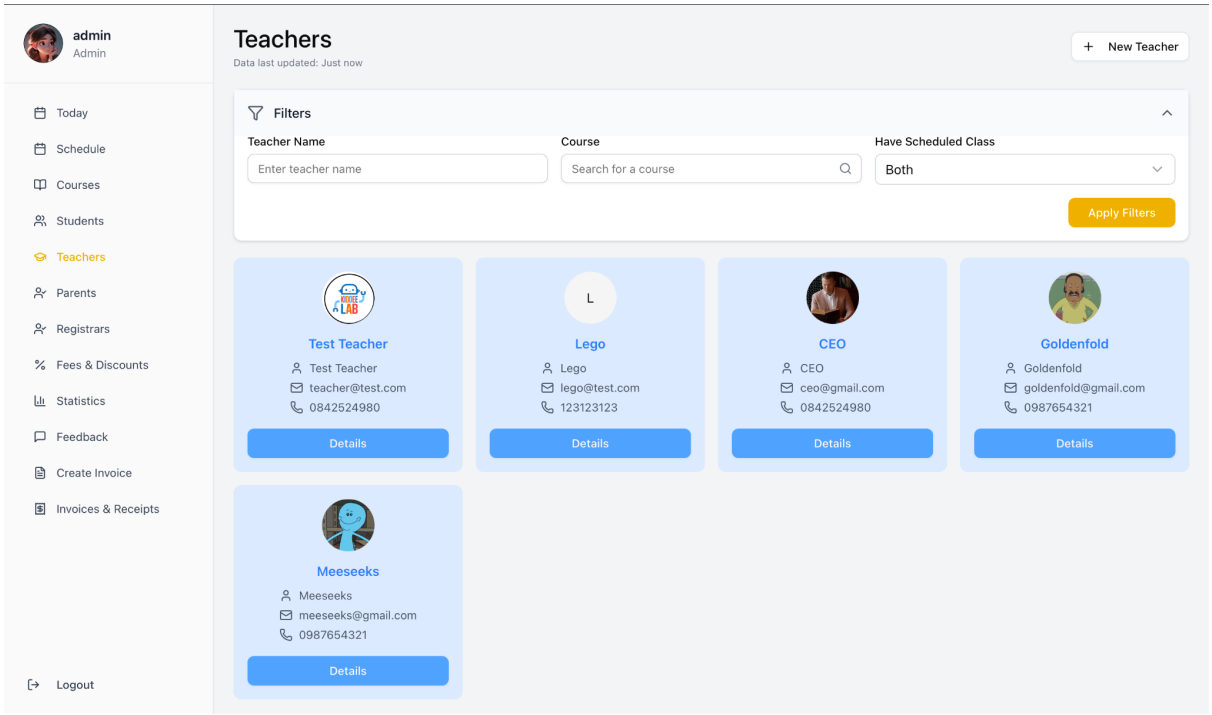
Confirming schedule with class conflict message screen



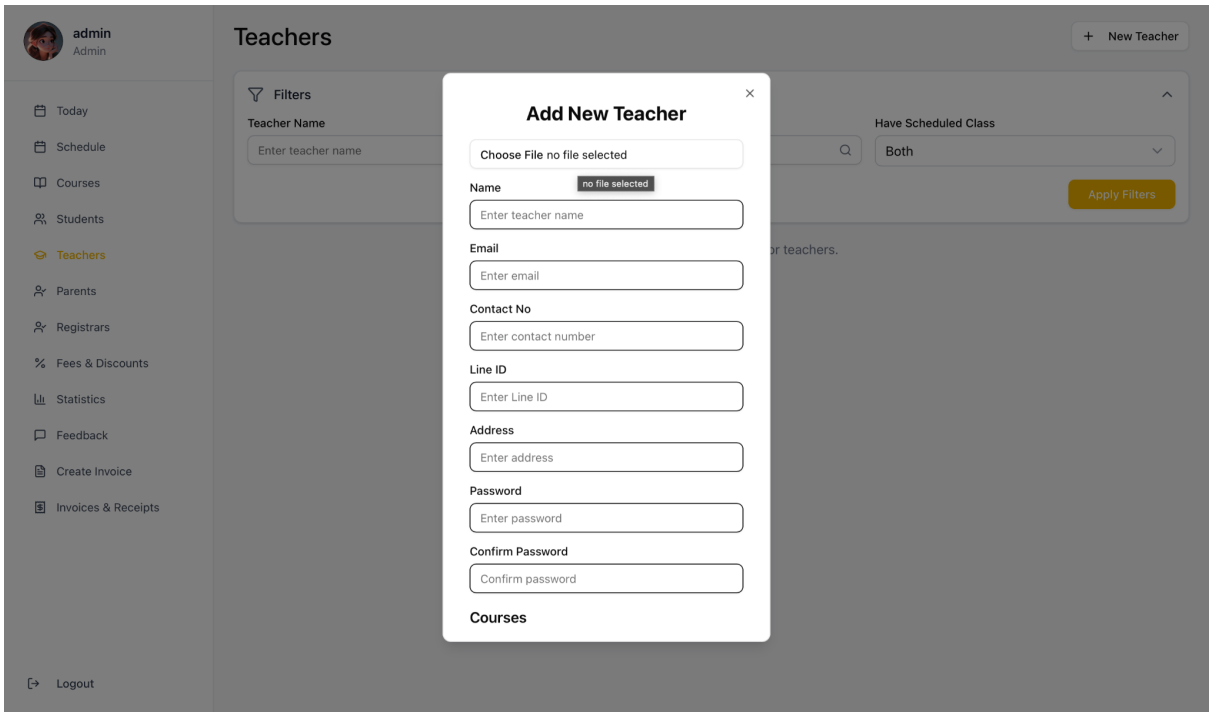
Teachers Page

On the **Teachers** page, the manager can view a list of existing teachers. New teachers can be added by entering their details. The email and password provided during the creation process will serve as the teacher's login credentials. Additionally, courses that the teacher is qualified to teach can be assigned within the system.

Teachers page with a list of teachers

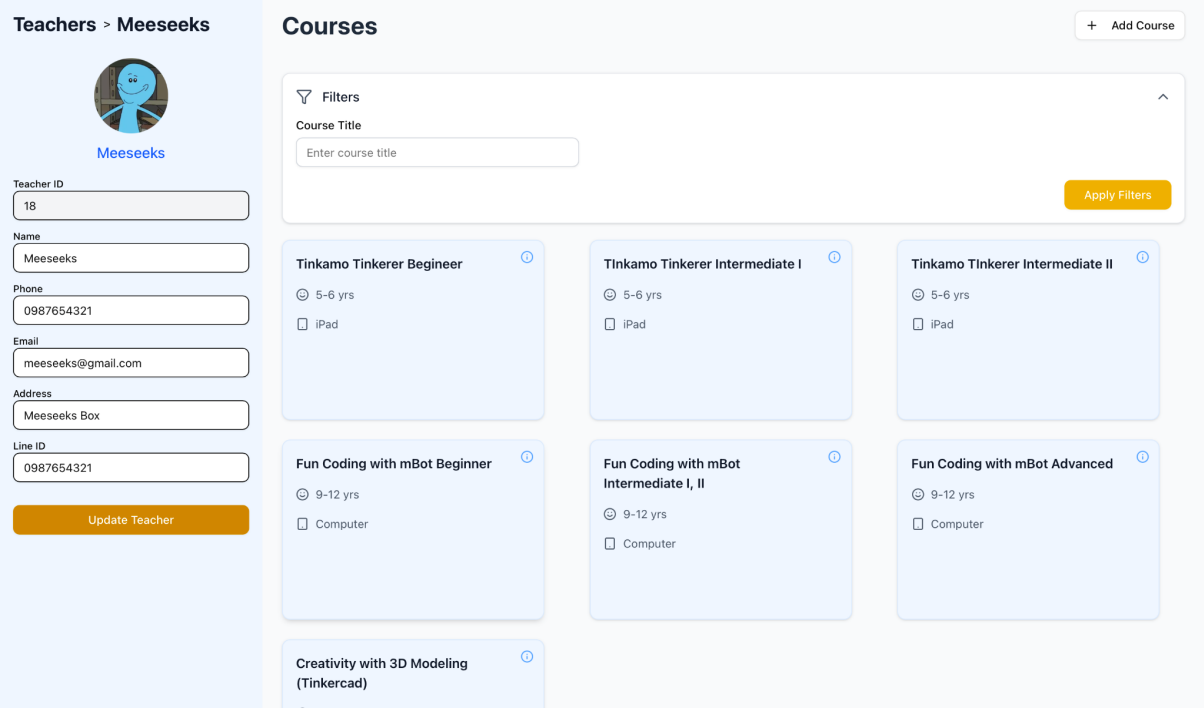


Creating a new teacher

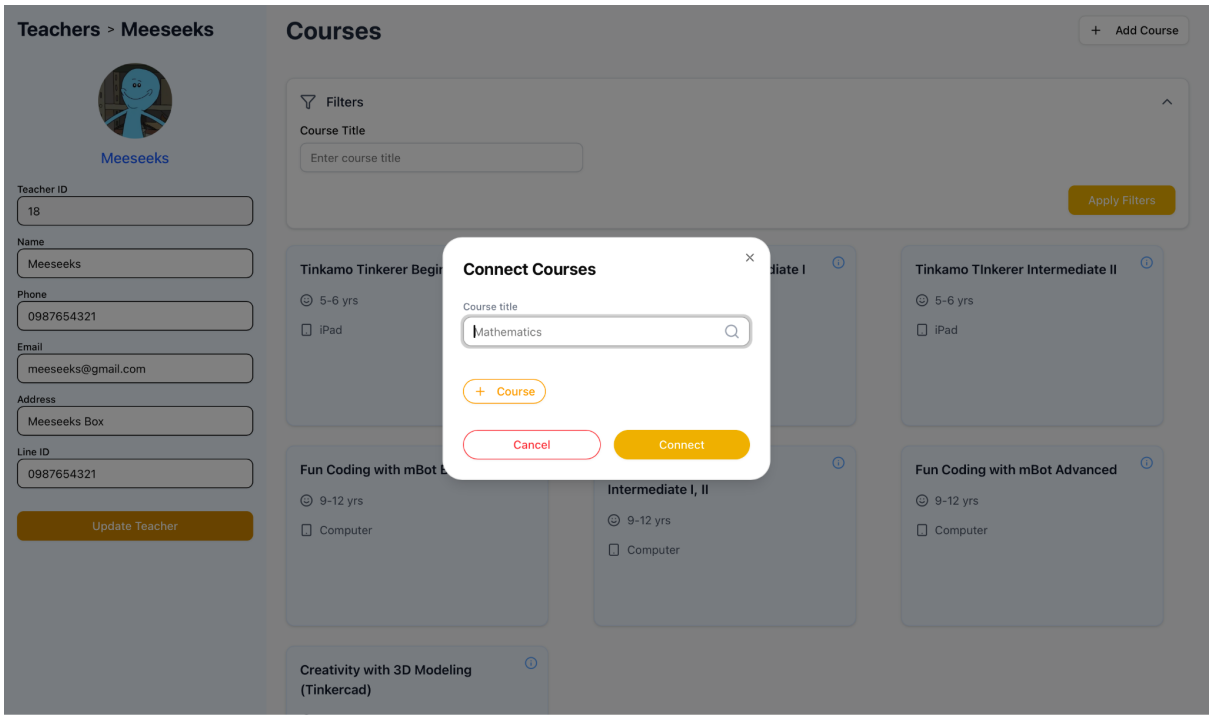


In the **Teacher Detail** view, the user can find the teacher's personal information along with a list of courses they are qualified to teach. By clicking the **Add Course** button, the user can assign additional courses to the teacher.

Teacher Detail Screen



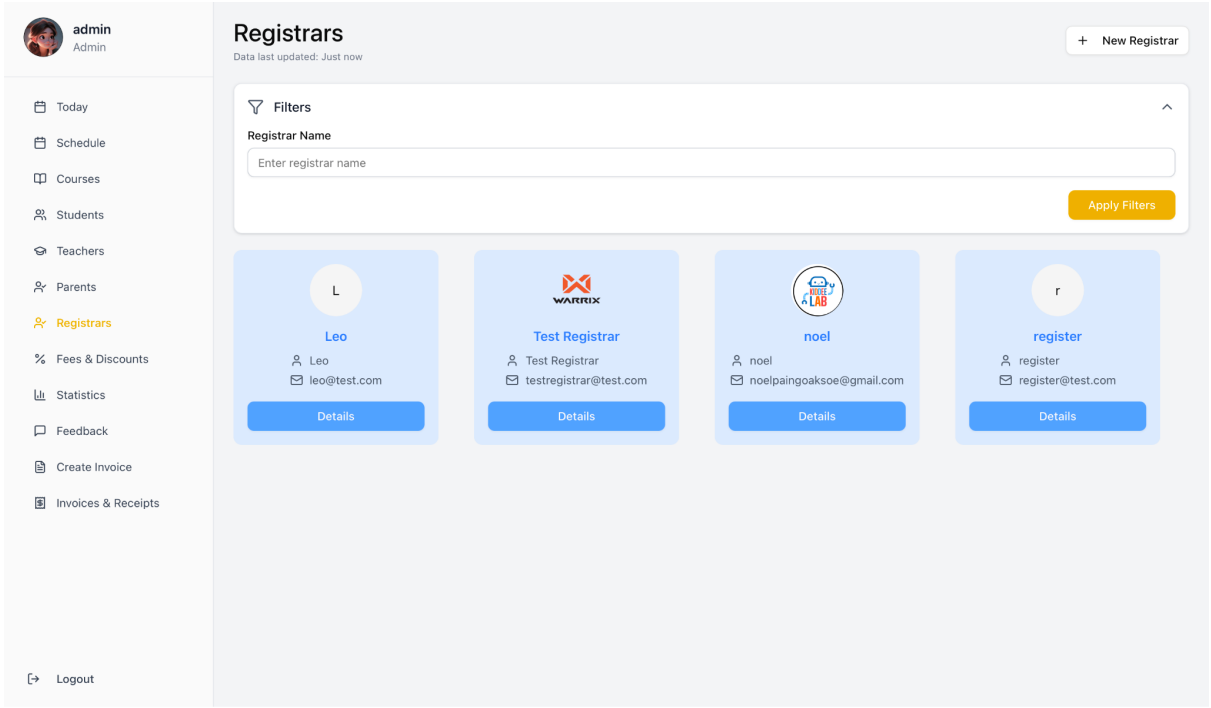
Connecting more course to teacher



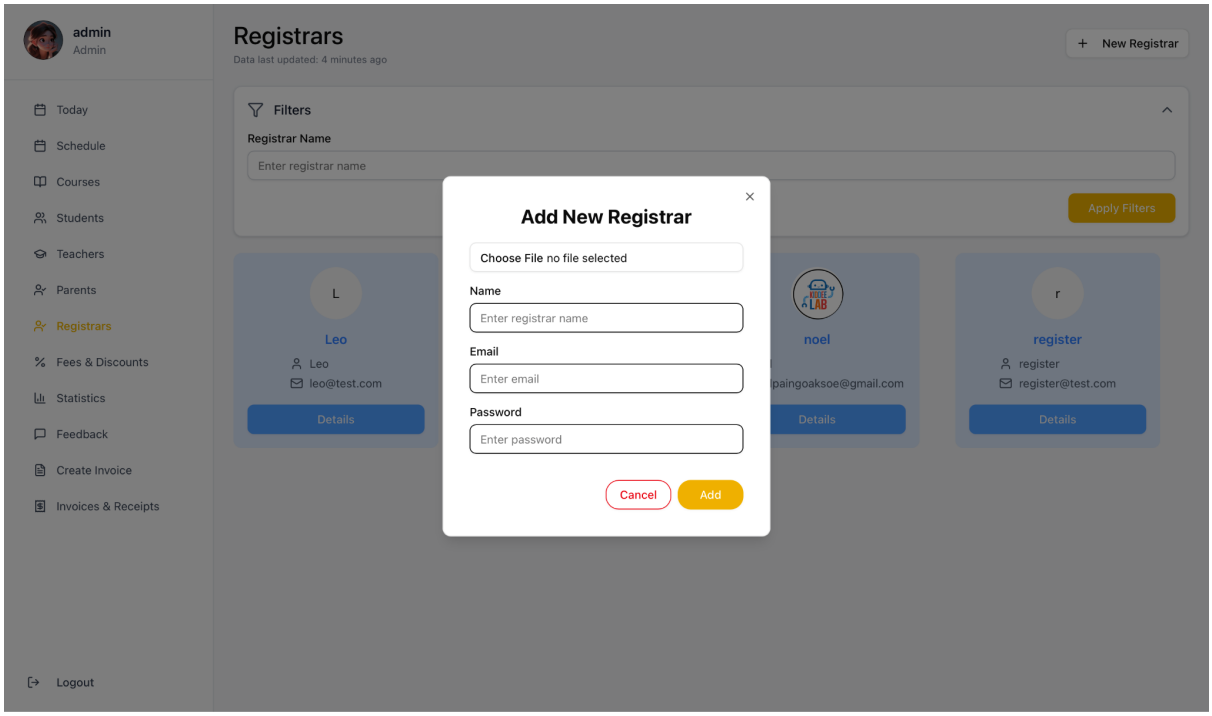
Registrars Page

On the **Registrars** page, the manager can view a list of existing registrars. New registrars can be added by entering their details. The email and password provided during the creation process will be used as their login credentials.

Registrars Page with a list of registrars



Creating a new registrar

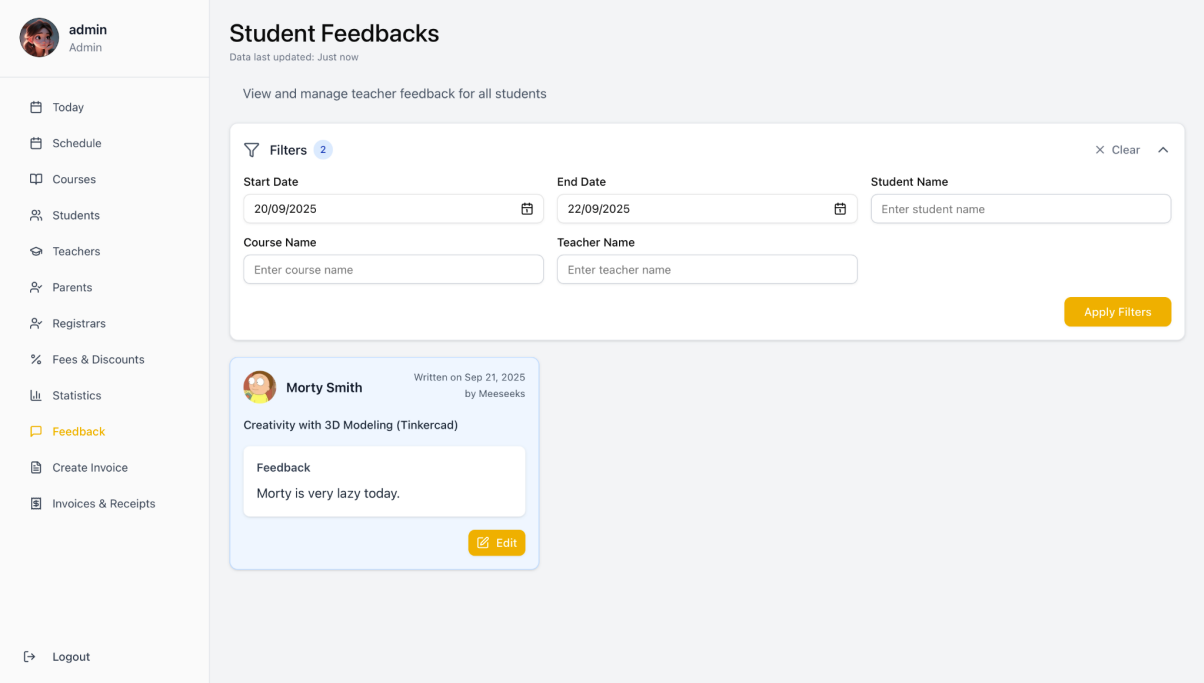


Feedback Page

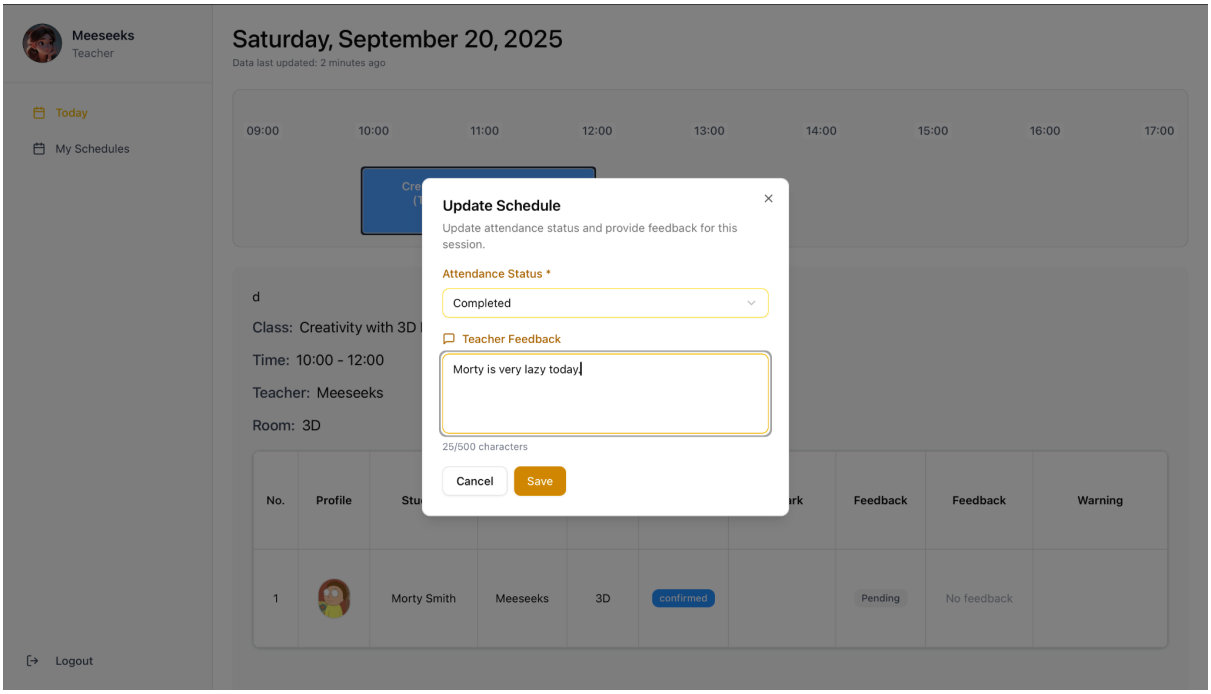
On the **Feedback** page, all feedback submitted by teachers is displayed. Users can modify and/or approve the feedback. Only approved feedback is marked as ready

to be shown to parents and can be accessed on the **Schedules** page or other relevant sections. The system also logs the author of each feedback entry for accountability.

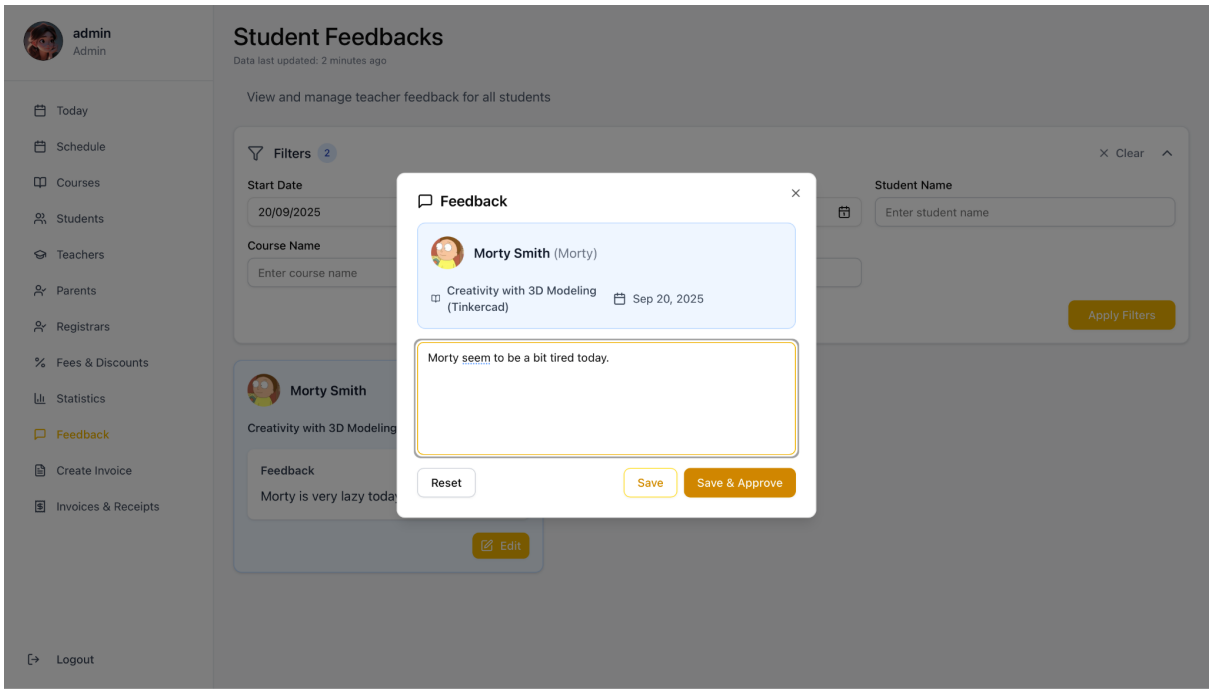
Student feedback page with a list of feedback



Modifying Original Feedback



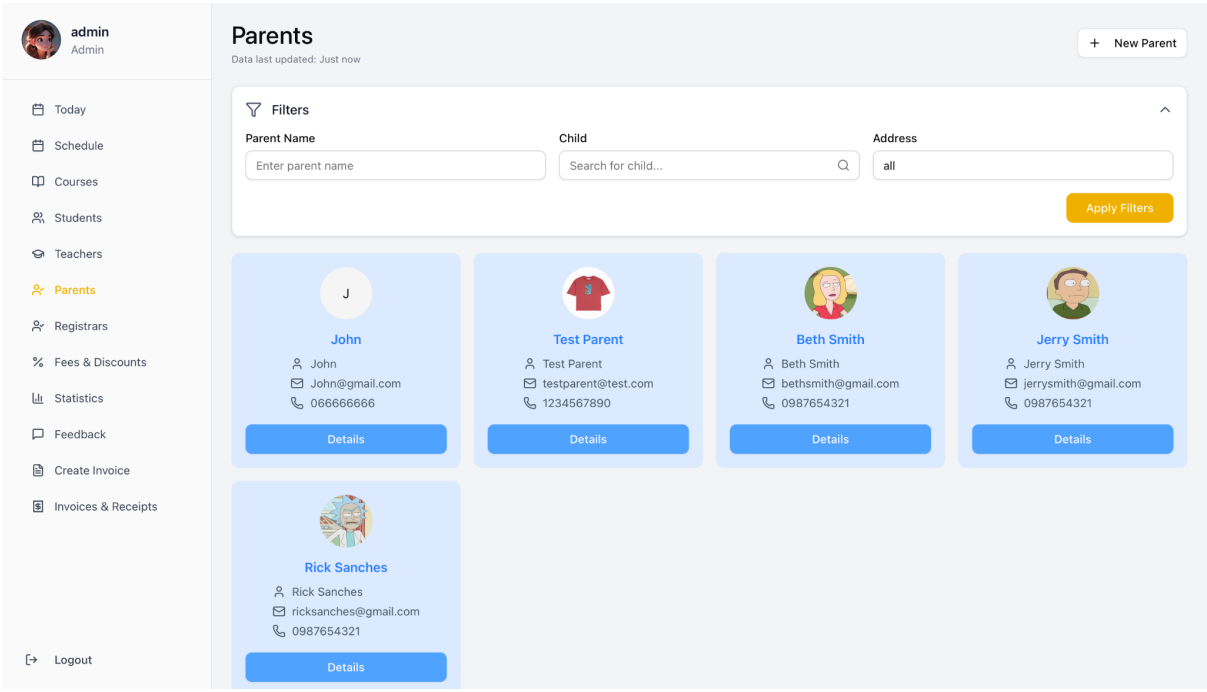
Modified & Approve feedback



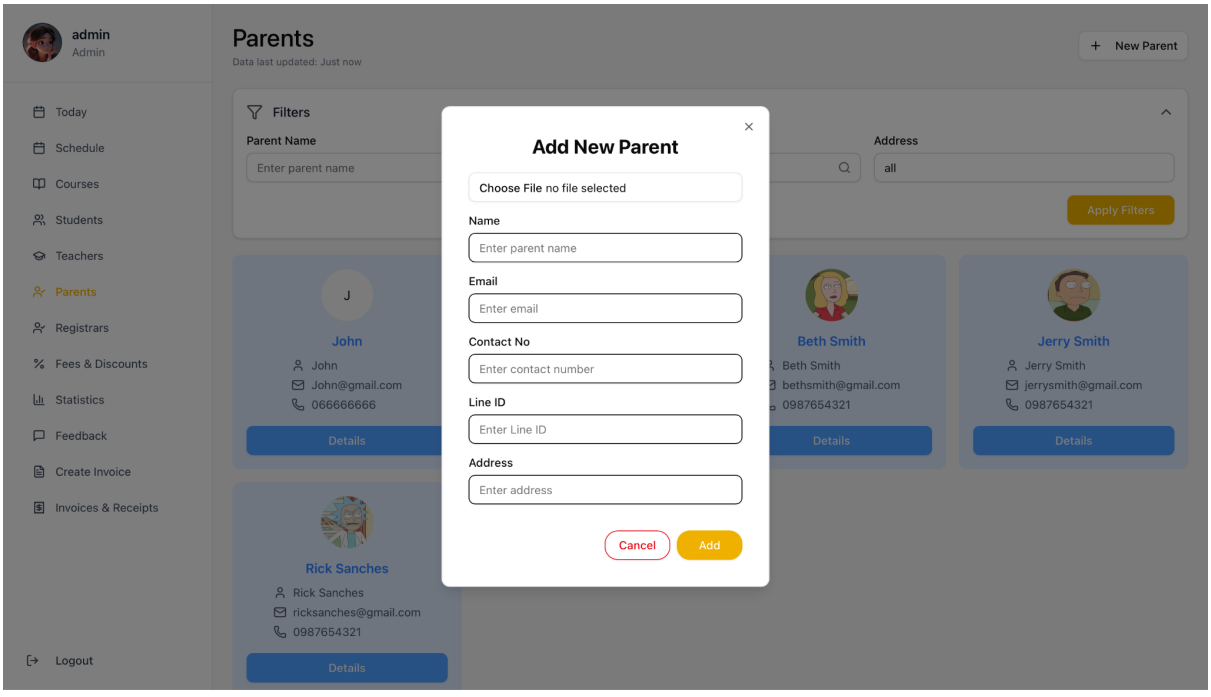
Parents Page

On the **Parents** page, the manager can view a list of existing parents. New parents can be added by entering their details.

Parent Page with a list of parents

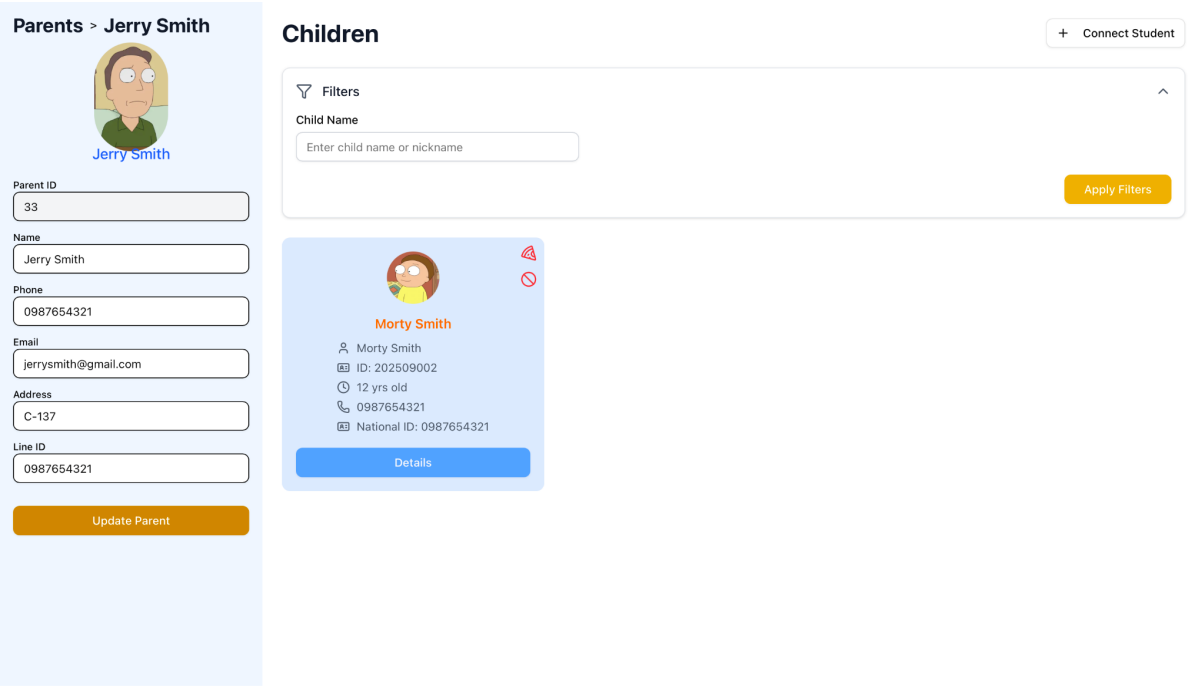


Creating new parent

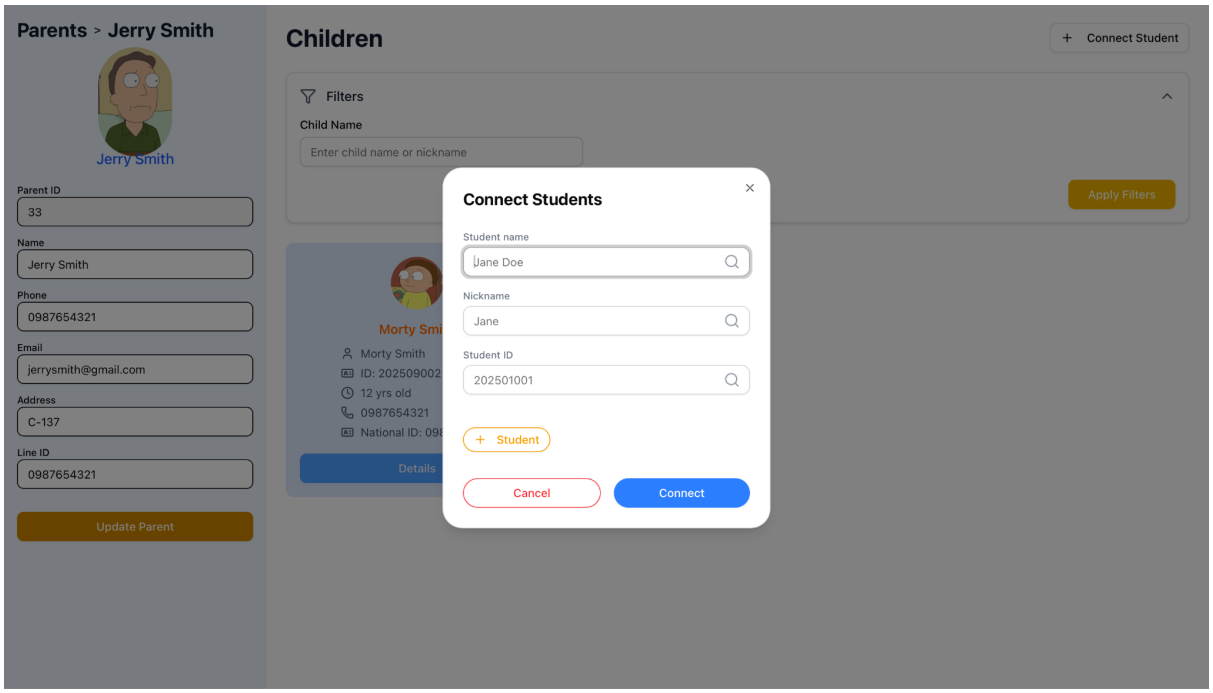


In the **Parent Details** view, the user can find the parent's personal information along with a list of their children. Additional children can be linked by clicking the **Connect Student** button.

Parent details screen



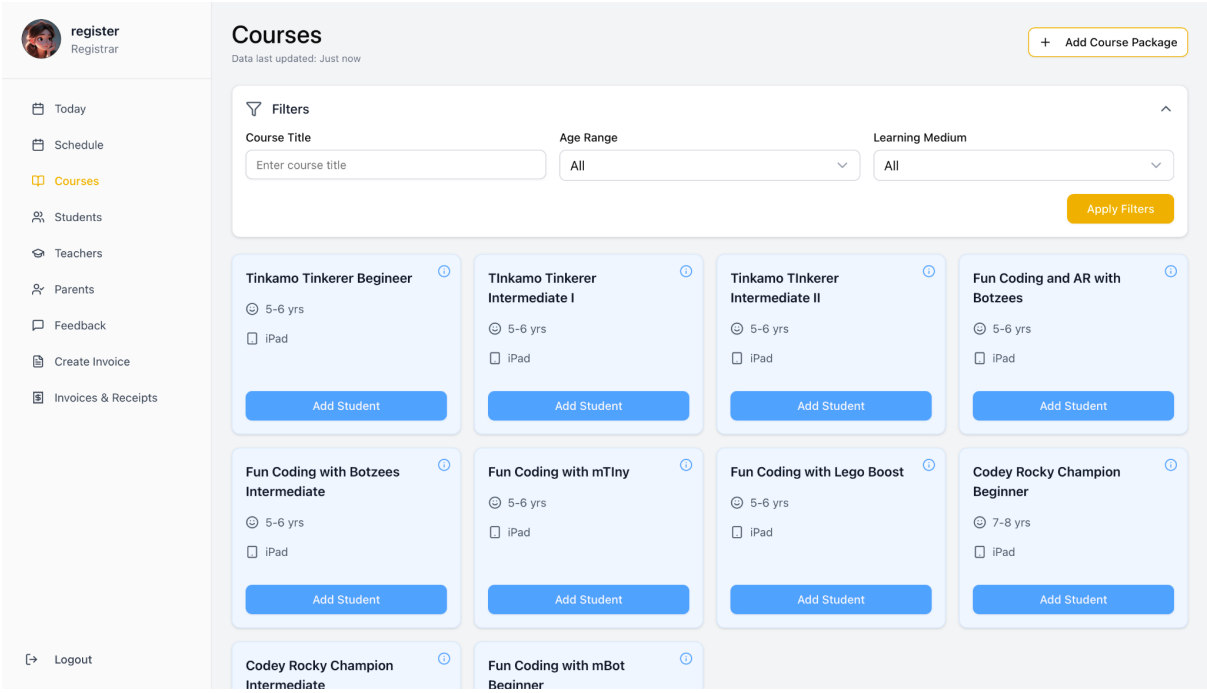
Connecting more students to parent



Registrar UI

Registrars can only access features specified in the Access Control List. Since all permitted features—except the Coursespage—are the same as those available to managers, the registrar interface largely mirrors the manager UI. However, on the Courses page, registrars are not allowed to create new courses; therefore, the New Course button is not visible to them.

Courses Page in Registrar POV



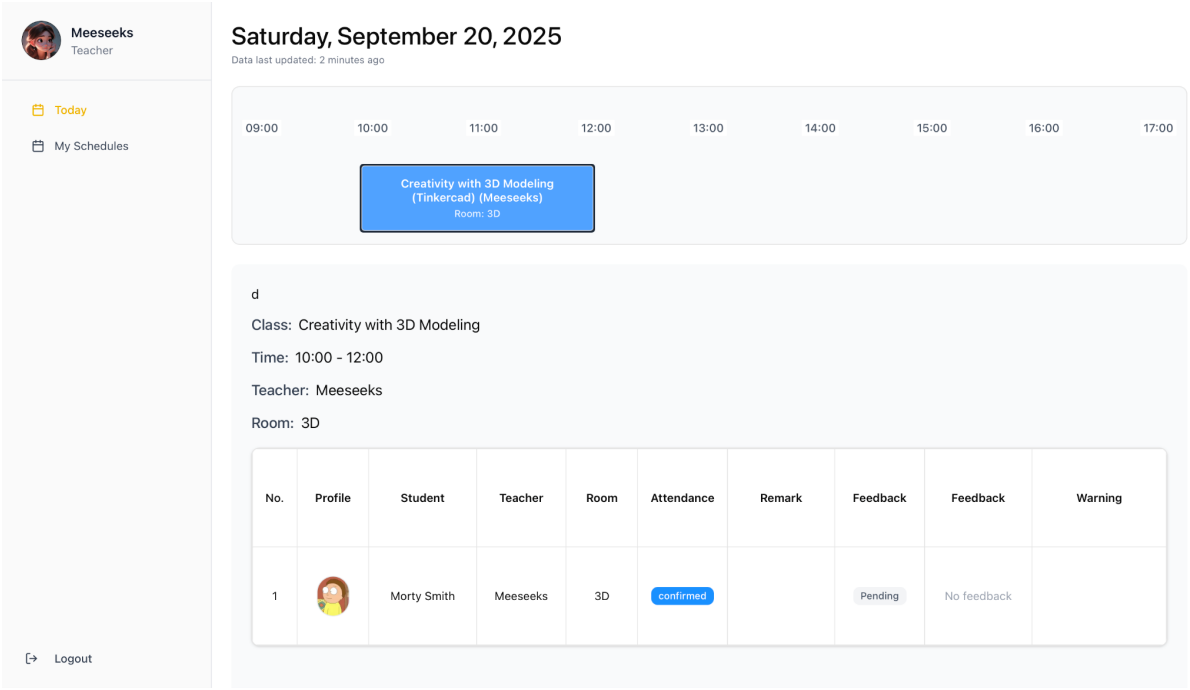
Teacher UI

Teachers can only access features specified in the Access Control List.

In the Today feature, teachers can view the classes scheduled for the day along with the list of students attending each class. By double-clicking a student's row,

teachers can update attendance status and add feedback. Additionally, teachers can access their own schedules, similar to the functionality available in the manager UI.

Web Application in Teacher POV



Conclusion

In conclusion, this software testing project for the Kiddee Lab Learning Management System has systematically evaluated the system's core functionalities to ensure they meet the specified requirements and user expectations. Through a structured and comprehensive testing process, we have validated the system's reliability, functionality, and user experience.

We began by clearly defining the requirements and implementing the corresponding code. This was followed by unit testing to verify the accuracy of individual components. System control flow analysis was performed to understand the program logic, supported by control flow testing using statement coverage to ensure all code paths were exercised.

To further ensure data correctness, we conducted data flow testing by identifying Def(), c-use(), and p-use() sets, helping us analyze variable usage and detect potential issues in data handling. Domain testing was applied to assess system behavior across valid and invalid input ranges.

We then carried out integration testing to confirm that the different modules of the Kiddee Lab LMS interact correctly. A coverage matrix was created to demonstrate that all listed requirements were addressed through our test plan.

Additionally, we conducted User Acceptance Testing (UAT) based on different user roles, validating that the system meets practical usage needs. We also evaluated the UI and UX flow to ensure the platform is intuitive, user-friendly, and efficient for all users, including registrars and teachers.

This project has emphasized the importance of structured and well-documented software testing, ensuring that Kiddee Lab LMS is both functional and ready for real-world application.